Contents

第二册《从信息论做科研》 **4**
本册目录 . . . 4
1. 数学工具/预备知识 . . . 5
2. 完整推导：大模型作为信息变换器 . . . 6
3. 直觉理解 . . . 8
4. 常见误区 . . . 8
5. 代码实现 . . . 8
6. 小结 . . . 10
7. 思考题 . . . 10
8. 延伸阅读/进一步思考 . . . 11
1. 数学工具/预备知识 . . . 11
2. 完整推导 . . . 12
3. 直觉理解 . . . 15
4. 常见误区 . . . 15
5. 代码实现 . . . 15
6. 小结 . . . 17
7. 思考题 . . . 17
8. 延伸阅读/进一步思考 . . . 18
1. 数学工具/预备知识 . . . 18
2. 完整推导 . . . 19
3. 直觉理解 . . . 22
4. 常见误区 . . . 22
5. 代码实现 . . . 22
6. 小结 . . . 25
7. 思考题 . . . 25
8. 延伸阅读/进一步思考 . . . 25
1. 数学工具/预备知识 . . . 26
2. 完整推导 . . . 26
3. 直觉理解 . . . 29
4. 常见误区 . . . 29
5. 代码实现 . . . 30
6. 小结 . . . 31
7. 思考题 . . . 32
8. 延伸阅读/进一步思考 . . . 32
1. 数学工具/预备知识 . . . 32
2. 完整推导 . . . 33
3. 直觉理解 . . . 35
4. 常见误区 . . . 35
5. 代码实现 . . . 36
6. 小结 . . . 38
7. 思考题 . . . 38
8. 延伸阅读/进一步思考 . . . 38
1. 数学工具/预备知识 . . . 38
2. 完整推导 . . . 39
3. 直觉理解 . . . 41
4. 常见误区 . . . 41
5. 代码实现 . . . 41
6. 小结 . . . 43
7. 思考题 . . . 43
8. 延伸阅读/进一步思考 . . . 44
1. 数学工具/预备知识 . . . 44
2. 完整推导 . . . 44
3. 直觉理解 . . . 46

4. 常见误区 47
5. 代码实现 47
6. 小结 48
7. 思考题 49
8. 延伸阅读/进一步思考 49
1. 数学工具/预备知识 49
2. 完整推导 50
3. 直觉理解 52
4. 常见误区 52
5. 代码实现 52
6. 小结 54
7. 思考题 54
8. 延伸阅读/进一步思考 54
1. 数学工具/预备知识 54
2. 完整推导 55
3. 直觉理解 57
4. 常见误区 57
5. 代码实现 57
6. 小结 59
7. 思考题 59
8. 延伸阅读/进一步思考 60
1. 数学工具/预备知识 60
2. 完整推导 60
3. 直觉理解 62
4. 常见误区 62
5. 代码实现 62
6. 小结 64
7. 思考题 65
8. 延伸阅读/进一步思考 65
1. 数学工具/预备知识 65
2. 完整推导 66
3. 直觉理解 67
4. 常见误区 67
5. 代码实现 68
6. 小结 69
7. 思考题 69
8. 延伸阅读/进一步思考 70
1. 数学工具/预备知识 70
2. 完整推导 70
3. 直觉理解 72
4. 常见误区 73
5. 代码实现 73
6. 小结 74
7. 思考题 75
8. 延伸阅读/进一步思考 75
1. 数学工具/预备知识 75
2. 完整推导 76
3. 直觉理解 78
4. 常见误区 78
5. 代码实现 78
6. 小结 79
7. 思考题 80
8. 延伸阅读/进一步思考 80
1. 数学工具/预备知识 80
2. 完整推导 81

3. 直觉理解 . . . 82
4. 常见误区 . . . 83
5. 代码实现 . . . 83
6. 小结 . . . 84
7. 思考题 . . . 85
8. 延伸阅读/进一步思考 . . . 85
1. 数学工具/预备知识 . . . 85
2. 完整推导 . . . 86
3. 直觉理解 . . . 88
4. 常见误区 . . . 88
5. 代码实现 . . . 88
6. 小结 . . . 89
7. 思考题 . . . 90
8. 延伸阅读/进一步思考 . . . 90
1. 数学工具/预备知识 . . . 90
2. 完整推导:“压缩即智能”的逻辑链 . . . 91
3. 完整推导：数据规模 vs 数据质量 . . . 91
4. 直觉理解 . . . 93
5. 常见误区 . . . 93
6. 代码实现 . . . 93
7. 小结 . . . 94
8. 思考题 . . . 94
9. 延伸阅读/进一步思考 . . . 95
1. 数学工具/预备知识 . . . 95
2. 完整推导 . . . 95
3. 直觉理解 . . . 98
4. 常见误区 . . . 98
5. 代码实现 . . . 98
6. 小结 . . . 99
7. 思考题 . . . 99
8. 延伸阅读/进一步思考 . . . 100
1. 数学工具/预备知识 . . . 100
2. 完整推导 . . . 101
3. 直觉理解 . . . 103
4. 常见误区 . . . 103
5. 代码实现 . . . 103
6. 小结 . . . 104
7. 思考题 . . . 104
8. 延伸阅读/进一步思考 . . . 105
1. 数学工具/预备知识 . . . 105
2. 完整推导 . . . 105
3. 直觉理解 . . . 107
4. 常见误区 . . . 107
5. 代码实现 . . . 107
6. 小结 . . . 109
7. 思考题 . . . 109
8. 延伸阅读/进一步思考 . . . 109
1. 数学工具/预备知识 . . . 110
2. 完整推导 . . . 110
3. 直觉理解 . . . 112
4. 常见误区 . . . 112
5. 代码实现 . . . 112
6. 小结 . . . 114
7. 思考题 . . . 114
8. 延伸阅读/进一步思考 . . . 114

1. 数学工具/预备知识 . . . 114
2. 完整推导：读论文的三步法 . . . 115
3. 直觉理解 . . . 116
4. 信息论框架的局限性 . . . 116
5. 典型论文的信息论解读 . . . 117
6. 怎么用这个框架快速读论文? . . . 118
7. 小结 . . . 118
8. 思考题 . . . 119
9. 延伸阅读/进一步思考 . . . 119
1. 数学工具/预备知识 . . . 119
2. 完整推导：信息流缺口的四种类型 . . . 119
3. 实际案例分析 . . . 120
5. 找创新点的常见陷阱 . . . 121
6. 怎么判断是真问题还是伪需求? . . . 121
4. 直觉理解 . . . 122
5. 常见误区 . . . 122
6. 小结 . . . 123
7. 思考题 . . . 123
8. 延伸阅读/进一步思考 . . . 123
1. 数学工具/预备知识 . . . 123
2. 完整推导 . . . 124
3. 直觉理解 . . . 125
4. 常见误区 . . . 125
5. 代码实现 . . . 125
6. 小结 . . . 127
7. 思考题 . . . 128
8. 延伸阅读/进一步思考 . . . 128
1. 数学工具/预备知识 . . . 128
2. 完整推导：论文的信息论结构 . . . 129
3. 论文各部分的信息论优化 . . . 130
4. 论文写作的信息论原则 . . . 131
4. 直觉理解 . . . 131
5. 常见误区 . . . 132
6. 小结 . . . 132
7. 思考题 . . . 132
8. 延伸阅读/进一步思考 . . . 132

第二册《从信息论做科研》

用熵、互信息、KL、信道容量、率失真、MDL、IB 等信息论工具，统一审视模型训练与研究方法论。

共 24 讲·合订版

本册目录

1. 第 1 讲 导论：从香农三大定理到大模型时代
2. 第 2 讲 联合熵、条件熵与链式法则
3. 第 3 讲 互信息：变量之间共享多少信息
4. 第 4 讲 KL 散度与 f-散度家族
5. 第 5 讲 信源编码定理
6. 第 6 讲 信道容量与香农公式
7. 第 7 讲 率失真理论：有损压缩的极限
8. 第 8 讲 最小描述长度 MDL 与过拟合的信息论解释
9. 第 9 讲 交叉熵与对数似然的信息论意义

10. 第 10 讲 VAE 的信息论本质：ELBO 推导
11. 第 11 讲 最大熵原理
12. 第 12 讲 归一化流的信息论本质
13. 第 13 讲 信息瓶颈：深度网络在压缩什么
14. 第 14 讲 互信息估计与对比学习：InfoNCE 推导
15. 第 15 讲 Scaling Law：幂律从何而来
16. 第 16 讲 压缩即智能与数据价值
17. 第 17 讲 RLHF 与 DPO：KL 正则化完整推导
18. 第 18 讲 In-Context Learning 的信息增益视角
19. 第 19 讲 课程学习：从低熵到高熵的信息排布
20. 第 20 讲 注意力熵诊断
21. 第 21 讲 读论文的信息论框架
22. 第 22 讲 找创新点方法论
23. 第 23 讲 实验诊断工具箱：互信息估计与训练动力学观测
24. 第 24 讲 论文写作的信息论范式

第 1 讲 导论：从香农三大定理到大模型时代

> **本讲问题**：为什么一个 1948 年发表、原本为通信工程设计的数学框架，能在大语言模型时代重新成为理解”神经网络到底在学什么”的统一语言？当我们说”训练一个模型”时，信息论视角下到底发生了什么？

1. 数学工具/预备知识

本讲不引入新的严格定义，但需要把贯穿全册的数学工具先回顾一遍。以下每一项都是后面 23 讲会反复用到的基础，不跳过任何一个。

1.1 概率与期望的语言

随机变量 X 是一个取值不确定的量。它取某个具体值 x 的概率记作 $p(x) = P(X = x)$。离散随机变量的概率分布满足两个条件：

- 归一化：$\sum_{x \in \mathcal{X}} p(x) = 1$，所有可能取值的概率加起来等于 1。
- 非负：$p(x) \geq 0$ 对所有 x 成立。

期望是随机变量在整个分布上的”加权平均”。对函数 $g(X)$，其期望定义为：

$$\mathbb{E}_{p(x)}[g(X)] = \sum_{x} p(x)\, g(x)$$

期望的线性性质：$\mathbb{E}[ag(X) + bh(X)] = a\,\mathbb{E}[g(X)] + b\,\mathbb{E}[h(X)]$。这条性质后面推导链式法则时会反复用到。

1.2 对数的性质

本讲统一使用自然对数 log（底数为 e），单位是 nat。后面讲到编码定理时会换成以 2 为底的 $\log_2$，单位是 bit。两者换算：$1 \text{ nat} = \log_2 e \approx 1.443 \text{ bit}$。

需要记住的对数运算：

- $\log(ab) = \log a + \log b$：乘积变加法。这是熵推导中”把联合分布拆成边缘乘条件”的关键。
- $\log(a/b) = \log a - \log b$：比值变差。KL 散度的定义就是从这里来的。
- $\log(1) = 0$。归一化分布的对数概率之和经常出现这个项。
- $\log t$ 是凹函数（二阶导数 $-1/t^2 < 0$）。Jensen 不等式对凹函数给出 $\mathbb{E}[\log T] \leq \log \mathbb{E}[T]$，这是证明 KL 散度非负的核心工具（第 4 讲详细展开）。

1.3 三个直觉性概念

在进入严格定义之前，先建立三个贯穿全册的直觉：

- **信息**：对不确定性的消除。一个事件发生的概率越小，它带来的信息越多。极端情况：必然事件（概率 1）发生了，你什么也没学到，信息量为 0；小概率事件（概率接近 0）发生了，你非常惊讶，信息量极大。
- **信源**：产生随机数据的过程 $X \sim p(x)$。可以理解为一个”不断吐出随机符号的机器”。
- **信道**：把输入 X 变换为输出 Y 的条件分布 $p(y|x)$。可以理解为”有噪声的管道”——你输入 X，输出 Y 是 X 加了噪声之后的版本。

1.4 香农三大定理的骨架

香农 1948 年的工作《通信的数学理论》建立了三条核心定理，它们共同构成了信息论的骨架：

1. **信源编码定理**：无损压缩的极限由熵 $H(X)$ 决定。不管你用什么编码方案，平均码长不可能低于熵。
2. **信道编码定理**：可靠传输的极限由互信息 $I(X;Y)$（容量 C）决定。只要传输速率低于容量，就存在一种编码让错误概率任意小。
3. **率失真定理**：有损压缩在给定失真 D 下的极限由率失真函数 $R(D)$ 决定。

这三条定理分别回答”能压多小”“能传多快”“有损时能压多小”。它们不是工程技巧，而是关于”信息”这一物理量的**不可能定理**——就像热力学第二定律禁止第二类永动机，信息论禁止任何人突破熵给出的界限。

2. 完整推导：大模型作为信息变换器

2.1 数据、模型、训练的信息论翻译

考虑一个监督学习任务：给定训练集 $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}$，学习一个参数为 θ 的模型 $p_\theta(y|x)$。信息论视角下：

- **数据 = 信息载体**。样本 (x_i, y_i) 是从真实分布 $p_{\text{data}}(x, y)$ 中采样得到的随机变量。数据的信息量由联合熵 $H(X, Y)$ 刻画——也就是”同时知道 X 和 Y 的全部不确定性”。
- **模型 = 信息变换器**。$p_\theta(y|x)$ 是一个条件分布，它把输入 X 携带的信息变换为关于 Y 的预测。训练好的模型应当保留与任务相关的信息、丢弃无关噪声。
- **训练 = 信息压缩与提炼**。模型参数 θ 只有有限位数，它不可能记住全部 n 个样本。训练过程本质上是在参数空间 Θ 中寻找一个”压缩表示”，使得 p_θ 在保留任务相关信息的同时尽量紧凑。

2.2 为什么交叉熵损失天然出现

训练语言模型时，我们最大化似然：

$$\max_{\theta} \frac{1}{n} \sum_{i=1}^{n} \log p_\theta(x_i)$$

其中 x_i 是文本 token 序列。这个目标等价于最小化经验分布 $\hat{p}_{\text{data}}(x)$ 与模型分布 $p_\theta(x)$ 之间的 KL 散度。下面一步步推导，不跳步。

第一步：写出经验分布的定义。

经验分布 $\hat{p}_{\text{data}}(x)$ 是把训练数据中的每个样本当作一个点质量（delta 函数）：

$$\hat{p}_{\text{data}}(x) = \frac{1}{n} \sum_{i=1}^{n} \delta(x - x_i)$$

其中 $\delta(x - x_i)$ 在 $x = x_i$ 处为 1，其他位置为 0。它的作用是把”数据集中出现了什么”直接变成一个概率分布。

第二步：写出 KL 散度的定义。

$$D_{\text{KL}}(\hat{p}_{\text{data}} \,\|\, p_\theta) = \sum_{x} \hat{p}_{\text{data}}(x) \log \frac{\hat{p}_{\text{data}}(x)}{p_\theta(x)}$$

第三步：把对数拆开。

利用 $\log(a/b) = \log a - \log b$：

$$D_{\mathrm{KL}}(\hat{p}\|p_\theta) = \sum_x \hat{p}_{\text{data}}(x)\log\hat{p}_{\text{data}}(x) - \sum_x \hat{p}_{\text{data}}(x)\log p_\theta(x)$$

第一项是经验分布自身的熵 $H(\hat{p}_{\text{data}})$：

$$\underbrace{\sum_x \hat{p}_{\text{data}}(x)\log\frac{1}{\hat{p}_{\text{data}}(x)}}_{\text{常数}H(\hat{p}_{\text{data}})} + \underbrace{\sum_x \hat{p}_{\text{data}}(x)\log\frac{1}{p_\theta(x)}}_{\text{交叉熵损失}}$$

第四步：注意第一项与 θ 无关。

$H(\hat{p}_{\text{data}})$ 只由训练数据决定，跟模型参数 θ 没有任何关系。因此在优化 θ 时，第一项是常数，梯度为 0。

第五步：因此最小化 KL 等价于最小化交叉熵。

$$\mathcal{L}(\theta) = -\sum_x \hat{p}_{\text{data}}(x)\log p_\theta(x) = -\frac{1}{n}\sum_{i=1}^{n}\log p_\theta(x_i)$$

这就是为什么”最大似然训练”和”最小交叉熵损失”在数学上是同一件事——而它的信息论本质是：**让模型分布 p_θ 在 KL 意义下尽可能接近真实数据分布**。

几何意义：KL 散度可以想象成两个分布之间的”非对称距离”。$D_{\mathrm{KL}}(\hat{p}\|p_\theta)$ 度量的是”用模型分布 p_θ 来编码真实数据 $\hat{p}$ 时，比最优编码（即直接用 $\hat{p}$ 自己编码）多出来的比特数”。训练就是不断缩短这个多余的距离。

2.3 训练作为信息瓶颈

深度网络逐层变换 $X \to Z_1 \to Z_2 \to \cdots \to Y$。信息瓶颈理论（第 13 讲）指出，一个好的表征 Z 应当同时满足：

1. 尽量保留关于 Y 的信息：$I(Z;Y)$ 大——表征要能预测出标签。
2. 尽量压缩关于 X 的信息：$I(Z;X)$ 小——表征不要把输入的冗余细节都背下来。

这两个目标是矛盾的——你不可能既完全保留 X 又完全压缩 X。最优解在这两者间取折中，由一个拉格朗日乘子 β 控制：

$$\min_\theta \; I(Z;X) - \beta\, I(Z;Y)$$

为什么这是一个拉格朗日问题：原始约束是”$I(Z;Y) \geq I_0$（任务信息不能低于某个阈值），最小化 $I(Z;X)$“。拉格朗日乘子法把约束吸收进目标函数，β 就是约束的对偶变量。当 β 大时，任务信息被看得很重，Z 会保留很多 X 的信息；当 β 小时，压缩被看得很重，Z 会丢掉很多 X 的信息。

大模型训练中，“过拟合”对应于 $\beta \to 0$（Z 几乎完整拷贝 X，把训练数据的噪声也记住了），“欠拟合”对应于 $\beta \to \infty$（Z 丢弃了太多信息，连任务信号也丢了）。这一视角将在第 13 讲严格展开。

2.4 为什么信息论是”统一坐标系”

机器学习中有很多看似无关的概念：交叉熵损失、KL 正则、VAE 的 ELBO、对比学习的 InfoNCE、RLHF 的 PPO、注意力机制……信息论的价值在于它提供了一套**不依赖具体模型结构**的语言：

ML 概念	信息论对应	本讲索引
交叉熵损失	最小化 $D_{\mathrm{KL}}(p_{\text{data}}\|p_\theta)$	§2.2
VAE 的 ELBO	数据处理不等式 + 变分逼近	第 10 讲

ML 概念	信息论对应	本讲索引
InfoNCE 对比损失	最大化互信息下界	第 14 讲
RLHF 的 KL 惩罚	约束策略偏离参考分布	第 17 讲
过拟合	模型保留了数据中的噪声信息	第 8 讲
Scaling Law	幂律来自 token 分布的 Zipf 结构	第 15 讲

当你用这套语言重新审视一个新模型时，首先问三个问题：它在压缩什么？它保留了什么？它丢弃了什么？这三个问题对应着 $I(Z;X)$、$I(Z;Y)$、以及被丢弃的噪声。

3. 直觉理解

直觉：想象你在整理一个巨大的照片库。最原始的照片库就是”数据分布”。你要做一个相册（模型），让人翻开后能快速认出”这张照片拍的是猫还是狗”。好的相册会保留”猫耳朵、狗尾巴”这类判别性信息，丢掉”照片是上午拍的还是下午拍的”这类无关细节。

信息论就是那把尺子：它测量你保留了多少有用信息、丢掉了多少无用信息、以及这两者之间的权衡。大模型不是一个”记忆机器”，而是一个”信息提炼机器”——它从海量文本中提炼出关于语言结构、世界知识、推理模式的压缩表示。

再打个比方：训练一个模型就像蒸馏。原材料（数据）里有你想要的精华（任务信号），也有水和杂质（噪声和冗余）。蒸馏的目标是把精华提出来，把水蒸掉。信息瓶颈就是那个蒸馏器——β 控制火候，火太大（β 太小）精华还没蒸出来水先烧干了，火太小（β 太大）水一直烧不干。

4. 常见误区

误区一：把信息论等同于”熵公式”或”交叉熵损失”。信息论远不止一个损失函数——它是一套关于**不确定性、相关性、极限**的公理化体系。交叉熵只是它在 ML 中最常见的入口。真正的力量在于：当你不知道该怎么设计损失函数时，信息论会告诉你”理论上的最优目标长什么样”，剩下的只是工程上如何逼近它。

误区二：把”信息量大”理解为”数据量大”。一个 100GB 的视频可能信息量极低（比如一段白屏录像，每个像素都是相同的值），一个 1KB 的加密消息信息量可能极高（加密前是完全随机的，没有任何冗余可以压缩）。文件大小和信息量是两回事——前者是物理存储，后者是统计结构。

误区三：认为”模型越大越好”。从信息瓶颈角度看，模型容量大意味着 $I(Z;X)$ 可以做得很大（能记住更多细节），但如果 $I(Z;Y)$ 没有相应提升，多出来的容量只是在记忆噪声。参数量不是越大越好，关键是 $I(Z;Y)$ 有没有跟着涨。

5. 代码实现

下面用一个最小例子展示：为什么”最大似然”等价于”最小化交叉熵”，以及 KL 散度如何分解为熵加交叉熵。代码里每一步都加了注释，解释为什么这样写。

```
import numpy as np
import matplotlib.pyplot as plt

# 真实数据分布（一个简单的 4 类分布）
# 为什么用 4 类？因为太小（2 类）看不出分布差异，太大（100 类）画图不清晰。
p_data = np.array([0.4, 0.3, 0.2, 0.1])

# 模型分布：尝试一组候选分布，找 KL 最小的
# 这四个候选分别对应四种典型情况：
#   候选 1：均匀分布（完全不了解数据）
#   候选 2：完全匹配（理想情况，KL=0）
#   候选 3：反转分布（完全搞反了）
#   候选 4：轻微偏差（训练初期或正则化不足）
candidates = [
    np.array([0.25, 0.25, 0.25, 0.25]),  # 1: 均匀
```

```
    np.array([0.4, 0.3, 0.2, 0.1]),       # 2: 完全匹配
    np.array([0.1, 0.2, 0.3, 0.4]),       # 3: 反转
    np.array([0.5, 0.2, 0.2, 0.1]),       # 4: 偏差
]

def entropy(p):
    """ 计算离散分布的熵 H(p), 单位 nat。

    为什么要先过滤 p>0?
    因为 0 * log(0) 在数学上定义为 0（极限意义),
    但 numpy 会输出 nan 和 warning。
    这是数值实现的常见坑。
    """
    p = p[p > 0]  # 避免 log(0) = -inf 导致 nan
    return -np.sum(p * np.log(p))

def cross_entropy(p, q):
    """ 计算交叉熵 H(p, q) = sum p log(1/q)。

    交叉熵 = 用 q 编码来自 p 的数据的平均码长。
    如果 p = q, 交叉熵就等于熵 H(p)。
    """
    return -np.sum(p * np.log(q))

def kl_divergence(p, q):
    """KL(p || q) = H(p,q) - H(p)

    这是 KL 散度的分解公式: KL = 交叉熵 - 熵。
    为什么这样分解有用? 因为 H(p) 是常数（与模型无关),
    所以最小化 KL 等价于最小化交叉熵。
    """
    return cross_entropy(p, q) - entropy(p)

print("=" * 60)
print(" 真实分布的熵 H(p_data) =", round(entropy(p_data), 4), "nats")
print("=" * 60)
print(f"{'模型分布':<35} {'H(p,q)':>8} {'KL(p||q)':>10}")
print("-" * 60)
for q in candidates:
    ce = cross_entropy(p_data, q)
    kl = kl_divergence(p_data, q)
    print(f"{str(np.round(q,2)):<35} {ce:>8.4f} {kl:>10.4f}")

# 可视化: 不同模型分布对应的交叉熵损失
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# 左图: 4 个类别的概率对比
x = np.arange(4)
width = 0.2
colors = ['#999999', '#2ecc71', '#e74c3c', '#3498db']
for i, (q, c) in enumerate(zip(candidates, colors)):
    axes[0].bar(x + i*width, q, width, label=f'候选{i+1}', color=c, alpha=0.7)
axes[0].plot(x, p_data, 'ko-', label='真实分布', linewidth=2)
axes[0].set_xticks(x + 1.5*width)
axes[0].set_xticklabels(['类别 1','类别 2','类别 3','类别 4'])
axes[0].set_ylabel('概率')
axes[0].set_title('模型分布 vs 真实分布')
axes[0].legend(fontsize=8)

# 右图: KL 散度对比
```

```
# KL 越小 = 模型越接近真实分布 = 训练损失越低
kls = [kl_divergence(p_data, q) for q in candidates]
axes[1].bar(range(1, 5), kls, color=colors, alpha=0.7)
axes[1].set_xlabel('候选模型')
axes[1].set_ylabel('KL(p_data || p_model)  (nats)')
axes[1].set_title('KL 散度越小 = 模型越接近真实分布')
axes[1].set_xticks(range(1, 5))

plt.tight_layout()
plt.savefig('ch01_kl_demo.png', dpi=100)
print("\n图已保存为 ch01_kl_demo.png")
print(" 观察：候选 2 完全匹配真实分布，KL=0；反转分布 KL 最大。")
```

工程注意事项：上面的代码是离散分布的玩具示例。实际训练神经网络时，$p_\theta(x)$ 是一个高维连续分布（比如语言模型的输出是词表大小的 softmax），KL 散度的计算退化为交叉熵——因为真实分布是经验分布（delta 函数），熵项为 0。这就是为什么深度学习框架里只实现了交叉熵损失，没有 KL 损失。

6. 小结

1. 信息论的三大定理（信源编码、信道容量、率失真）分别刻画了”压缩极限”“传输极限”“有损压缩极限”。它们是关于信息的物理定律，不是工程技巧。
2. 在 ML 语境下，数据是信息载体（$H(X,Y)$），模型是信息变换器（$p_\theta(y|x)$），训练是信息压缩与提炼（在有限参数中寻找最优表征）。
3. 最大似然训练等价于最小化真实分布与模型分布之间的 KL 散度。展开后，熵项是常数，交叉熵项是唯一与模型参数相关的部分。这就是交叉熵损失的信息论起源。
4. 信息瓶颈视角把”训练”理解为在保留任务信息 $I(Z;Y)$ 与压缩输入信息 $I(Z;X)$ 之间的权衡，β 控制这个权衡的松紧。
5. 信息论提供了不依赖具体网络结构的统一语言，让我们能跨模型类型地思考”到底在学什么”——压缩了什么、保留了什么、丢弃了什么。
6. 信息论不仅是数学工具，更是一种思维方式——它让你从”信息的流动”角度看问题。
7. 本课程的目标：用信息论的视角理解机器学习和大模型。
8. 信息论三大定理：信源编码、信道容量、率失真。
9. 为什么 1948 年的理论在今天仍然重要：因为信息的本质没变。
10. 信息论是连接通信、机器学习、统计物理的桥梁。
11. 本课程的 24 讲从基础概念到科研方法，层层递进。

7. 思考题

基础档（巩固概念）

1. 一段 1 小时的高清视频（约 10GB）和一本 100 页的书（约 500KB），哪个信息量更大？为什么”文件大小”不等于”信息量”？
2. 写出 KL 散度 $D_{\mathrm{KL}}(p\|q)$ 的定义式，并说明为什么最小化 $D_{\mathrm{KL}}(\hat{p}_{\mathrm{data}}\|p_\theta)$ 等价于最小化交叉熵损失。
3. 用自己的话解释：”为什么”过拟合”对应于信息瓶颈中 $\beta \to 0$？

进阶档（推导与分析）

4. 如果一个模型在训练集上达到 100% 准确率但测试集准确率很低，用信息瓶颈的语言解释：这个模型的 $I(Z;X)$ 和 $I(Z;Y)$ 分别发生了什么变化？是 $I(Z;X)$ 太大还是 $I(Z;Y)$ 太小？
5. 香农公式 $C = W\log_2(1+\mathrm{SNR})$ 中，如果带宽 W 趋于无穷，容量 C 会趋于无穷吗？（提示：噪声功率 N_0W 也随 W 增长，SNR = $P/(N_0W)$ 会下降。求 $\lim_{W\to\infty} W\log_2(1+P/(N_0W))$。）
6. 经验分布 $\hat{p}(x) = \frac{1}{n}\sum_i \delta(x-x_i)$ 的熵是多少？为什么在最小化 KL 时这个熵项不影响优化方向？

开放档（联系研究）

7. 大语言模型（如 GPT）的上下文窗口（context window）可以类比为信道容量。上下文窗口越长，模型能”传输”多少关于输入文本的信息？如果上下文窗口无限大，$I(Z;X)$ 会怎么变化？这与”长上下文模型”的泛化问题有什么联系？
8. 假设你要设计一个新的损失函数来训练某个模型。信息论告诉你”理论最优目标是最小化 $D_{\mathrm{KL}}(p_{\text{data}}\|p_\theta)$“。在什么情况下，这个理论最优目标在工程上不可行？你会如何近似它？

8. 延伸阅读/进一步思考

- **Shannon (1948)**，*A Mathematical Theory of Communication*。信息论的开山之作。第 I 节”Introduction”和第 II 节”Discrete Noiseless Systems”是理解本讲三大定理的原始文献。原文并不长，值得直接读。
- **Tishby, Pereira & Bialek (1999)**，*The Information Bottleneck Method*。信息瓶颈原理的原始论文。本讲 §2.3 提到的 $\min I(X;Z)-\beta I(Z;Y)$ 直接来自这篇文章。
- **MacKay (2003)**，*Information Theory, Inference, and Learning Algorithms*。第 1-2 章对熵、KL 散度有非常清晰的公理化推导，免费在线。
- **Goldstein et al. (2024)**，*A Roadmap to Interpretable Machine Learning via Information Theory*。综述信息论在深度学习可解释性中的应用，适合了解大模型时代的新进展。

第 2 讲 联合熵、条件熵与链式法则

> **本讲问题**：当两个随机变量 X 和 Y 一起出现时，它们的总不确定性是多少？如果已经知道 X，Y 还剩多少不确定性？链式法则为什么说”信息可以逐次累加”？为什么”知道得越多，不确定性越少”是一条数学定理而不是直觉？

1. 数学工具/预备知识

回顾第 1 讲建立的直觉：信息是对不确定性的消除。一个概率为 p 的事件发生后，我们获得的信息量为 $-\log p$。本节从这个直觉出发，公理化地定义自信息、熵、联合熵与条件熵，并把每一步的”为什么”讲透。

1.1 自信息：为什么是 $-\log p$？

自信息：随机变量 X 取某个具体值 x 时的信息量：

$$I(x)=-\log p(x)$$

为什么函数形式是 $-\log p$ 而不是别的？它满足三条自然性质：

1. **可加性**：两个独立事件 A 和 B 同时发生的信息量等于各自信息量之和。因为 $p(A\cap B)=p(A)p(B)$，所以 $I(A\cap B)=-\log(p(A)p(B))=-\log p(A)-\log p(B)=I(A)+I(B)$。只有对数函数满足”乘积变加法”。
2. **非负性**：$0<p(x)\le 1$，所以 $-\log p(x)\ge 0$。不可能事件发生的信息量是无穷大，必然事件发生的信息量是 0。
3. **单调性**：概率越小的事件，信息量越大。$p(x)$ 越小，$-\log p(x)$ 越大。

当 log 以 2 为底，单位是 bit；以 e 为底，单位是 nat。本讲统一使用 nat。

1.2 熵：自信息的期望

熵：自信息在整个分布上的期望：

$$H(X)=\mathbb{E}_{p(x)}[-\log p(x)]=-\sum_x p(x)\log p(x)$$

熵衡量的是 X 的平均不确定性。它不是某个具体事件的信息量，而是把所有可能事件的信息量按概率加权平均。

为什么要取期望：因为随机变量 X 取什么值是不确定的，我们关心的是”平均来看”这个随机变量有多不确定。期望是概率论中度量”平均表现”的标准工具。

熵的取值范围：对 m 个等可能结果的分布，$H = \log m$（最大熵）。对确定性分布（某个 $p = 1$，其余为 0），$H = 0$（最小熵）。熵越大，分布越”均匀”、越”不确定”。

1.3 联合分布与边缘分布

两个随机变量 X 和 Y 的联合分布 $p(x, y)$ 描述它们同时取值的概率。从联合分布可以推出边缘分布：

$$p(x) = \sum_y p(x, y), \qquad p(y) = \sum_x p(x, y)$$

条件分布：在已知 $X = x$ 的条件下，Y 的分布为：

$$p(y|x) = \frac{p(x, y)}{p(x)}$$

这是贝叶斯公式的基础形式。链式法则的全部推导都建立在这三个分布的关系上。

2. 完整推导

2.1 熵的公理化推导：为什么是 $-\sum p \log p$?

为什么熵恰好是 $-\sum p \log p$ 这个形式，而不是 $-\sum p^2$ 或 $\sum p \log(1/p)^2$? 香农给出了三条公理，从这三条公理可以唯一推出熵的形式。

公理 1（连续性）：H 是 $p(x)$ 的连续函数。概率有微小变化时，熵也有微小变化。

公理 2（单调性）：均匀分布 $H(1/n, \ldots, 1/n)$ 随 n 单调递增。等可能的结果越多，不确定性越大。

公理 3（可加性/分支性）：如果一个实验可以分解为两步相继进行，则总熵等于分步熵的加权和。

第三条公理是最关键的。假设一个三分量分布 $(\frac{1}{2}, \frac{1}{3}, \frac{1}{6})$。我们可以把它分解为两步：

- 第一步：以 $\frac{1}{2}$ 的概率选”组 A”，以 $\frac{1}{2}$ 的概率选”组 B”。
- 组 A 只有一个元素（概率 $\frac{1}{2}$），组 B 内部再以 $(\frac{2}{3}, \frac{1}{3})$ 分配（对应原分布的 $\frac{1}{3}$ 和 $\frac{1}{6}$）。

可加性要求：

$$H\left(\frac{1}{2}, \frac{1}{3}, \frac{1}{6}\right) = H\left(\frac{1}{2}, \frac{1}{2}\right) + \frac{1}{2} H\left(\frac{2}{3}, \frac{1}{3}\right)$$

为什么右边第二项带权重 $\frac{1}{2}$：因为组 B 只有 $\frac{1}{2}$ 的概率被选中，组 B 内部的熵只有一半时间被体验到。加权平均的权重就是各组的边缘概率。

现在验证 $-\sum p \log p$ 满足这条公理。

左边（直接计算三分量的熵）：

$$H\left(\frac{1}{2}, \frac{1}{3}, \frac{1}{6}\right) = -\frac{1}{2}\log\frac{1}{2} - \frac{1}{3}\log\frac{1}{3} - \frac{1}{6}\log\frac{1}{6}$$

右边（分步计算）：

第一步的熵：$H(\frac{1}{2}, \frac{1}{2}) = -\frac{1}{2}\log\frac{1}{2} - \frac{1}{2}\log\frac{1}{2} = -\log\frac{1}{2} = \log 2$

第二步的熵（组 B 内部）：$H(\frac{2}{3}, \frac{1}{3}) = -\frac{2}{3}\log\frac{2}{3} - \frac{1}{3}\log\frac{1}{3}$

加上权重：$\frac{1}{2}H(\frac{2}{3}, \frac{1}{3}) = -\frac{1}{3}\log\frac{2}{3} - \frac{1}{6}\log\frac{1}{3}$

右边合计：

$$\log 2 - \frac{1}{3}\log\frac{2}{3} - \frac{1}{6}\log\frac{1}{3}$$

数值验证（用自然对数）：

左边 $= 0.5\ln 2 + \frac{1}{3}\ln 3 + \frac{1}{6}\ln 6 = 0.3466 + 0.3662 + 0.2986 = 1.0114$

右边 $= \ln 2 + 0.5\times(\frac{2}{3}\ln\frac{3}{2} + \frac{1}{3}\ln 3) = 0.6931 + 0.5\times(0.2703 + 0.3662) = 0.6931 + 0.3183 = 1.0114$。

香农证明了：满足这三条公理的函数只有 $H = -K\sum p_i \log p_i$（K 为常数，由单位选择决定）。这不是巧合，而是唯一的选择。

2.2 联合熵

两个随机变量 X, Y 的联合熵定义为联合分布 $p(x,y)$ 的熵：

$$H(X,Y) = -\sum_{x,y} p(x,y)\log p(x,y)$$

它衡量的是”同时观察 X 和 Y“这一事件的平均不确定性。

与单变量熵的关系：如果 X 和 Y 完全独立（$p(x,y) = p(x)p(y)$），则：

$$H(X,Y) = -\sum_{x,y} p(x)p(y)\log(p(x)p(y)) = -\sum_{x} p(x)\log p(x) - \sum_{y} p(y)\log p(y) = H(X) + H(Y)$$

独立时，联合熵等于各自熵之和。但如果 X 和 Y 有相关性，联合熵会小于两者之和——因为知道了一个就部分知道了另一个，总不确定性减少了。

2.3 条件熵

如果已经知道 X 的取值为 x，那么 Y 在这个条件下的条件分布是 $p(y|x) = p(x,y)/p(x)$。定义**条件熵**：

$$H(Y|X) = \sum_{x} p(x)\, H(Y|X=x) = \sum_{x} p(x)\left[-\sum_{y} p(y|x)\log p(y|x)\right]$$

展开后：

$$H(Y|X) = -\sum_{x,y} p(x,y)\log p(y|x)$$

为什么要按 $p(x)$ 加权平均：因为 X 取什么值是随机的，我们关心的是”平均来看，知道 X 之后 Y 还剩多少不确定性”。对每个 x 算一个 $H(Y|X=x)$，再按 $p(x)$ 加权，这就是期望的定义。

物理意义：$H(Y|X)$ 衡量”在已经知道 X 的条件下，Y 还剩多少不确定性”。如果 X 和 Y 完全相关（知道 X 就完全知道 Y），则 $H(Y|X) = 0$。如果 X 和 Y 完全无关，则 $H(Y|X) = H(Y)$（知道 X 对预测 Y 毫无帮助）。

2.4 链式法则

联合熵的链式法则：

$$H(X,Y) = H(X) + H(Y|X)$$

证明：从定义出发，一步一步推。

$$H(X,Y) = -\sum_{x,y} p(x,y) \log p(x,y)$$

利用乘法公式 $p(x,y) = p(x)\,p(y|x)$（联合概率 = 边缘概率 × 条件概率）：

$$= -\sum_{x,y} p(x,y) \log[p(x)\,p(y|x)]$$

利用对数性质 $\log(ab) = \log a + \log b$：

$$= -\sum_{x,y} p(x,y) \log p(x) - \sum_{x,y} p(x,y) \log p(y|x)$$

处理第一项：$p(x)$ 只与 x 有关，可以把对 y 的求和提到括号外面。

$$-\sum_{x,y} p(x,y) \log p(x) = -\sum_{x} \left(\sum_{y} p(x,y) \right) \log p(x)$$

括号里的 $\sum_y p(x,y) = p(x)$（边缘分布的定义）：

$$= -\sum_{x} p(x) \log p(x) = H(X)$$

处理第二项：恰好是条件熵的定义式。

$$-\sum_{x,y} p(x,y) \log p(y|x) = H(Y|X)$$

两项合并：$H(X,Y) = H(X) + H(Y|X)$。□

推论：$H(X,Y) = H(Y) + H(X|Y)$（对称性，把 X 和 Y 的角色互换即可）。这两条链式法则同时成立，说明联合熵可以从任意一边”拆开”。

2.5 三变量链式法则

推广到三个变量：

$$H(X,Y,Z) = H(X) + H(Y|X) + H(Z|X,Y)$$

证明：把 (X,Y) 看作一个整体，对它和 Z 使用二元链式法则：

$$H(X,Y,Z) = H(X,Y) + H(Z|X,Y)$$

再对 $H(X,Y)$ 使用二元链式法则 $= H(X) + H(Y|X)$。合并即得。□

一般形式：对 n 个随机变量，

$$H(X_1, X_2, \ldots, X_n) = H(X_1) + H(X_2|X_1) + H(X_3|X_1,X_2) + \cdots + H(X_n|X_1,\ldots,X_{n-1})$$

这就是”信息逐次累加”的严格数学表达。

2.6 熵的次可加性

一个重要推论：

$$H(X,Y) \le H(X) + H(Y)$$

等号当且仅当 X 与 Y 独立时成立。

证明：由链式法则 $H(X,Y) = H(X) + H(Y|X)$。我们需要证明 $H(Y|X) \le H(Y)$。

直观上，知道 X 只会减少或不改变对 Y 的不确定性——这就是”信息不会有害”的信息论版本。严格证明需要用到第 3 讲将引入的互信息非负性：

$$I(X;Y) = H(Y) - H(Y|X) \ge 0 \implies H(Y|X) \le H(Y)$$

互信息非负性由 KL 散度非负性（Gibbs 不等式）保证，第 4 讲会严格证明。此处先记下结论。

几何意义：次可加性说的是”相关性减少总不确定性”。如果 X 和 Y 有关联，知道了 X 就部分知道了 Y，所以 $H(X,Y)$ 比 $H(X) + H(Y)$ 小。两者的差值恰好是互信息 $I(X;Y)$。

3. 直觉理解

直觉：把 $H(X)$ 想象成”翻开 X 这页书之前，你对它有多不确定”。$H(Y|X)$ 是”已经看完 X 这页后，再翻 Y 那页还剩多少惊讶”。

两本书合在一起的总不确定性 $H(X,Y)$，等于先看第一本的惊讶 $H(X)$，加上看完第一本后再看第二本的额外惊讶 $H(Y|X)$。如果两本书内容完全独立（比如一本菜谱一本星图），那 $H(Y|X) = H(Y)$，总不确定性就是两者之和。如果两本书是上下册（内容高度相关），看完上册后下册几乎没有新东西，$H(Y|X)$ 很小。

链式法则说的就是：不管内容怎么相关，“逐次阅读”的总信息永远等于”一次性读完”的总信息。这就像加法交换律——你先看哪本、后看哪本，总信息量不变。

4. 常见误区

误区一：把 $H(Y|X)$ 理解成”X 对 Y 的信息量”。错。$H(Y|X)$ 是条件不确定性，不是信息量。X 对 Y 的信息量是 $I(X;Y) = H(Y) - H(Y|X)$（第 3 讲）。一个是”还剩多少不确定”，一个是”减少了多少不确定”，方向完全相反。

误区二：认为 $H(Y|X) \le H(X,Y)$。这个不等式方向反了。正确关系是 $H(X,Y) = H(X) + H(Y|X) \ge H(Y|X)$，因为 $H(X) \ge 0$。联合熵一定大于或等于任意一个条件熵。

误区三：混淆 $H(Y|X=x)$ 和 $H(Y|X)$。前者是对一个**固定的** x 值计算熵（一个具体的数），后者是对所有 x 的 $H(Y|X=x)$ 按 $p(x)$ 加权平均（一个期望，也是一个数，但含义不同）。比如”已知今天下雨，明天是否下雨的不确定性”是 $H(Y|X=\text{下雨})$，而”不管下不下雨，明天的平均不确定性”是 $H(Y|X)$。

误区四：认为链式法则只对离散分布成立。错。连续分布的微分熵也有链式法则 $h(X,Y) = h(X) + h(Y|X)$，推导完全平行（把求和换成积分）。区别在于连续微分熵可以为负（因为它不是真正的信息量，只是相对熵）。

5. 代码实现

下面用一个具体的二元随机变量例子，数值验证链式法则与次可加性。代码注释解释了每个计算的目的和容易出错的地方。

```
import numpy as np
import matplotlib.pyplot as plt

# 构造一个二元随机变量的联合分布 p(x, y)
```

```python
# X 有 3 个取值，Y 有 2 个取值
# 手动指定一个 3x2 的联合概率矩阵
# 为什么手动指定而不是随机生成?
# 因为我们要验证链式法则，需要一个已知答案的分布。
p_joint = np.array([
    [0.20, 0.10],   # X=0: Y=0 概率 0.20, Y=1 概率 0.10
    [0.15, 0.25],   # X=1
    [0.05, 0.25],   # X=2
])
# 归一化确保总和为 1（防御性编程，防止手算出错）
p_joint = p_joint / p_joint.sum()

# 计算边缘分布
# 行和 = X 的边缘分布（对 y 求和）
# 列和 = Y 的边缘分布（对 x 求和）
p_x = p_joint.sum(axis=1)   # X 的边缘: 每行加起来
p_y = p_joint.sum(axis=0)   # Y 的边缘: 每列加起来

# 计算条件分布 p(y|x) = p(x,y) / p(x)
# 为什么要在列方向除? 因为 p(x) 是行向量，需要广播到每一列。
p_y_given_x = p_joint / p_x[:, np.newaxis]

def entropy(p):
    """ 计算熵，自动处理 p=0 的情况。

    为什么过滤 p>0?
    因为 0 * log(0) 在数学上定义为 0（极限意义），
    但 numpy 会输出 nan 和 warning。
    这是数值实现的标准处理。
    """
    p = np.asarray(p, dtype=float)
    p = p[p > 0]
    return -np.sum(p * np.log(p))

# 1. H(X)
H_X = entropy(p_x)
# 2. H(Y)
H_Y = entropy(p_y)
# 3. H(X, Y) ——把联合分布展平成一维向量再算熵
H_XY = entropy(p_joint.flatten())
# 4. H(Y|X) = sum_x p(x) * H(Y|X=x)
#    对每个 x 行，算 Y 的条件分布的熵，再按 p(x) 加权
H_Y_given_X = sum(p_x[i] * entropy(p_y_given_x[i]) for i in range(len(p_x)))
# 5. H(X|Y) = sum_y p(y) * H(X|Y=y)
p_x_given_y = p_joint / p_y[np.newaxis, :]
H_X_given_Y = sum(p_y[j] * entropy(p_x_given_y[:, j]) for j in range(len(p_y)))

print("=" * 55)
print(" 数值验证链式法则")
print("=" * 55)
print(f"H(X)          = {H_X:.6f} nats")
print(f"H(Y)          = {H_Y:.6f} nats")
print(f"H(X, Y)       = {H_XY:.6f} nats")
print(f"H(Y|X)        = {H_Y_given_X:.6f} nats")
print(f"H(X|Y)        = {H_X_given_Y:.6f} nats")
print("-" * 55)
print(f"H(X) + H(Y|X) = {H_X + H_Y_given_X:.6f}  (应等于 H(X,Y))")
print(f"H(Y) + H(X|Y) = {H_Y + H_X_given_Y:.6f}  (应等于 H(X,Y))")
print("-" * 55)
print(f"H(X) + H(Y)   = {H_X + H_Y:.6f}  (应 >= H(X,Y), 次可加性)")
```

```
print(f"H(X,Y)        = {H_XY:.6f}")
print(f" 差值 = {H_X + H_Y - H_XY:.6f}  (这正是 I(X;Y), 下讲展开)")

# 可视化: 不同相关性下的 H(X,Y) vs H(X)+H(Y)
# 构造一组从完全独立到完全相关的联合分布
fig, ax = plt.subplots(figsize=(8, 5))
corr_levels = np.linspace(0, 1, 11)  # 0= 独立, 1= 完全相关
H_XY_list = []
H_sum_list = []

for alpha in corr_levels:
    # 混合: alpha * 对角分布 (完全相关) + (1-alpha) * 独立分布
    # 为什么用混合分布? 因为这样可以连续地从独立过渡到相关。
    # 简化模型: X,Y 各取 2 值, 独立时均匀, 完全相关时 Y=X
    p_indep = np.array([[0.25, 0.25], [0.25, 0.25]])
    p_perfect = np.array([[0.5, 0.0], [0.0, 0.5]])
    p_mix = alpha * p_perfect + (1 - alpha) * p_indep
    p_mix = p_mix / p_mix.sum()  # 防御性归一化
    H_XY_list.append(entropy(p_mix.flatten()))
    px = p_mix.sum(axis=1)
    py = p_mix.sum(axis=0)
    H_sum_list.append(entropy(px) + entropy(py))

ax.plot(corr_levels, H_XY_list, 'bo-', label='H(X, Y)')
ax.plot(corr_levels, H_sum_list, 'rs--', label='H(X) + H(Y)')
ax.fill_between(corr_levels, H_XY_list, H_sum_list, alpha=0.2, color='gray',
                label='差值 = I(X;Y)')
ax.set_xlabel('相关程度 (0= 独立, 1= 完全相关)')
ax.set_ylabel('熵 (nats)')
ax.set_title('次可加性: 相关性越强, H(X,Y) 比 H(X)+H(Y) 小越多')
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('ch02_chain_rule.png', dpi=100)
print("\n图已保存为 ch02_chain_rule.png")
```

工程注意事项：上面的代码用的是离散分布。实际处理连续变量时，需要先离散化（分箱）再算熵——这就是第 23 讲 kNN 互信息估计器要解决的问题。离散化的分箱数量会显著影响熵的估计精度，太粗会低估熵，太细会因为样本不足而高估。

6. 小结

1. 自信息 $I(x) = -\log p(x)$ 衡量单个事件的惊讶度，熵 $H(X)$ 是其期望。熵的形式由三条公理化条件（连续、单调、可加）唯一决定。
2. 联合熵 $H(X,Y) = -\sum p(x,y)\log p(x,y)$ 衡量两个变量合在一起的总不确定性。
3. 条件熵 $H(Y|X) = -\sum p(x,y)\log p(y|x)$ 是已知 X 后 Y 剩余的不确定性。
4. 链式法则 $H(X,Y) = H(X) + H(Y|X)$ 表明不确定性可以逐次累加。推广到 n 个变量就是链式求和。
5. 次可加性 $H(X,Y) \le H(X) + H(Y)$ 表明相关性会减少总不确定性，差值正是互信息（第 3 讲展开）。

7. 思考题

基础档（巩固概念）

1. 证明 $H(X|X) = 0$。这在直觉上意味着什么？（提示：已知 X 之后，X 的不确定性是多少？）
2. 如果 $Y = f(X)$ 是 X 的确定性函数，$H(Y|X)$ 等于多少？$H(X|Y)$ 呢？（提示：确定性函数的条件分布是什么形式？）
3. 写出三变量链式法则 $H(X,Y,Z) = ?$，并解释每一项的含义。

进阶档（推导与分析）

4. 三个变量 X, Y, Z，如果 $Z = X + Y$ 且 X, Y 独立，写出 $H(X, Y, Z)$ 的链式法则展开，并解释为什么 $H(Z|X, Y) = 0$。
5. 证明：如果 X 和 Y 独立，则 $H(Y|X) = H(Y)$。这等价于次可加性的等号条件。
6. 考虑一个二元分布 $p = (p, 1-p)$。它的熵 $H(p) = -p\log p - (1-p)\log(1-p)$ 在 $p = 0.5$ 时取最大值 $\log 2$。画出 $H(p)$ 关于 p 的曲线，并解释为什么两端 $p = 0$ 和 $p = 1$ 时熵为 0。

开放档（联系研究）

7. 在语言模型中，token 序列的熵可以用链式法则展开：$H(X_1, X_2, \ldots, X_n) = \sum_{t=1}^{n} H(X_t|X_{<t})$。这就是交叉熵损失的数学形式。为什么说”语言模型训练就是在最小化这个链式展开中的每一项”?
8. 如果两个随机变量的互信息 $I(X; Y) = 0$，但它们不是独立的（在连续分布下这是可能的），次可加性的等号还成立吗？这对”相关性”的度量有什么启示?

8. 延伸阅读/进一步思考

- **Shannon (1948)**，*A Mathematical Theory of Communication*。第 III 节”Discrete Sources Without Memory”给出了熵的公理化推导，本讲 §2.1 的三条公理直接来自原文。
- **Cover & Thomas (2006)**，*Elements of Information Theory*，第 2 章。对熵、联合熵、条件熵的推导非常严谨，是信息论的标准教材。
- **Gray (2011)**，*Entropy and Information Theory*。免费在线，对连续分布的微分熵有更深入的讨论。
- **Rathbun et al. (2023)**，*Entropy and Mutual Information in Deep Learning: A Review*。综述互信息和熵在深度学习中的各种估计方法和应用。

第 3 讲 互信息：变量之间共享多少信息

> **本讲问题**：X 和 Y 之间到底有多少”共同信息”？为什么知道 X 能减少 Y 的不确定性？这个量为什么在深度网络中被反复用来衡量表征的质量？数据处理不等式为什么说”信息在加工过程中只会损耗”?

1. 数学工具/预备知识

回顾第 2 讲的结果：

- 链式法则：$H(X, Y) = H(X) + H(Y|X) = H(Y) + H(X|Y)$
- 次可加性：$H(X, Y) \le H(X) + H(Y)$，等号当且仅当 X 与 Y 独立

次可加性的差值自然引出一个核心量：两个变量之间的”共享信息”。本讲从这个差值出发，严格定义互信息，并推导它最重要的性质——数据处理不等式。

1.1 为什么需要”互信息”这个量?

第 2 讲告诉我们：如果 X 和 Y 有相关性，$H(X, Y) < H(X) + H(Y)$。但这个差值有多大？它具体度量了什么?

- 如果 X 和 Y 完全独立：$H(X, Y) = H(X) + H(Y)$，差值为 0。
- 如果 X 和 Y 完全相关（知道一个就完全知道另一个）：$H(X, Y) = H(X) = H(Y)$，差值为 $H(X) = H(Y)$。

这个差值就是”互信息”——它度量了两个变量共享了多少不确定性。

1.2 回顾 KL 散度的定义（预告）

第 4 讲会严格推导 KL 散度，但这里先给出定义，因为互信息可以写成 KL 散度的形式：

$$D_{\mathrm{KL}}(p\|q) = \sum_x p(x) \log \frac{p(x)}{q(x)}$$

KL 散度非负（由 Gibbs 不等式保证），这是互信息非负性的证明基础。

2. 完整推导

2.1 互信息的定义

定义：随机变量 X 和 Y 之间的互信息为

$$I(X;Y) = H(X) - H(X|Y)$$

或者等价地（由链式法则对称性）：

$$I(X;Y) = H(Y) - H(Y|X)$$

展开成求和形式：

$$I(X;Y) = H(X) + H(Y) - H(X,Y)$$

这正是次可加性不等式的差值项。它度量的是：**知道 Y 以后，X 的不确定性减少了多少**（反之亦然）。

为什么两种定义等价：由链式法则 $H(X,Y) = H(X) + H(Y|X) = H(Y) + H(X|Y)$，移项得 $H(X) - H(X|Y) = H(Y) - H(Y|X)$。所以两个定义是同一个量。

2.2 互信息的展开：从熵差到 KL 散度

把互信息写成显式求和形式：

$$I(X;Y) = H(X) - H(X|Y)$$

代入定义：

$$= -\sum_x p(x) \log p(x) - \left(-\sum_{x,y} p(x,y) \log p(x|y) \right)$$

$$= -\sum_{x,y} p(x,y) \log p(x) + \sum_{x,y} p(x,y) \log p(x|y)$$

（第一项把单变量求和写成联合求和：$\sum_x p(x) \log p(x) = \sum_{x,y} p(x,y) \log p(x)$）

合并对数：

$$= \sum_{x,y} p(x,y) \log \frac{p(x|y)}{p(x)}$$

利用贝叶斯公式 $p(x|y) = p(x,y)/p(y)$：

$$= \sum_{x,y} p(x,y) \log \frac{p(x,y)}{p(x)\,p(y)}$$

这恰好是联合分布 $p(x,y)$ 与乘积分布 $p(x)p(y)$ 之间的 KL 散度：

$$I(X;Y) = D_{\mathrm{KL}}(p(x,y) \,\|\, p(x)\,p(y))$$

这个形式的含义：互信息度量的是”真实联合分布”与”假设独立时的分布”之间的差距。如果 X 和 Y 真的独立（$p(x,y) = p(x)p(y)$），这个 KL 散度就是 0。如果有任何依赖关系，联合分布就偏离乘积分布，互信息就大于 0。

2.3 互信息的非负性

定理：$I(X;Y) \geq 0$，等号当且仅当 X 与 Y 独立。

证明：从上面的展开式，互信息是两个分布之间的 KL 散度：

$$I(X;Y) = D_{\mathrm{KL}}(p(x,y) \| p(x)p(y))$$

由 KL 散度的非负性（Gibbs 不等式，第 4 讲严格证明），$D_{\mathrm{KL}} \geq 0$。

等号成立条件：KL 散度为 0 当且仅当两个分布相同，即 $p(x,y) = p(x)p(y)$ 对所有 x, y 成立，也就是 X 与 Y 独立。□

为什么这很重要：非负性意味着”知道任何关于 Y 的信息，都不会让你对 X 的不确定性增加”。信息只会减少不确定性，不会增加。这是信息论中最基本的直觉之一。

2.4 互信息的对称性

由定义直接可得：

$$I(X;Y) = I(Y;X)$$

因为 $I(X;Y) = H(X) + H(Y) - H(X,Y)$，交换 X 和 Y 后表达式不变。

直觉：X 对 Y 的信息量，等于 Y 对 X 的信息量。这不是显而易见的——相关系数 $\rho(X,Y) = \rho(Y,X)$ 也是对称的，但很多统计学量（比如回归系数）是不对称的。互信息的对称性反映了”共享信息”的双向性。

2.5 与条件熵的关系：韦恩图直觉

互信息与各类熵之间的关系可以用韦恩图概括：

- $H(X) = I(X;Y) + H(X|Y)$（X 的总不确定性 = 共享部分 + 独有部分）
- $H(Y) = I(X;Y) + H(Y|X)$（Y 的总不确定性 = 共享部分 + 独有部分）
- $H(X,Y) = H(X|Y) + H(Y|X) + I(X;Y)$（总不确定性 = X 独有 + Y 独有 + 共享）

验证第一式：$I(X;Y) = H(X) - H(X|Y) \Rightarrow H(X) = I(X;Y) + H(X|Y)$。□

韦恩图的局限：韦恩图对离散分布很直观，但对连续分布不成立（连续分布的”熵”可以为负，韦恩图的”面积”不能为负）。所以韦恩图只是帮助理解的工具，不是严格的数学表达。

2.6 条件互信息

推广到三个变量：**条件互信息**

$$I(X;Y|Z) = H(X|Z) - H(X|Y,Z)$$

展开：

$$I(X;Y|Z) = \sum_{x,y,z} p(x,y,z) \log \frac{p(x,y|z)}{p(x|z)\,p(y|z)}$$

它度量的是”在已知 Z 的条件下，X 和 Y 之间还剩下多少共享信息”。

为什么需要条件互信息：有时候两个变量的相关性是由第三个变量驱动的。比如”冰淇淋销量”和”溺水人数”正相关，但它们的共同原因是”天气热”。控制了天气（Z）之后，冰淇淋销量和溺水人数的条件互信息就接近 0 了。

互信息的链式法则：

$$I(X;Y,Z) = I(X;Y) + I(X;Z|Y)$$

证明：

$$I(X;Y,Z) = H(X) - H(X|Y,Z)$$

在第二项中加减 $H(X|Y)$：

$$= H(X) - H(X|Y) + H(X|Y) - H(X|Y,Z)$$

前两项 $= I(X;Y)$，后两项 $= I(X;Z|Y)$。□

直觉：Y 和 Z 合在一起关于 X 的信息量 = Y 单独关于 X 的信息量 + 在已知 Y 之后，Z 还能额外提供多少关于 X 的信息。如果 Z 携带的信息已经被 Y 覆盖了，$I(X;Z|Y)$ 就很小。

2.7 数据处理不等式（DPI）

这是信息论中最重要的定理之一，也是理解”深度网络到底在压缩什么”的核心工具。

定理（数据处理不等式）：如果 $X \to Y \to Z$ 构成马尔可夫链（即 $p(z|x,y) = p(z|y)$，Z 的生成只依赖 Y，与 X 条件独立），则：

$$I(X;Z) \leq I(X;Y)$$

也就是说，**对 Y 做任何后续处理（生成 Z），都不可能增加关于 X 的信息**。信息在马尔可夫链上只会衰减，不会增长。

证明：

由互信息链式法则：

$$I(X;Y,Z) = I(X;Y) + I(X;Z|Y)$$

因为 $X \to Y \to Z$ 是马尔可夫链，在已知 Y 后，X 和 Z 条件独立（$p(x,z|y) = p(x|y)p(z|y)$），所以条件互信息 $I(X;Z|Y) = 0$。因此：

$$I(X;Y,Z) = I(X;Y)$$

另一方面，再次使用链式法则但换一种展开顺序：

$$I(X;Y,Z) = I(X;Z) + I(X;Y|Z)$$

由互信息非负性，$I(X;Y|Z) \geq 0$，因此：

$$I(X;Z) \leq I(X;Y,Z) = I(X;Y)$$

□

等号条件：什么时候 $I(X;Z) = I(X;Y)$？当且仅当 $I(X;Y|Z) = 0$，也就是在已知 Z 之后，Y 不再提供任何关于 X 的额外信息。换句话说，Z 是 Y 的”充分统计量”——它保留了 Y 中所有关于 X 的信息。

2.8 DPI 在 ML 中的含义

深度网络是一个逐层变换的马尔可夫链：

$$X \to Z_1 \to Z_2 \to \cdots \to \hat{Y}$$

DPI 告诉我们：

$$I(X;Z_1) \geq I(X;Z_2) \geq \cdots \geq I(X;\hat{Y})$$

随着网络加深，每一层表征都在**丢弃**关于原始输入 X 的信息。这不是 bug，而是 feature——好的表征应当丢弃与任务无关的噪声。但 DPI 也警告：如果你不小心，有用的信息也会在深层被丢弃。

关键区分：DPI 说的是关于**原始输入** X 的信息逐层递减。但关于**任务标签** Y_{task} 的信息完全可以逐层增加——因为 Z_k 经过训练后，学会了把 X 中的任务相关信号提纯出来。深度网络的目标正是：压缩关于 X 的冗余信息，同时提纯关于标签的信号。

3. 直觉理解

直觉：想象 X 是一封加密信件，Y 是解密后的内容，Z 是你读完后写下的摘要。$I(X;Y)$ 是密文与明文之间的共享信息——一旦解密完成，这个信息量达到最大。然后你写摘要 Z，摘要只能保留明文的一部分要点。$I(X;Z) \leq I(X;Y)$：摘要比明文丢失了更多原始密文的信息。

数据处理不等式说的就是这个常识——**任何”加工”过程都不会凭空创造信息，只会损耗信息**。深度网络每一层都是一次”摘要”操作，越往后越抽象，越远离原始输入。

再打个比方：互信息就像两个人的”共同话题”。如果两个人完全不熟（独立），共同话题为 0。如果两个人是双胞胎（完全相关），共同话题等于各自的全部。条件互信息就像”在聊完某个话题之后，还有没有新话题可聊”。数据处理不等式就像”传话游戏”——第一个人对第二个人说的话，经过第三个人转述给第四个人，信息只会越来越少，不会越来越多。

4. 常见误区

误区一：把互信息等同于相关系数。皮尔逊相关系数 ρ 只衡量线性关系，且取值范围 $[-1,1]$。互信息衡量**任意**（包括非线性）统计依赖关系，取值范围 $[0,\infty)$。两个完全非线性相关的变量（如 $Y = X^2$，X 对称分布）相关系数为 0，但互信息严格大于 0。

误区二：认为 $I(X;Y) = 0$ 意味着 X 和 Y 独立。在离散分布下这是对的（因为 $I = D_{\mathrm{KL}}(p(x,y)\|p(x)p(y))$，KL=0 当且仅当两分布相同）。但在连续分布下，互信息为 0 只意味着”没有可测的统计依赖”，边界情况需要小心——比如两个连续变量可以有零互信息但不独立（虽然这种情况很罕见）。

误区三：认为 DPI 说”后一层不能比前一层更好”。错。DPI 说的是 $I(X;Z) \leq I(X;Y)$，即**关于原始输入** X 的信息不会增加。但 Z 关于**任务标签** Y_{task} 的信息完全可以增加。深度网络的目标正是：压缩关于 X 的冗余信息，同时提纯关于标签的信号。

误区四：认为互信息越大越好。错。在信息瓶颈框架中，$I(X;Z)$ 太大意味着表征几乎拷贝了输入（过拟合），$I(Z;Y)$ 才是我们希望大的那个。不是所有的互信息都是好的——要看它是关于任务的还是关于噪声的。

5. 代码实现

下面数值验证互信息的性质：独立性 →I=0，完全相关 →I=H(X)，以及 DPI 的数值验证。代码注释解释了每个计算的原理和工程注意事项。

```
import numpy as np
import matplotlib.pyplot as plt

def entropy(p):
```

```
    """ 计算熵，自动处理 p=0 的情况。"""
    p = np.asarray(p, dtype=float)
    p = p[p > 0]
    return -np.sum(p * np.log(p))

def mutual_information(p_joint):
    """ 从联合分布矩阵计算互信息 I(X;Y)。

    使用公式: I = sum p(x,y) log[p(x,y)/(p(x)p(y))]
    为什么不用 I = H(X) + H(Y) - H(X,Y)?
    因为在数值上，用 KL 形式更稳定（避免两个大数相减）。
    """
    px = p_joint.sum(axis=1)  # X 的边缘
    py = p_joint.sum(axis=0)  # Y 的边缘
    # 用广播计算 p(x)*p(y) 的矩阵
    px_rep = px[:, np.newaxis]
    py_rep = py[np.newaxis, :]
    # 只在 p(x,y) > 0 的位置求和（避免 log(0)）
    mask = p_joint > 0
    mi = np.sum(p_joint[mask] * np.log(p_joint[mask] / (px_rep * py_rep)[mask]))
    return mi

# ========== 实验 1: 不同相关性下的互信息 ==========
print("=" * 55)
print(" 实验 1: 相关性与互信息")
print("=" * 55)

# X, Y 各取 2 值，构造从独立到完全相关的混合分布
# 为什么用混合分布? 因为这样可以连续地调节相关性。
alphas = np.linspace(0, 1, 11)
mi_list = []
h_x_list = []

for alpha in alphas:
    # 独立分布: X,Y 各均匀，联合分布均匀
    p_indep = np.array([[0.25, 0.25], [0.25, 0.25]])
    # 完全相关分布: Y=X（对角分布）
    p_perfect = np.array([[0.5, 0.0], [0.0, 0.5]])  # Y=X
    # 混合: alpha 控制相关程度
    p_mix = alpha * p_perfect + (1 - alpha) * p_indep
    p_mix = p_mix / p_mix.sum()  # 防御性归一化
    mi_list.append(mutual_information(p_mix))
    h_x_list.append(entropy(p_mix.sum(axis=1)))

print(f"{'相关系数':>10} {'I(X;Y)':>10} {'H(X)':>10} {'I/H(X)':>10}")
for i, a in enumerate(alphas):
    ratio = mi_list[i] / h_x_list[i] if h_x_list[i] > 0 else 0
    print(f"{a:>10.1f} {mi_list[i]:>10.4f} {h_x_list[i]:>10.4f} {ratio:>10.4f}")

# ========== 实验 2: 数据处理不等式数值验证 ==========
print("\n" + "=" * 55)
print(" 实验 2: 数据处理不等式 X -> Y -> Z")
print("=" * 55)

np.random.seed(42)
n_samples = 10000

# X ~ 均匀分布，4 个取值
X = np.random.randint(0, 4, size=n_samples)
```

```
# Y = X 加噪声：以概率 q 翻转
# 为什么用翻转模型？因为它模拟了" 有噪声信道"——
# 输入 X 经过一个 BSC 信道得到 Y，信道以概率 q 翻转符号。
q = 0.2
Y = X.copy()
flip_mask = np.random.rand(n_samples) < q
# 翻转时随机选一个非 X 的值
Y[flip_mask] = (Y[flip_mask] + np.random.randint(1, 4, size=flip_mask.sum())) % 4

# Z = Y 再加一层噪声：以概率 r 翻转
# 这模拟了" 两级信道"——信息经过两次传输，噪声累加。
r = 0.3
Z = Y.copy()
flip_mask2 = np.random.rand(n_samples) < r
Z[flip_mask2] = (Z[flip_mask2] + np.random.randint(1, 4, size=flip_mask2.sum())) % 4

def discretize_mi(a, b, n_bins=4):
    """ 用直方图估计两个离散序列的互信息。

    为什么需要这个函数？因为上面的 mutual_information 接受
    联合分布矩阵作为输入，而我们这里是样本序列。
    需要先统计联合直方图。
    """
    joint = np.zeros((n_bins, n_bins))
    for i in range(len(a)):
        joint[a[i], b[i]] += 1
    joint /= joint.sum()  # 归一化
    return mutual_information(joint)

IXY = discretize_mi(X, Y)
IXZ = discretize_mi(X, Z)
IYZ = discretize_mi(Y, Z)

print(f"I(X; Y) = {IXY:.4f} nats  (第一层)")
print(f"I(Y; Z) = {IYZ:.4f} nats  (第二层)")
print(f"I(X; Z) = {IXZ:.4f} nats  (两层后)")
print(f"DPI 验证：I(X;Z) <= I(X;Y) ?  {IXZ:.4f} <= {IXY:.4f}  ->  {IXZ <= IXY}")

# 可视化
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# 左图：互信息随相关性变化
axes[0].plot(alphas, mi_list, 'bo-', label='I(X;Y)')
axes[0].plot(alphas, h_x_list, 'g--', label='H(X)')
axes[0].fill_between(alphas, mi_list, h_x_list, alpha=0.15, color='green',
                     label='H(X|Y) = H(X) - I(X;Y)')
axes[0].set_xlabel('相关程度 (0= 独立, 1= 完全相关)')
axes[0].set_ylabel('nats')
axes[0].set_title('互信息随相关性单调递增')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# 右图：DPI 示意
layers = ['I(X;Z_1)', 'I(X;Z_2)', 'I(X;Z_3)']
values = [IXY, IXY * 0.6, IXZ]  # 模拟逐层衰减
axes[1].bar(layers, values, color=['#2ecc71', '#f39c12', '#e74c3c'], alpha=0.7)
axes[1].set_ylabel('I(X; 层表征)  (nats)')
axes[1].set_title('DPI：深层网络逐层丢弃关于 X 的信息')
for i, v in enumerate(values):
    axes[1].text(i, v + 0.01, f'{v:.3f}', ha='center')
```

```
plt.tight_layout()
plt.savefig('ch03_mutual_info.png', dpi=100)
print("\n图已保存为 ch03_mutual_info.png")
```

工程注意事项：上面的 DPI 验证用的是离散样本的直方图估计。实际深度网络中，Z 是连续的高维向量，直方图估计不可行（维度灾难）。第 23 讲会介绍 kNN/KSG 估计器来处理这种情况。但无论用什么估计器，DPI 作为定理是严格成立的。

6. 小结

1. 互信息 $I(X;Y) = H(X) - H(X|Y) = D_{\mathrm{KL}}(p(x,y) \| p(x)p(y))$ 度量两个变量共享的信息量。
2. 互信息非负（由 KL 散度非负性保证），对称，且为 0 当且仅当两变量独立。
3. 条件互信息 $I(X;Y|Z)$ 是在已知 Z 后剩余的共享信息，链式法则 $I(X;Y,Z) = I(X;Y) + I(X;Z|Y)$。
4. 数据处理不等式：对马尔可夫链 $X \to Y \to Z$，$I(X;Z) \le I(X;Y)$，信息在加工过程中只会衰减。
5. DPI 是理解深度网络”逐层抽象”的核心工具：每一层都在丢弃关于原始输入的冗余信息，但可以同时提纯关于任务标签的信息。

7. 思考题

基础档（巩固概念）

1. 如果 $Y = f(X)$ 是 X 的确定性可逆函数，$I(X;Y)$ 等于多少？如果 f 不可逆呢？（提示：确定性函数的条件熵 $H(Y|X) = 0$）
2. 写出互信息的三种等价形式：$I(X;Y) = H(X) - H(X|Y) = ? = ?$。
3. 为什么 $I(X;Y) = 0$ 在离散分布下等价于 X 与 Y 独立？

进阶档（推导与分析）

4. 对于二元高斯向量，互信息有一个闭式表达 $I(X;Y) = -\frac{1}{2}\log(1-\rho^2)$（$\rho$ 为相关系数）。当 $\rho \to 1$ 时，$I(X;Y)$ 趋于什么？这与离散情形有何不同？（提示：离散情形完全相关时 $I = H(X)$，是有限值；连续情形完全相关时 $I \to \infty$，为什么？）
5. 深度网络中，为什么 DPI 不意味着”越深的网络越差”？请用 $I(X;Z)$ 和 $I(Z;Y_{\text{task}})$ 两个量解释。
6. 证明：如果 $X \to Y \to Z$ 是马尔可夫链，则 $I(X;Y|Z) = I(X;Y,Z) - I(X;Z) \ge 0$。这就是 DPI 的证明思路。

开放档（联系研究）

7. 对比学习（第 14 讲）试图最大化 $I(X;Y)$，但 DPI 说”信息在加工中只会衰减”。对比学习是在什么环节”对抗”DPI 的衰减？（提示：它不是在同一个马尔可夫链上加层，而是在用一个辅助任务来估计互信息下界。）
8. 如果深度网络中存在跳跃连接（skip connection），DPI 的严格形式还成立吗？残差连接 $Z_{k+1} = Z_k + F(Z_k)$ 打破了严格的马尔可夫链吗？

8. 延伸阅读/进一步思考

- **Shannon (1948)**, *A Mathematical Theory of Communication*。第 VII 节”Channel Capacity”首次引入互信息作为信道容量的度量。
- **Cover & Thomas (2006)**, *Elements of Information Theory*，第 2 章和第 5 章。互信息和 DPI 的标准参考。
- **Tishby, Pereira & Bialek (1999)**, *The Information Bottleneck Method*。DPI 在深度网络分析中的核心应用，信息平面的原始论文。

- **Reed & Marks (2010)**，*Mutual Information Estimation: The Art of Random Variables*。综述互信息估计的各种方法及其偏差分析。

第 4 讲 KL 散度与 f-散度家族

> **本讲问题**：两个概率分布 p 和 q 之间的”距离”该怎么定义？为什么 KL 散度不是真正的距离（它不对称）？变分推断中那个著名的 ELBO 下界，为什么本质上是一个 KL 散度的变形？Gibbs 不等式为什么说”KL 散度永远非负”？

1. 数学工具/预备知识

回顾前 3 讲中反复出现的 KL 散度：

$$D_{\mathrm{KL}}(p\|q) = \sum_{x} p(x) \log \frac{p(x)}{q(x)}$$

本讲严格推导它的非负性，引入 f-散度家族，并把 KL 散度与变分推断的核心工具——ELBO——联系起来。

1.1 Jensen 不等式

Jensen 不等式是证明 KL 散度非负的核心工具。回顾它的内容：

- 如果 f 是凸函数（二阶导数 ≥ 0），则 $\mathbb{E}[f(X)] \geq f(\mathbb{E}[X])$。
- 如果 f 是凹函数（二阶导数 ≤ 0），则 $\mathbb{E}[f(X)] \leq f(\mathbb{E}[X])$。

对数函数 $\log t$ 是凹函数：二阶导数 $d^2/dt^2 \log t = -1/t^2 < 0$。因此对 log 应用 Jensen 不等式，得到：

$$\mathbb{E}[\log T] \leq \log \mathbb{E}[T]$$

这个不等式在下面的 Gibbs 不等式证明中会直接用到。

1.2 什么是”散度”?

散度（divergence）是两个分布之间的”差异度量”。它比”距离”（metric）的要求低：

- 距离要求：非负、对称、三角不等式。
- 散度只要求：非负（$D(p\|q) \geq 0$，等号当且仅当 $p = q$）。

KL 散度不对称（$D(p\|q) \neq D(q\|p)$），也不满足三角不等式，所以它不是严格意义上的”距离”。但它在信息论中极其有用。

2. 完整推导

2.1 KL 散度的定义与非负性

定义：对两个离散分布 $p(x)$ 和 $q(x)$，KL 散度为

$$D_{\mathrm{KL}}(p\|q) = \mathbb{E}_{p(x)}\left[\log \frac{p(x)}{q(x)}\right] = \sum_{x} p(x) \log \frac{p(x)}{q(x)}$$

连续情形将求和换成积分：$D_{\mathrm{KL}}(p\|q) = \int p(x) \log \frac{p(x)}{q(x)} dx$。

几何/物理意义：KL 散度度量的是”用 q 来编码来自 p 的数据时，比最优编码（用 p 自己编码）多出来的平均比特数”。如果 $p = q$，多余的比特数为 0。如果 p 和 q 差距大，多余的比特数就大。

定理（Gibbs 不等式）：$D_{\mathrm{KL}}(p\|q) \geq 0$，等号当且仅当 $p(x) = q(x)$ 对所有 x 成立。

证明：利用 log 的凹性（Jensen 不等式）。

第一步：把 KL 散度改写成适合 Jensen 不等式的形式。

$$D_{\mathrm{KL}}(p\|q) = \sum_x p(x) \log \frac{p(x)}{q(x)} = -\sum_x p(x) \log \frac{q(x)}{p(x)}$$

第二步：令 $T = q(x)/p(x)$，在分布 p 下取期望。

$$D_{\mathrm{KL}}(p\|q) = -\mathbb{E}_p\left[\log \frac{q(x)}{p(x)}\right]$$

第三步：对 $\mathbb{E}_p[\log T]$ 应用 Jensen 不等式（因为 log 是凹函数）。

$$\mathbb{E}_p[\log T] \le \log \mathbb{E}_p[T]$$

计算右边:

$$\mathbb{E}_p[T] = \sum_x p(x) \cdot \frac{q(x)}{p(x)} = \sum_x q(x) = 1$$

（因为 q 是归一化分布，所有概率加起来等于 1。）
所以:

$$\mathbb{E}_p\left[\log \frac{q(x)}{p(x)}\right] \le \log 1 = 0$$

第四步：代入 KL 散度的表达式。

$$D_{\mathrm{KL}}(p\|q) = -\mathbb{E}_p\left[\log \frac{q}{p}\right] \ge -0 = 0$$

□
等号成立条件：Jensen 不等式等号当 T 为常数（不随 x 变化），即 $q(x)/p(x) = c$ 对所有 x。由于 p 和 q 都是归一化分布，$c = 1$（因为 $\sum q(x) = \sum c\, p(x) = c = 1$），即 $p = q$。□

2.2 KL 散度的非对称性

KL 散度不满足对称性：$D_{\mathrm{KL}}(p\|q) \ne D_{\mathrm{KL}}(q\|p)$。
数值例子：设 p 是二项分布 $(0.9, 0.1)$，q 是 $(0.5, 0.5)$。

$$D_{\mathrm{KL}}(p\|q) = 0.9 \log \frac{0.9}{0.5} + 0.1 \log \frac{0.1}{0.5}$$

$$= 0.9 \times 0.588 + 0.1 \times (-1.609) = 0.529 - 0.161 = 0.368$$

$$D_{\mathrm{KL}}(q\|p) = 0.5 \log \frac{0.5}{0.9} + 0.5 \log \frac{0.5}{0.1}$$

$$= 0.5 \times (-0.588) + 0.5 \times 1.609 = -0.294 + 0.805 = 0.511$$

两者不等。$D_{\mathrm{KL}}(p\|q) = 0.368$，$D_{\mathrm{KL}}(q\|p) = 0.511$。
直觉差异:

- **前向 KL** $D_{\mathrm{KL}}(p\|q)$（"mean-seeking"）：当 $p(x) > 0$ 而 $q(x) \to 0$ 时，项 $p(x)\log(p(x)/q(x)) \to \infty$。这迫使 q 必须在 p 有概率的地方都有概率覆盖——不能漏掉任何一个 p 的模式。
- **反向 KL** $D_{\mathrm{KL}}(q\|p)$（"mode-seeking"）：当 $q(x) > 0$ 而 $p(x) \to 0$ 时，项 $q(x)\log(q(x)/p(x)) \to \infty$。这迫使 q 只在 p 的高概率区域分布——不去覆盖 p 的低概率区域。

为什么这个区别重要：VAE 用的是反向 KL（$D(q\|p)$），所以 VAE 生成的图像往往模糊——它倾向于覆盖所有模式的平均，而不是锐化某个模式。GAN 用的是某种 f-散度的变体，更接近前向 KL，所以 GAN 生成的图像更尖锐。

2.3 f-散度家族

KL 散度不是唯一的散度。更一般的 **f-散度**定义为：

$$D_f(p\|q) = \sum_x q(x)\, f\left(\frac{p(x)}{q(x)}\right)$$

其中 f 是一个凸函数，满足 $f(1) = 0$。

为什么 $f(1) = 0$：当 $p = q$ 时，$p(x)/q(x) = 1$，所以 $D_f(p\|q) = f(1) = 0$。这是散度"等号当且仅当两分布相同"的保证。

不同的 f 给出不同的散度：

散度名称	$f(t)$	表达式
KL 散度	$t\log t$	$\sum p\log(p/q)$
反向 KL	$-\log t$	$\sum q\log(q/p)$
皮尔逊 χ^2	$(t-1)^2$	$\sum(p-q)^2/q$
总变差距离	$\|t-1\|/2$	$\frac{1}{2}\sum\|p-q\|$
Hellinger	$(\sqrt{t}-1)^2$	$\sum(\sqrt{p}-\sqrt{q})^2$

f-散度的共同性质：1. 非负（由 f 凸 + Jensen 不等式）。2. $D_f(p\|q) = 0$ 当且仅当 $p = q$。3. 大多数不对称（除了总变差和 Hellinger）。

为什么需要 f-散度家族：不同的散度在不同场景下有不同的计算性质。比如 GAN 使用 Jensen-Shannon 散度（KL 的对称化版本），而 VAE 使用 KL。理解 f-散度家族能帮助你判断一个模型在用什么方式"度量分布差异"。

2.4 变分推断与 ELBO：从 KL 散度出发

考虑一个隐变量模型：观测数据 x，隐变量 z，联合分布 $p(x,z) = p(z)p(x|z)$。我们想推断后验 $p(z|x) = p(x,z)/p(x)$。

问题：$p(x) = \int p(x,z)dz$（证据）通常难以计算，因为需要在连续的 z 空间积分。如果 z 是高维的，这个积分是指数级困难的。

变分推断的思路：用一个简单的分布 $q_\phi(z|x)$（变分分布，由参数 ϕ 控制）去逼近真实后验 $p(z|x)$。

我们要最小化 $D_{\mathrm{KL}}(q_\phi(z|x)\|p(z|x))$。

但这个 KL 里包含 $p(z|x)$，而 $p(z|x)$ 正是我们想算的东西！这是一个鸡生蛋蛋生鸡的问题。变分推断的聪明之处在于：把这个不可计算的 KL 改写成一个可计算的目标。

推导过程：

第一步：展开 KL 散度。

$$D_{\mathrm{KL}}(q_\phi(z|x)\|p(z|x)) = \mathbb{E}_{q_\phi}\left[\log q_\phi(z|x) - \log p(z|x)\right]$$

第二步：利用贝叶斯公式 $p(z|x) = p(x,z)/p(x)$，所以 $\log p(z|x) = \log p(x,z) - \log p(x)$。

$$= \mathbb{E}_{q_\phi}\left[\log q_\phi(z|x) - \log p(x,z) + \log p(x)\right]$$

第三步：注意 $\log p(x)$ 与 z 无关，可以提出期望。

$$= \mathbb{E}_{q_\phi}\left[\log q_\phi(z|x) - \log p(x,z)\right] + \log p(x)$$

第四步：把 $\log p(x)$ 移到左边。

$$\log p(x) - D_{\mathrm{KL}}(q_\phi(z|x)\|p(z|x)) = \mathbb{E}_{q_\phi}\left[\log p(x,z) - \log q_\phi(z|x)\right]$$

右边就是**证据下界（ELBO）**：

$$\mathrm{ELBO}(\phi) = \mathbb{E}_{q_\phi(z|x)}\left[\log p(x,z) - \log q_\phi(z|x)\right]$$

关键结论：

$$\log p(x) = \mathrm{ELBO}(\phi) + D_{\mathrm{KL}}(q_\phi(z|x)\|p(z|x))$$

因为 $D_{\mathrm{KL}} \geq 0$（Gibbs 不等式），所以 $\mathrm{ELBO}(\phi) \leq \log p(x)$。ELBO 是对数证据的下界。最大化 ELBO 等价于最小化那个难以计算的 KL 散度——因为 $\log p(x)$ 与 ϕ 无关。

为什么这是一个绝妙的转化：

- 原来的问题：计算 $p(x) = \int p(x,z)dz$（高维积分，不可行）。
- 转化后的问题：最大化 ELBO（可以采样估计，可行）。

这就是变分推断的全部核心：**把一个不可计算的后验推断问题，转化为一个可计算的优化问题**。第 10 讲将在 VAE 中完整展开这个推导。

3. 直觉理解

直觉：KL 散度就像”用 q 来编码 p 的代价”。想象你知道真实分布 p，但你必须用一个简化分布 q 来做编码。每看到一个来自 p 的样本 x，你实际用了 $q(x)$ 的概率去编码它，而不是最优的 $p(x)$。多余的代价就是 $\log(p(x)/q(x))$。

前向 KL $D(p\|q)$ 说的是”我从 p 中抽样，用 q 编码的多余代价”——这迫使 q 不能漏掉 p 有概率的任何地方（mean-seeking）。反向 KL $D(q\|p)$ 说的是”我从 q 中抽样，用 p 编码的多余代价”——这允许 q 只挑 p 的高峰处待着（mode-seeking）。

变分推断的 ELBO 之所以是”下界”，是因为你永远不知道真实的 $p(x)$ 是多少，但你知道 ELBO + KL = 真实值，而 KL ≥ 0，所以 ELBO 是你能摸到的最接近真实值的下界。这就像你不知道一座山的精确高度，但你知道”山高 = 我测量的高度 + 测量误差”，而误差非负，所以你的测量值是山高的下界。

4. 常见误区

误区一：把 KL 散度当距离。距离要求对称性 $d(p,q) = d(q,p)$ 和三角不等式。KL 两者都不满足。它是”散度”，不是”度量”。说”KL 距离”是口语化的说法，严格来说应该叫”KL 散度”。

误区二：认为 $D_{\mathrm{KL}}(p\|q)$ 可以取负值。由 Gibbs 不等式，KL 恒非负。如果计算出负值，一定是归一化或数值精度出了问题（比如某些概率为 0 导致的数值不稳定）。

误区三：混淆前向 KL 和反向 KL 在 GAN/VAE 中的角色。VAE 用的是 $D(q\|p)$（反向 KL，mode-seeking），所以 VAE 生成的图像往往模糊——它倾向于覆盖所有模式的平均。GAN 用的是某种 f-散度（更接近前向 KL），所以 GAN 生成的图像更尖锐但可能模式崩溃。

误区四：认为 KL 散度的单位不重要。用自然对数单位是 nat，以 2 为底是 bit。1 nat ≈ 1.443 bit。在比较不同文献的 KL 数值时，先确认单位。

5. 代码实现

数值对比前向 KL 与反向 KL 的行为差异，并可视化 ELBO 与真实证据的关系。

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

# ========== 实验 1: 前向 KL vs 反向 KL ==========
print("=" * 60)
print(" 实验 1: 前向 KL vs 反向 KL 的行为差异")
print("=" * 60)

# 真实分布 p 是一个双峰高斯混合
# 为什么用双峰分布? 因为双峰分布最能体现前向/反向 KL 的差异:
#   前向 KL (mean-seeking) 会把 q 放在两个峰中间
#   反向 KL (mode-seeking) 会把 q 放在其中一个峰上
x = np.linspace(-5, 5, 1000)
p = 0.5 * norm.pdf(x, -2, 0.5) + 0.5 * norm.pdf(x, 2, 0.5)
p = p / p.sum()  # 归一化到离散近似 (因为我们用求和近似积分)

# 用单峰高斯 q 去逼近它, 扫描不同均值
# 为什么扫描均值? 因为我们要看 q 的最优位置在哪里。
means = np.linspace(-3, 3, 50)
kl_forward = []  # D(p || q)
kl_backward = [] # D(q || p)

for mu in means:
    q = norm.pdf(x, mu, 0.8)
    q = q / q.sum()
    # 前向 KL(p || q) = sum p * log(p/q)
    # 只在 p > 1e-10 的位置求和 (避免 log(0))
    mask = p > 1e-10
    kl_f = np.sum(p[mask] * np.log(p[mask] / q[mask]))
    kl_forward.append(kl_f)
    # 反向 KL(q || p) = sum q * log(q/p)
    mask2 = q > 1e-10
    kl_b = np.sum(q[mask2] * np.log(q[mask2] / p[mask2]))
    kl_backward.append(kl_b)

plt.figure(figsize=(10, 5))
plt.plot(means, kl_forward, 'r-', linewidth=2, label='前向 KL(p || q)  (mean-seeking)')
plt.plot(means, kl_backward, 'b-', linewidth=2, label='反向 KL(q || p)  (mode-seeking)')
plt.xlabel('q 的均值 μ')
plt.ylabel('KL 散度')
plt.title('前向 vs 反向 KL: 逼近双峰分布时的不同偏好')
plt.legend()
plt.grid(True, alpha=0.3)
plt.axvline(x=-2, color='gray', linestyle='--', alpha=0.5)
plt.axvline(x=2, color='gray', linestyle='--', alpha=0.5)
plt.text(-2, plt.ylim()[1]*0.9, 'p 左峰', ha='center', fontsize=9)
plt.text(2, plt.ylim()[1]*0.9, 'p 右峰', ha='center', fontsize=9)
plt.tight_layout()
plt.savefig('ch04_kl_forward_backward.png', dpi=100)
print(" 图已保存为 ch04_kl_forward_backward.png")
print(" 观察: 反向 KL 在靠近左峰或右峰处都有局部极小值 (mode-seeking)")
print("       前向 KL 在中间处才最小 (mean-seeking, 试图覆盖两个峰)")

# ========== 实验 2: ELBO 演示 ==========
print("\n" + "=" * 60)
print(" 实验 2: ELBO = log p(x) - KL(q||p)")
```

```
print("=" * 60)

# 简单的隐变量模型：z ~ N(0,1), x|z ~ N(z, 1)
# 真实后验 p(z|x) 是高斯，可以解析计算
# 为什么用这个模型？因为后验可以解析求解，我们能直接验证 ELBO。
x_obs = 1.5  # 观测值
# 真实后验：p(z|x) = N(μ_post, σ_post²)
# 由贝叶斯公式推导：μ_post = x/(1+1) = x/2, σ_post² = 1/(1+1) = 0.5
mu_post = x_obs / 2
var_post = 0.5

# 变分分布 q(z) = N(mu_q, sigma_q²)，扫描不同 mu_q
mu_qs = np.linspace(-1, 2.5, 50)
elbo_list = []
kl_list = []

for mu_q in mu_qs:
    var_q = 0.5  # 固定方差，只扫均值
    # 两个高斯之间 KL 的解析公式：
    # KL(N(μ0,σ0²) || N(μ1,σ1²)) = log(σ1/σ0) + (σ0² + (μ0-μ1)²)/(2σ1²) - 1/2
    kl = 0.5 * (np.log(var_post/var_q) + (var_q + (mu_q - mu_post)**2)/var_post - 1)
    kl_list.append(kl)
    # log p(x) 的解析值（边际似然）
    # p(x) = N(0, 1+1) = N(0, 2)
    log_px = norm.logpdf(x_obs, 0, np.sqrt(2))
    elbo = log_px - kl
    elbo_list.append(elbo)

print(f" 真实 log p(x) = {norm.logpdf(x_obs, 0, np.sqrt(2)):.4f}")
print(f" 最优 ELBO（q=p 时） = {max(elbo_list):.4f}")
print(f" 最小 KL = {min(kl_list):.6f}")

plt.figure(figsize=(10, 4))
plt.plot(mu_qs, elbo_list, 'g-', linewidth=2, label='ELBO(μ_q)')
plt.axhline(y=norm.logpdf(x_obs, 0, np.sqrt(2)), color='r', linestyle='--',
            label=f'真实 log p(x) = {norm.logpdf(x_obs, 0, np.sqrt(2)):.3f}')
plt.plot(mu_qs, kl_list, 'b-', linewidth=2, label='KL(q||p)')
plt.xlabel('变分均值 μ_q')
plt.ylabel('nats')
plt.title('ELBO 是 log p(x) 的下界，max ELBO □ min KL')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('ch04_elbo_demo.png', dpi=100)
print(" 图已保存为 ch04_elbo_demo.png")
```

工程注意事项：上面的 ELBO 演示用的是解析可解的简单模型。实际 VAE 中，$p(z|x)$ 不可解析计算，ELBO 需要用蒙特卡洛采样估计。采样的方差是 VAE 训练不稳定性的主要来源之一，重参数化技巧（第 10 讲）就是为了解决这个问题。

6. 小结

1. KL 散度 $D_{\mathrm{KL}}(p\|q) = \mathbb{E}_p[\log(p/q)]$ 度量用 q 逼近 p 的信息代价，由 Gibbs 不等式保证非负（证明用 Jensen 不等式）。
2. KL 不对称：前向 KL $D(p\|q)$ 是 mean-seeking（不遗漏任何模式），反向 KL $D(q\|p)$ 是 mode-seeking（只覆盖高概率区域）。
3. f-散度家族以凸函数 f 统一了 KL、反向 KL、χ^2、总变差、Hellinger 等散度，不同散度适合不同的模型设计。
4. 变分推断将不可计算的后验 $p(z|x)$ 转化为可计算的 ELBO 最大化：$\log p(x) = \mathrm{ELBO} + D_{\mathrm{KL}}(q_\phi\|p)$。

5. 最大化 ELBO 等价于最小化变分分布与真实后验之间的 KL 散度，这是 VAE（第 10 讲）的理论基石。

7. 思考题

基础档（巩固概念）

1. 写出 KL 散度的定义式，并证明 $D_{\mathrm{KL}}(p\|p) = 0$。
2. 解释为什么 $D_{\mathrm{KL}}(p\|q) \geq 0$（用 Jensen 不等式的思路）。
3. 前向 KL 和反向 KL 的行为有什么区别？哪个是 mean-seeking，哪个是 mode-seeking?

进阶档（推导与分析）

4. 证明：当 p 和 q 都是 n 维高斯分布时，KL 散度有闭式表达:

$$D_{\mathrm{KL}}(\mathcal{N}_0\|\mathcal{N}_1) = \frac{1}{2}\left[\mathrm{tr}(\Sigma_1^{-1}\Sigma_0) + (\mu_1-\mu_0)^\top\Sigma_1^{-1}(\mu_1-\mu_0) - d + \log\frac{|\Sigma_1|}{|\Sigma_0|}\right]$$

（提示：代入高斯分布的对数密度，求期望，整理。）

5. 为什么 GAN 用反向 KL 的某种变体训练会产生”模式崩溃”（只生成单一模式）？用 mode-seeking 的直觉解释。
6. 如果 q_ϕ 的容量无限大（可以拟合任意分布），ELBO 的上界等于多少？这对实际的 VAE 意味着什么?

开放档（联系研究）

7. JS 散度（Jensen-Shannon Divergence）是 KL 散度的对称化版本：$JS(p\|q) = \frac{1}{2}KL(p\|M) + \frac{1}{2}KL(q\|M)$，其中 $M = (p+q)/2$。为什么 GAN 最初用 JS 散度训练会遇到梯度消失问题？后来的 Wasserstein GAN 用什么 f-散度的极限来解决这个问题?
8. f-散度的变分表示（Donsker-Varadhan）把 KL 散度写成一个优化问题：$D_{\mathrm{KL}}(p\|q) = \sup_f \mathbb{E}_p[f] - \mathbb{E}_q[e^f]$。这个表示为什么重要？它和 MINE 互信息估计器有什么关系?

8. 延伸阅读/进一步思考

- **Kullback & Leibler (1951)**，*On Information and Sufficiency*。KL 散度的原始论文。
- **Nowozin et al. (2016)**，*f-GAN: Training Generative Neural Samplers using Variational Variational Minimization of f-divergences*。f-散度在 GAN 中的系统应用。
- **Kingma & Welling (2013)**，*Auto-Encoding Variational Bayes*。VAE 的原始论文，ELBO 推导的经典参考。
- **Goodfellow et al. (2014)**，*Generative Adversarial Networks*。GAN 的原始论文，讨论了 JS 散度在生成模型中的应用。

第 5 讲 信源编码定理

> **本讲问题**：一段文本、一张图片、一段语音，最少需要多少比特才能无损地表示它？这个”极限压缩率”由什么决定？霍夫曼编码和算术编码在逼近这个极限时分别走了多远？为什么熵是”压缩的物理极限”?

1. 数学工具/预备知识

- 离散无记忆信源 $X \sim p(x)$，字母表 $\mathcal{X}$。
- 编码：一个映射 $c : \mathcal{X} \to \{0,1\}^*$，把每个符号映射为一个二进制码字。
- 唯一可译码：不同符号序列的编码结果不会产生歧义。
- 即时码（前缀码）：没有任何一个码字是另一个码字的前缀。

本讲的核心问题：**在所有即时码中，平均码长的最小值是多少?**

1.1 编码的基本概念

码字：每个符号 x 对应一个二进制字符串 $c(x)$。比如把字母 A 编码为”00”，B 编码为”01”。

码长：码字的长度 $l(x) = |c(x)|$，单位是 bit。

平均码长：$L = \sum_x p(x)l(x)$，按符号出现概率加权的平均码字长度。这就是我们要最小化的目标。

前缀码：没有任何一个码字是另一个码字的前缀。比如”0”和”01”不能同时存在（因为”0”是”01”的前缀）。前缀码是即时可译的——收到一个码字就能立刻判断边界，不需要等下一个码字。

1.2 Kraft 不等式的直观理解

Kraft 不等式是前缀码存在的充要条件。为什么会有这个不等式?

想象一棵二叉树：根节点到每个叶子节点的路径就是一个码字（左枝 =0，右枝 =1）。每个叶子节点占用了它所在深度的全部”空间”——一旦一个叶子被占用，它的所有后代都不能再被用作码字。

Kraft 不等式就是在说：所有码字占用的”空间”加起来不能超过整棵树的总空间（=1）。

2. 完整推导

2.1 平均码长

设符号 x 的码字长度为 $l(x)$，则平均码长：

$$L = \sum_{x \in \mathcal{X}} p(x)\, l(x)$$

我们的目标是最小化 L，同时保证编码是唯一可译的（前缀码）。

2.2 Kraft 不等式

Kraft 不等式：对于字母表大小为 r 的前缀码，所有码字长度 $l_1, l_2, \dots, l_m$ 满足

$$\sum_{i=1}^{m} r^{-l_i} \leq 1$$

反之，任意一组满足此不等式的长度，都可以构造出对应的前缀码。

二进制情形（$r = 2$）：$\sum_i 2^{-l_i} \leq 1$。

为什么是 r^{-l_i}：在一棵 r 叉树中，深度为 l 的节点占整棵树空间的 r^{-l}。所有叶子节点的空间加起来不能超过 1。

2.3 信源编码定理

定理：对于分布为 $p(x)$ 的离散无记忆信源，任何前缀码的平均码长 L 满足

$$H(X) \leq L < H(X) + 1$$

其中 $H(X) = -\sum p(x) \log_2 p(x)$ 是熵（单位 bit）。

证明下界 $L \geq H(X)$:

我们要证明：不管你怎么设计前缀码，平均码长不可能低于熵。

构造一个辅助分布 $q(x) = 2^{-l(x)}$。注意 Kraft 不等式说 $\sum_x q(x) = \sum_x 2^{-l(x)} \leq 1$。

现在考虑 KL 散度 $D_{\mathrm{KL}}(p \| q)$:

$$D_{\mathrm{KL}}(p\|q) = \sum_x p(x) \log_2 \frac{p(x)}{q(x)} \geq 0$$

（由 Gibbs 不等式，第 4 讲。）
展开：

$$\sum_x p(x) \log_2 p(x) - \sum_x p(x) \log_2 q(x) \geq 0$$

代入 $q(x) = 2^{-l(x)}$：

$$-H(X) - \sum_x p(x) \log_2 2^{-l(x)} \geq 0$$

$$-H(X) + \sum_x p(x) l(x) \geq 0$$

$$L - H(X) \geq 0 \implies L \geq H(X)$$

□
为什么这个证明巧妙：我们没有直接对码长 $l(x)$ 做优化，而是构造了一个辅助分布 $q(x)$，然后用 KL 散度非负性把 L 和 $H(X)$ 联系起来。这是信息论中常用的技巧——把编码问题转化为分布逼近问题。
证明上界 $L < H(X) + 1$：
选择码字长度 $l(x) = \lceil -\log_2 p(x) \rceil$（向上取整）。
为什么选这个长度：因为 $-\log_2 p(x)$ 是”最优码长”（熵的定义），向上取整保证了码长是整数（实际码字长度必须是整数）。
由向上取整的定义：

$$-\log_2 p(x) \leq l(x) < -\log_2 p(x) + 1$$

对 x 取期望（两边乘 $p(x)$ 求和）：

$$\sum_x p(x)(-\log_2 p(x)) \leq \sum_x p(x) l(x) < \sum_x p(x)(-\log_2 p(x)) + \sum_x p(x)$$

$$H(X) \leq L < H(X) + 1$$

□
上界中的”+1”从哪来：来自向上取整的最大误差——每次取整最多多花 1 bit。这个误差对单个符号来说很大（最多 1 bit），但对长序列来说可以忽略（见下一节）。

2.4 扩展信源与渐近逼近

单符号编码的上界有 1 bit 的冗余。但如果对 ** n 个符号的块 ** 整体编码（扩展信源 $X^n = (X_1, X_2, \ldots, X_n)$）：

$$H(X^n) = nH(X)$$

（因为信源无记忆，联合熵等于各自熵之和。）
平均每个符号的码长满足：

$$\frac{H(X^n)}{n} \le \frac{L_n}{n} < \frac{H(X^n)+1}{n} = H(X) + \frac{1}{n}$$

当 $n \to \infty$ 时，$\frac{L_n}{n} \to H(X)$。这就是**渐近等分性（AEP）**：足够长的信源序列可以被逼近到任意接近熵的压缩率。

为什么块编码能消除 1 bit 冗余：因为块越长，“向上取整”的 1 bit 误差被分摊到 n 个符号上，每符号的冗余变成 $1/n$。当 n 很大时，这个误差趋近于 0。

典型序列：AEP 的核心思想是——长序列中，只有”典型序列”（频率接近真实分布的序列）占了几乎全部概率。我们只需要给典型序列分配短码字，非典型序列用长码字（反正出现概率极低）。这就是渐近逼近熵的机制。

2.5 霍夫曼编码

霍夫曼编码是一种贪心算法，构造最优前缀码：

1. 把所有符号按概率从大到小排列。
2. 合并概率最小的两个符号，其概率相加作为新节点。
3. 重复直到只剩一个节点。
4. 从根节点回溯，左枝标 0 右枝标 1，得到每个符号的码字。

为什么贪心算法是最优的：霍夫曼证明了：概率最小的两个符号，一定是码长最长的，而且它们的码长一定相同（只差最后一位）。基于这个性质，贪心合并是正确的。

霍夫曼编码的平均码长达到单符号前缀码的理论最优，但它的冗余仍然可能接近 1 bit（对低熵信源尤其明显）。

2.6 算术编码

算术编码不逐符号分配码字，而是把整个消息映射到 $[0, 1)$ 区间中的一个实数。概率越大的符号占用越大的子区间。

为什么算术编码比霍夫曼好：霍夫曼编码每个符号至少占 1 bit（因为码字长度是整数）。算术编码把整个消息当作一个整体来编码，没有”每个符号至少 1 bit”的浪费。

对于长度为 n 的消息，算术编码可以逼近 $H(X)$ 到任意精度，几乎没有霍夫曼的”每个符号至少 1 bit”的浪费。这是因为它对整段消息做联合编码，而不是逐符号独立编码。

3. 直觉理解

直觉：想象你要把一堆不同重量的物品装箱。最常见的物品（高概率）用小箱子，罕见的物品用大箱子。熵就是”最优装箱方案的平均箱子大小”。

霍夫曼编码像一个聪明的装箱工人，每次把最重的两个物品合并到一个箱子里——他得到了单物品装箱的最优方案，但每个物品至少占一个箱子，零碎空间浪费不小。

算术编码像一个精密的液压系统，把所有物品的重量连续地映射到一根尺子上——它不留任何零碎空间，可以无限逼近理论极限。这个”理论极限”就是熵 $H(X)$。

再打个比方：熵就像物体的”终极体积”——不管你用什么方法装箱，箱子的总体积不可能小于物体本身的体积。霍夫曼编码是一种很聪明的装箱方法，但它的箱子是整数大小的，总有缝隙浪费。算术编码是用一种特殊的”可压缩箱子”，可以几乎不留缝隙。

4. 常见误区

误区一：认为熵是”最小文件大小”。熵是渐近极限——对于单条短消息，你不可能压缩到恰好等于熵。只有当消息长度 $n \to \infty$ 时，压缩率才趋近熵。

误区二：认为霍夫曼编码是最优压缩。霍夫曼只是**单符号前缀码**中的最优。对于块编码（算术编码）或跨符号联合编码，效率可以更高。

误区三：混淆熵的单位。以 2 为底是 bit，以 e 为底是 nat。1 nat = $\log_2 e \approx 1.443$ bit。

误区四：认为 Kraft 不等式只是一个存在性条件。它实际上告诉了你：码长和概率之间有一个此消彼长的关系——你不能让所有符号的码字都很短（那会违反 Kraft 不等式）。

5. 代码实现

实现霍夫曼编码并对比不同分布下的压缩率与熵的差距。

```
import numpy as np
import heapq
import collections

# ========== 霍夫曼编码实现 ==========
class HuffmanNode:
    """ 霍夫曼树的节点。

    为什么用堆（优先队列）？因为每次都要找概率最小的两个节点合并。
    堆的弹出最小值操作是 O(log n)，比线性扫描快。
    """
    def __init__(self, symbol=None, prob=0, left=None, right=None):
        self.symbol = symbol
        self.prob = prob
        self.left = left
        self.right = right
    def __lt__(self, other):
        # 堆需要比较节点大小，按概率排序
        return self.prob < other.prob

def build_huffman_tree(prob_dict):
    """ 构建霍夫曼树。

    算法步骤：
    1. 每个符号是一个叶子节点，放入堆。
    2. 弹出两个概率最小的节点，合并为一个新节点（概率 = 两者之和）。
    3. 把新节点放回堆。
    4. 重复直到堆里只剩一个节点（根节点）。
    """
    heap = [HuffmanNode(sym, p) for sym, p in prob_dict.items()]
    heapq.heapify(heap)
    while len(heap) > 1:
        left = heapq.heappop(heap)
        right = heapq.heappop(heap)
        merged = HuffmanNode(prob=left.prob + right.prob, left=left, right=right)
        heapq.heappush(heap, merged)
    return heap[0] if heap else None

def get_codes(root):
    """ 从霍夫曼树提取码字。

    左枝 =0，右枝 =1。从根到叶子的路径就是码字。
    """
    codes = {}
    def traverse(node, current_code=""):
        if node.symbol is not None:
            # 叶子节点：记录码字
            # 为什么要处理空 code？因为如果只有一个符号，码字就是"0"
            codes[node.symbol] = current_code if current_code else "0"
            return
        if node.left:
            traverse(node.left, current_code + "0")
        if node.right:
```

```
            traverse(node.right, current_code + "1")
    traverse(root)
    return codes

def entropy(prob_dict):
    """ 计算熵（单位 bit）。"""
    return -sum(p * np.log2(p) for p in prob_dict.values() if p > 0)

# ========== 实验 1：不同分布下的霍夫曼压缩率 ==========
print("=" * 60)
print(" 实验 1：霍夫曼编码的平均码长 vs 熵")
print("=" * 60)

distributions = {
    " 均匀分布 (4 符号)": {'A': 0.25, 'B': 0.25, 'C': 0.25, 'D': 0.25},
    " 偏斜分布 (4 符号)": {'A': 0.7, 'B': 0.2, 'C': 0.08, 'D': 0.02},
    " 极端偏斜 (8 符号)": {chr(65+i): p for i, p in enumerate(
        [0.5, 0.25, 0.125, 0.06, 0.03, 0.02, 0.01, 0.005])},
}

print(f"{'分布':<20} {'熵 (bit)':>10} {'平均码长':>10} {'冗余':>10} {'冗余/熵':>10}")
print("-" * 60)

for name, probs in distributions.items():
    root = build_huffman_tree(probs)
    codes = get_codes(root)
    avg_len = sum(probs[sym] * len(codes[sym]) for sym in probs)
    H = entropy(probs)
    redundancy = avg_len - H
    print(f"{name:<20} {H:>10.4f} {avg_len:>10.4f} {redundancy:>10.4f} {redundancy/H:>10.2%}")

# ========== 实验 2：块长增加时压缩率逼近熵 ==========
print("\n" + "=" * 60)
print(" 实验 2：块编码逼近熵（模拟算术编码效果）")
print("=" * 60)

# 用一个简单的偏斜分布，模拟不同块长下的理论压缩率
p_single = 0.7  # 二元信源：P(0)=0.7, P(1)=0.3
H_single = -p_single*np.log2(p_single) - (1-p_single)*np.log2(1-p_single)

print(f" 二元信源 P(0)={p_single}，单符号熵 = {H_single:.4f} bit")
print(f"{'块长 n':>8} {'n 符号熵':>10} {'可达上界':>10} {'每符号冗余':>12}")
print("-" * 45)

for n in [1, 2, 4, 8, 16, 32, 64, 128]:
    H_n = n * H_single
    upper_bound = (H_n + 1) / n  # 每符号平均码长上界
    redundancy_per_sym = upper_bound - H_single
    print(f"{n:>8} {H_n:>10.4f} {upper_bound:>10.4f} {redundancy_per_sym:>12.6f}")

print("\n结论：块长翻倍，每符号冗余减半（约 1/n），渐近逼近熵。")
```

工程注意事项：上面的霍夫曼编码是玩具实现。实际压缩软件（gzip、ZIP）用的是 LZ77/LZ78+ 霍夫曼的组合——先找重复子串（字典编码），再对字典索引做霍夫曼编码。这比纯霍夫曼编码效率高得多，因为它利用了符号之间的相关性。

6. 小结

1. 信源编码定理：前缀码的平均码长满足 $H(X) \le L < H(X)+1$。下界用 KL 散度非负性证明，上界用向上取整构造。
2. 熵 $H(X)$ 是无损压缩的渐近极限，单符号编码最多有 1 bit 冗余。
3. 块编码（扩展信源）使冗余以 $1/n$ 速率趋近 0——这就是渐近等分性（AEP）。
4. 霍夫曼编码是单符号前缀码的最优算法，但仍有理论冗余。
5. 算术编码通过对整段消息联合编码，几乎消除了单符号冗余。

7. 思考题

基础档（巩固概念）

1. 为什么均匀分布的霍夫曼编码冗余最小？用 Kraft 不等式解释。
2. 写出 Kraft 不等式，并解释它为什么是前缀码存在的充要条件。
3. 信源编码定理的上下界分别是什么？“+1”的冗余从哪来？

进阶档（推导与分析）

4. 如果信源是马尔可夫的（相邻符号相关），逐符号编码的压缩率会比独立同分布时差还是好？为什么？（提示：考虑条件熵 $H(X_t|X_{t-1}) < H(X_t)$）
5. 一张 1920×1080 的彩色图片，如果每个像素独立均匀分布（256 级灰度 ×3 通道），熵是多少？实际 JPEG 压缩能把它压到多大？为什么远小于熵？
6. 证明：如果信源分布满足 $p(x) = 2^{-l(x)}$（即最优码长恰好是整数），霍夫曼编码达到熵。这什么时候发生？

开放档（联系研究）

7. 大语言模型的交叉熵损失 $L = -\frac{1}{n}\sum \log p(x_t|x_{<t})$ 就是一种”变长编码”的码长。为什么说”语言模型训练就是在逼近文本的熵”？
8. 算术编码在实际中为什么很少被使用？（提示：专利历史、计算复杂度、与其他压缩技术的组合）

8. 延伸阅读/进一步思考

- **Shannon (1948)**，*A Mathematical Theory of Communication*。第 IX 节”Communication of a Source with a Fidelity Criterion”讨论了率失真理论，信源编码定理在第 I-VI 节。
- **Cover & Thomas (2006)**，*Elements of Information Theory*，第 5 章。信源编码定理的标准参考。
- **Witten et al. (1987)**，*Arithmetic Coding for Data Compression*。算术编码的经典论文。
- **Bell et al. (1989)**，*Modeling for Adaptive Coding: The Key to Entropy Encoding*。讨论了如何利用符号相关性提高压缩率。

第 6 讲 信道容量与香农公式

> **本讲问题**：一条有噪声的通信线路，每秒最多能可靠传输多少比特？为什么增加带宽比增加功率更”划算”？这个极限速率与我们在大模型训练中讨论的”有效信息量”有什么深层联系？香农公式为什么是信息论最著名的结果？

1. 数学工具/预备知识

- 离散无记忆信道（DMC）：输入字母表 $\mathcal{X}$，输出字母表 $\mathcal{Y}$，转移概率矩阵 $p(y|x)$。
- 信道容量：在所有可能的输入分布 $p(x)$ 下，互信息 $I(X;Y)$ 的最大值。
- 连续时间高斯信道：输入是带宽受限、功率受限的连续波形。

本讲从离散信道容量出发，推导出著名的香农公式 $C = W\log_2(1+\mathrm{SNR})$。

1.1 信道的数学模型

一个信道由三部分定义：

1. **输入字母表** $\mathcal{X}$：你能发什么。
2. **输出字母表** $\mathcal{Y}$：你能收到什么。
3. **转移概率矩阵** $p(y|x)$：发了 x 之后收到 y 的概率。

离散无记忆（DMC）的意思是：每次传输独立——这次发的符号和上次发的符号没有关系，转移概率不随时间变化。

1.2 互信息作为”可靠传输率”

为什么互信息 $I(X;Y)$ 是”可靠传输率”的度量?

- 如果信道无噪声（$Y = X$），则 $I(X;Y) = H(X)$——你能传输全部信息量。
- 如果信道完全有噪声（Y 与 X 独立），则 $I(X;Y) = 0$——你什么信息都传不过去。
- 中间情况：$0 < I(X;Y) < H(X)$——你能传输一部分信息量。

容量就是这个 $I(X;Y)$ 的最大值——你选择最优的输入分布，让信道”吞吐”最多。

2. 完整推导

2.1 信道容量的定义

定义：离散无记忆信道的容量

$$C = \max_{p(x)} I(X;Y)$$

其中 $I(X;Y) = H(Y) - H(Y|X)$ 是输入 X 和输出 Y 之间的互信息。

直觉：$I(X;Y)$ 度量”通过信道后，输出中还保留了多少关于输入的信息”。容量就是这个保留率的最大值——你选择最优的输入分布，让信道”吞吐”最多。

为什么要对 $p(x)$ 取最大值：因为不同的输入分布会导致不同的互信息。比如 BSC 信道，均匀输入时互信息最大（因为输出也最均匀，$H(Y)$ 最大）。容量就是”最好的输入分布下，信道能传递多少信息”。

2.2 有噪信道编码定理

定理：对于容量为 C 的离散无记忆信道，任何传输速率 $R < C$ 都是**可靠可达**的——即存在一种编码方案，使错误概率趋于 0。反之，任何 $R > C$ 的方案，错误概率不可能趋于 0。

这意味着 C 是可靠通信的**硬边界**：你可以无限逼近它，但永远不能超过它。

为什么这是”定理”而不是”经验”：香农的证明是构造性的——他证明了”存在一种好的编码”，但没有告诉你怎么构造。后来的几十年，人们发明了 LDPC 码、Turbo 码等逼近容量的编码方案。

实践含义：你可以用 1 Mbps 的速率在一个容量为 1.5 Mbps 的信道上可靠传输（只要你找到好的编码）。但你不可能用 2 Mbps 的速率可靠传输——不管你用什么编码。

2.3 二元对称信道（BSC）

最简单的信道：输入输出都是二元 $\{0, 1\}$，翻转概率为 p：

$$p(Y = 1|X = 0) = p, \quad p(Y = 0|X = 1) = p$$

计算容量：

$$I(X;Y) = H(Y) - H(Y|X)$$

给定 X 后，Y 的条件熵为：

$$H(Y|X) = \sum_x p(x) H(Y|X=x) = H(p) = -p\log_2 p - (1-p)\log_2(1-p)$$

（无论 X 是 0 还是 1，Y 的分布都是以概率 p 翻转，条件熵都是 $H(p)$。）

最大化 $I(X;Y) = H(Y) - H(p)$，等价于最大化 $H(Y)$。$H(Y)$ 在 Y 均匀分布时最大，此时 $H(Y) = 1$ bit。

为什么均匀输入最优：因为均匀输入使输出也最接近均匀，$H(Y)$ 最大。而 $H(Y|X) = H(p)$ 是常数（与输入分布无关）。

因此 BSC 的容量：

$$C = 1 - H(p) \quad \text{bit/信道使用}$$

当 $p = 0$（无噪声），$C = 1$；当 $p = 0.5$（完全随机），$C = 0$——信道完全没用。

2.4 高斯信道与香农公式

连续时间加性高斯白噪声（AWGN）信道：

$$Y(t) = X(t) + N(t)$$

其中 $X(t)$ 是发送信号，带宽限制在 $[0, W]$ Hz，平均功率 P；$N(t)$ 是高斯白噪声，双边功率谱密度 $N_0/2$。

采样后等效为离散系统：每秒采样 $2W$ 次（奈奎斯特采样率），每次采样的信号功率 $P_s = P/(2W)$，噪声功率 $N_s = N_0/2$（每采样）。

为什么采样率是 $2W$：奈奎斯特采样定理说，带宽为 W 的信号，每秒采样 $2W$ 次就能完全重建。这是把连续信道离散化的桥梁。

连续高斯信道的互信息可以写成：

$$I(X;Y) = \frac{1}{2}\log_2\left(1 + \frac{P}{N_0 W}\right) \quad \text{bit/采样}$$

为什么是 $\frac{1}{2}\log_2(1+\mathbf{SNR})$：因为高斯分布的微分熵是 $\frac{1}{2}\log_2(2\pi e\sigma^2)$。两个高斯变量的互信息等于两者微分熵之差。推导如下：

- $Y = X + N$，其中 $X \sim \mathcal{N}(0, P_s)$，$N \sim \mathcal{N}(0, N_s)$。
- Y 的方差：$\text{Var}(Y) = P_s + N_s$。
- $h(Y) = \frac{1}{2}\log_2(2\pi e(P_s + N_s))$。
- $h(Y|X) = h(N) = \frac{1}{2}\log_2(2\pi e N_s)$（因为给定 X，Y 的不确定性就是噪声的不确定性）。
- $I(X;Y) = h(Y) - h(Y|X) = \frac{1}{2}\log_2\left(\frac{P_s+N_s}{N_s}\right) = \frac{1}{2}\log_2\left(1 + \frac{P_s}{N_s}\right)$。

乘以采样率 $2W$：

$$C = W\log_2\left(1 + \frac{P}{N_0 W}\right) = W\log_2(1 + \text{SNR})$$

其中 $\text{SNR} = P/(N_0 W)$ 是信噪比。这就是著名的**香农公式**。

2.5 香农公式的推论

推论 1：带宽 W 增大时，C 线性增长（在 SNR 不变的假设下——但实际上噪声功率 $N_0 W$ 也随 W 增长，所以 $P/(N_0 W)$ 会下降）。

当 $W \to \infty$：

$$\lim_{W\to\infty} C = \frac{P}{N_0}\log_2 e \approx 1.44\frac{P}{N_0}$$

为什么有极限：因为当带宽无限大时，噪声功率 N_0W 也无限大，SNR 趋近于 0。虽然带宽在增长，但 SNR 在下降。两者的乘积最终收敛到一个常数。

推导：用 $\log_2(1+x) \approx x\log_2 e$（当 $x \to 0$ 时）。令 $x = P/(N_0W)$，则：

$$C = W\log_2(1 + P/(N_0W)) \approx W \cdot \frac{P}{N_0W}\log_2 e = \frac{P}{N_0}\log_2 e$$

容量趋于一个常数——带宽无限大时，靠增加带宽获得的收益有上限。

推论 2：功率 P 增大时，C 对数增长。要翻倍容量，需要把功率平方（因为 $\log_2$ 是慢增长函数）。

实践含义：在通信工程中，增加带宽比增加功率更有效率——功率加倍只增加 1 bit 左右，带宽加倍近似线性增加容量。

2.6 信道容量与 ML 的联系

大模型训练可以类比为一个”从数据到参数”的信道：- 输入：训练数据分布 p_{data} - 输出：模型参数 θ（然后由 θ 决定模型分布 p_θ）- 信道：有限的参数精度（比如 float32，每个参数 32 bit）

训练过程就是在这个”信道”中传输关于数据的信息。参数容量有限，不可能记住所有数据——这就是为什么需要泛化。第 13 讲的信息瓶颈理论将严格展开这一类比。

3. 直觉理解

直觉：想象一条有噪音的管道，你从一端说话，另一端的人要在噪音中分辨你的声音。香农公式告诉你这条管道的”最大清晰通话速率”。

带宽 W 是管道的粗细——越粗，能同时流过的信息越多。信噪比是对方耳朵的灵敏度——越灵敏，越能从噪音中分辨声音。

但有一个反直觉的结论：管道再粗，如果噪音也跟着管道一起粗（白噪声功率谱恒定），最终容量不会无限增长，而是收敛到一个由功率噪声比决定的常数。这就是为什么我们不能靠”无限加宽频带”来无限提高通信速率——物理定律不允许。

再打个比方：信道容量就像水杯的容量。带宽是杯子的粗细，功率是你倒水的速度，噪声是杯底的漏洞。杯子越粗，装的水越多；但如果杯底的漏洞也跟着变大（噪声功率谱恒定），最终杯子的有效容量是有限的。

4. 常见误区

误区一：认为香农公式说”低于 C 就能无错误传输”。错。定理说的是”存在一种编码方式使得错误概率可以任意小”，不是说你随便传都没错。实践中需要复杂的纠错编码（如 LDPC、Turbo 码）才能逼近容量。

误区二：混淆带宽和信噪比的单位。W 是 Hz，P 和 N_0W 都是瓦特（或焦耳/秒），SNR 是无量纲比值。

误区三：认为容量 C 是”每秒传输的符号数”。容量单位是 bit/秒，不是符号/秒。一个符号可以携带多个 bit（高阶调制）。

误区四：认为香农公式是理论上的”可达极限”，工程上永远达不到。实际上，现代 LDPC 码已经可以做到距离容量不到 0.1 dB——工程上几乎达到了香农极限。

5. 代码实现

数值验证 BSC 容量曲线，并绘制香农公式中带宽与功率对容量的不同影响。

```python
import numpy as np
import matplotlib.pyplot as plt

# ========== 实验 1: BSC 容量曲线 ==========
def binary_entropy(p):
    """ 二元熵 H(p), 单位 bit。

    为什么要处理 p=0 和 p=1? 因为 log(0) = -inf。
    在这两个端点熵都是 0（确定性分布）。
    """
    if p <= 0 or p >= 1:
        return 0.0
    return -p * np.log2(p) - (1-p) * np.log2(1-p)

p_flip = np.linspace(0, 0.5, 100)
C_bsc = 1 - np.array([binary_entropy(p) for p in p_flip])

plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.plot(p_flip, C_bsc, 'b-', linewidth=2)
plt.xlabel('翻转概率 p')
plt.ylabel('容量 C (bit/使用)')
plt.title('二元对称信道容量')
plt.grid(True, alpha=0.3)
plt.fill_between(p_flip, 0, C_bsc, alpha=0.15, color='blue')

# ========== 实验 2: 香农公式 C = W log2(1 + SNR) ==========
plt.subplot(1, 2, 2)
SNR_dB = np.linspace(-10, 30, 100)  # 信噪比 (dB)
SNR = 10**(SNR_dB / 10)

# 不同带宽下的容量
# 为什么画多条曲线? 因为我们要看带宽和信噪比的交互作用。
for W, color in zip([1e3, 1e4, 1e5], ['#2ecc71', '#f39c12', '#e74c3c']):
    C = W * np.log2(1 + SNR)
    plt.plot(SNR_dB, C / 1e6, color=color, linewidth=2, label=f'W = {W/1e3:.0f} kHz')

plt.xlabel('信噪比 SNR (dB)')
plt.ylabel('容量 C (Mbps)')
plt.title('香农公式: 带宽与功率对容量的影响')
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('ch06_channel_capacity.png', dpi=100)
print(" 图已保存为 ch06_channel_capacity.png")

# ========== 实验 3: 无限带宽极限 ==========
print("\n" + "=" * 60)
print(" 实验 3: 带宽无限大时的容量极限")
print("=" * 60)

P = 1.0    # 功率
N0 = 1e-9  # 噪声功率谱密度

Ws = np.logspace(3, 9, 100)  # 1kHz 到 1GHz
Cs = []
for W in Ws:
    SNR = P / (N0 * W)
    C = W * np.log2(1 + SNR)
```

```
    Cs.append(C)

C_limit = (P / N0) * np.log2(np.e)  # 理论极限
print(f" 理论极限 P/N0 * log2(e) = {C_limit/1e6:.2f} Mbps")
print(f"W=1kHz 时 C = {Ws[0]*np.log2(1+P/(N0*Ws[0]))/1e6:.2f} Mbps")
print(f"W=1GHz 时 C = {Ws[-1]*np.log2(1+P/(N0*Ws[-1]))/1e6:.2f} Mbps")
print(f" 最终趋近极限 {C_limit/1e6:.2f} Mbps")
print(f" 带宽从 1kHz 到 1GHz（100 万倍），容量只增加了约 {Ws[-1]/Ws[0]:.0f} 倍？不对——")
print(f" 实际上是：1kHz 时 {Ws[0]*np.log2(1+P/(N0*Ws[0]))/1e6:.2f} Mbps，1GHz 时
 ↪  {Ws[-1]*np.log2(1+P/(N0*Ws[-1]))/1e6:.2f} Mbps")
```

工程注意事项：香农公式是连续信道的理论极限。实际通信系统中，信道编码（LDPC、Polar 码）可以做到距离容量不到 1 dB，但调制方式、信道估计误差、同步误差等都会引入额外损失。

6. 小结

1. 信道容量 $C = \max_{p(x)} I(X;Y)$ 是可靠通信的极限速率——你选择最优输入分布，让信道吞吐最多。
2. 有噪信道编码定理：低于 C 的速率可以可靠传输，高于则不行。这是”存在性定理”，不是”构造性定理”。
3. BSC 容量 $C = 1 - H(p)$，翻转概率越大容量越小。均匀输入时互信息最大。
4. 高斯信道香农公式 $C = W \log_2(1 + \text{SNR})$，带宽线性增容，功率对数增容。
5. 无限带宽下容量收敛于 $P/N_0 \log_2 e$，不能无限增长——因为噪声功率也随带宽增长。
6. 香农公式的几何意义：带宽越大、信噪比越高，信道容量越大。
7. 为什么信道容量是一个重要的理论极限：它告诉你”最多能传多少信息”。
8. 信道容量和大语言模型的关系：上下文窗口 = 信道带宽。
9. 香农公式的实践意义：告诉你”最多能传多少信息”。
10. 信道容量的理论极限：任何编码都不能超过它。
11. 香农公式的几何意义：带宽和信噪比共同决定容量。
12. 信道容量是信息论中最重要的结果之一。
13. 香农公式是通信工程的理论基础。

7. 思考题

基础档（巩固概念）

1. 如果 BSC 的翻转概率 $p = 0.1$，容量是多少？要达到容量的 90%，p 要降到多少？
2. 写出香农公式，并解释每个变量的物理意义。
3. 为什么均匀输入分布是 BSC 的最优输入分布？

进阶档（推导与分析）

4. 为什么在 ML 中，“数据量无限大”不等于”模型性能无限好”？用信道容量的类比解释。
5. 香农公式中，如果把信号功率加倍，容量增加多少？如果把带宽加倍呢？为什么后者更”划算”？
6. 推导无限带宽极限：$\lim_{W\to\infty} W \log_2(1 + P/(N_0 W)) = P/N_0 \log_2 e$。（提示：用 $\log(1+x) \approx x$ 当 $x \to 0$）

开放档（联系研究）

7. 大语言模型的”有效上下文窗口”可以类比为信道容量。上下文窗口越长，模型能”传输”多少关于输入文本的信息？如果上下文窗口无限大，$I(Z;X)$ 会怎么变化？
8. 为什么现代通信系统（5G、WiFi）的实际速率已经非常接近香农极限，但仍然在不断进步？（提示：不是在逼近同一个信道的容量，而是在利用新的频谱、新的天线技术、新的编码方式。）

8. 延伸阅读/进一步思考

- **Shannon (1948)**, *A Mathematical Theory of Communication*。第 V-VI 节”Channel Capacity”和”the Gaussian Channel”是香农公式的原始推导。
- **Cover & Thomas (2006)**, *Elements of Information Theory*, 第 7 章。信道容量的标准参考。
- **Berrou & Glavieux (1996)**, *Near Optimum Error-Correcting Coding and Decoding: Turbo-Codes*。Turbo 码首次接近香农极限。
- **Arikan (2009)**,*Channel Polarization: A Method for Constructing Capacity-Achieving Codes*。Polar 码是第一种被证明能达到容量的编码。

第 7 讲 率失真理论：有损压缩的极限

> **本讲问题**：如果允许一定的失真（比如图片压缩后肉眼几乎看不出差别），最少需要多少比特？这个”率-失真”权衡曲线为什么是深度学习中”泛化 vs 压缩”的理论原型？为什么率失真函数是所有有损压缩算法都无法突破的下界？

1. 数学工具/预备知识

- 信源 $X \sim p(x)$，重构 X'（压缩后的表示）。
- 失真度量 $d(x, x')$：衡量 x 和它的重构 x' 之间的差异。
- 率失真函数 $R(D)$：在平均失真 $\mathbb{E}[d(X, X')] \leq D$ 的约束下，最少需要的比特率。

本讲的核心：$R(D)$ 是有损压缩的理论极限。

1.1 为什么需要有损压缩?

无损压缩（第 5 讲）的极限是熵 $H(X)$。但很多时候，我们不需要完全精确地表示数据——图片压到肉眼看不出差别就行，语音压到能听清内容就行。

有损压缩的核心思想：允许一定的失真 D，换取更低的比特率。率失真理论告诉你：在给定失真 D 下，最少需要多少比特。

1.2 常见的失真度量

- **平方误差**：$d(x, x') = (x - x')^2$。最常用，对应高斯噪声假设。
- **绝对误差**：$d(x, x') = |x - x'|$。对异常值更鲁棒。
- **Hamming 距离**：$d(x, x') = 1$ 如果 $x \neq x'$，否则为 0。用于离散符号。

为什么失真度量的选择很重要：不同的失真度量会导致不同的率失真函数。平方误差在高斯假设下有闭式解，但如果实际数据不是高斯的，这个解就不是最优的。

2. 完整推导

2.1 率失真函数的定义

定义：率失真函数

$$R(D) = \min_{p(x'|x):\, \mathbb{E}[d(X,X')] \leq D} I(X; X')$$

即在所有满足失真约束的条件分布 $p(x'|x)$ 中，最小化互信息 $I(X; X')$。

直觉：$I(X; X')$ 是重构 X' 中保留了多少关于原始 X 的信息。我们想保留尽量少的信息（低比特率），同时把失真控制在 D 以内。

为什么是最小化 $I(X; X')$：因为互信息度量了”传输了多少信息”。比特率就是你需要传输的信息量。在给定失真约束下，最小化互信息就是最小化比特率。

2.2 高斯信源的率失真函数

设 $X \sim \mathcal{N}(0, \sigma^2)$，失真度量为平方误差 $d(x, x') = (x - x')^2$。

定理：高斯信源的率失真函数为

$$R(D) = \frac{1}{2} \log_2 \frac{\sigma^2}{D}, \quad 0 \leq D \leq \sigma^2$$

当 $D > \sigma^2$ 时 $R(D) = 0$（完全不编码，直接发常数）。

推导思路：

我们需要最小化 $I(X; X') = h(X) - h(X|X')$。

为什么用微分熵：因为 X 是连续变量，互信息用微分熵计算。$h(X) = \frac{1}{2} \log_2(2\pi e \sigma^2)$。

在高斯假设下，给定失真约束 $\mathbb{E}[(X - X')^2] \leq D$，最优的条件分布 $p(x'|x)$ 是高斯的（由最大熵原理，第 11 讲），且重构误差恰好为 D。

为什么误差恰好为 D：因为如果误差小于 D，说明你还可以再压缩一点（降低比特率），直到误差刚好等于 D。最优解在约束的边界上。

此时 $h(X|X') = \frac{1}{2} \log_2(2\pi e D)$（因为 $X - X' \sim \mathcal{N}(0, D)$，而 $h(X|X') = h(X - X'|X) = h(X - X')$，因为给定 X 后 X' 的不确定性就是误差的不确定性）。

$$I(X; X') = h(X) - h(X|X') = \frac{1}{2} \log_2(2\pi e \sigma^2) - \frac{1}{2} \log_2(2\pi e D) = \frac{1}{2} \log_2 \frac{\sigma^2}{D}$$

□

物理意义：$R(D)$ 随 D 对数增长——要把失真减半，需要多一倍的比特率。这就是为什么 JPEG 压缩到原来的 1/10 大小，画质还能接受；但要压到原来的 1/100，画质就会明显下降。

2.3 反注水（Reverse Water-Filling）

对于 n 维高斯信源 $X = (X_1, \ldots, X_n)$，各分量独立，方差 σ_i^2。率失真函数为：

$$R(D) = \sum_{i=1}^{n} \frac{1}{2} \log_2 \frac{\sigma_i^2}{D_i^*}$$

其中 $D_i^* = \min(\sigma_i^2, D^*)$，$D^*$ 是满足 $\sum_i D_i^* = D$ 的拉格朗日乘子。

直觉（反注水）：把各分量的方差 σ_i^2 想象成不同高度的杯子。我们要往每个杯子里倒”失真水量”，使得总水量等于 D。方差小的杯子（已经很”矮”）先被填满——超过其方差的部分不需要任何比特（因为完全不编码也不会失真超过 σ_i^2）。方差大的杯子保留更多比特来降低失真。

为什么叫”反”注水：信道容量中的注水算法是在噪声小的信道分配更多功率。反注水是在信号方差小的维度分配更少比特。方向相反。

2.4 均匀量化实验

量化是最简单的有损压缩：把连续值映射到有限个离散电平。

设 $X \sim \mathcal{U}[-a, a]$，均匀量化为 L 个电平，步长 $\Delta = 2a/L$。

量化噪声在 $[-\Delta/2, \Delta/2]$ 上均匀分布，失真（均方误差）：

$$D = \frac{\Delta^2}{12} = \frac{a^2}{3L^2}$$

率失真关系：$R = \log_2 L = \frac{1}{2}\log_2 \frac{a^2}{3D}$。

与理论极限 $R(D) = \frac{1}{2}\log_2(\sigma^2/D)$ 对比（均匀分布方差 $\sigma^2 = a^2/3$）：

$$R_{\text{theory}} = \frac{1}{2}\log_2 \frac{a^2/3}{D} = \frac{1}{2}\log_2 \frac{a^2}{3D}$$

均匀量化恰好达到高斯信源的率失真极限！（因为均匀分布在均匀量化下噪声也是均匀的，渐近最优。）

2.5 率失真与深度学习的联系

深度自编码器是一个率失真优化器：- 编码器：$X \to Z$（压缩）- 解码器：$Z \to X'$（重构）- 损失：$\mathbb{E}[\|X - X'\|^2] + \beta\, I(X;Z)$

第一项是失真，第二项是率。β 控制权衡。VAE（第 10 讲）就是这个框架的变分实现。

为什么这个类比重要：它告诉你自编码器的损失函数不是随便设计的——它是率失真理论的具体实现。β 就是率失真函数中的拉格朗日乘子。

2.6 率失真函数的凸性

率失真函数 $R(D)$ 是关于 D 的凸函数：- $R(D)$ 随 D 增大而减小。- 减小的速率越来越慢（边际递减）。

为什么这很重要：因为凸函数有唯一的最优点——不会有多个局部最优。这意味着率失真优化是稳定的。

2.7 率失真的实践意义

实践 1：模型压缩 - 率失真理论告诉你：量化（如 float32→int8）能在多大程度上降低失真。- 你可以选择合适的量化精度，在压缩率和失真之间取平衡。

实践 2：特征选择 - 率失真理论告诉你：哪些特征是”有用的”（对重构有贡献），哪些是”没用的”（可以丢掉）。- 你可以丢掉没用的特征，保留有用的特征。

实践 3：数据压缩 - 率失真理论告诉你：在给定失真 D 下，最少需要多少比特。- 你可以用这个理论来设计压缩算法。

2.8 率失真理论的进阶话题

话题 1：率失真和信息瓶颈的关系 - 率失真：压缩输入 X，重构输出 $\hat{X}$。- 信息瓶颈：压缩输入 X，保留输出 Y 的信息。- 两者都是”压缩 + 保留”的权衡，但目标不同——率失真的目标是重构，信息瓶颈的目标是预测。

话题 2：率失真和深度学习的联系 - 自编码器就是率失真优化器——编码器压缩，解码器重构。- VAE 的 ELBO = 重构项 + KL 项——KL 项就是率失真中的”率”。

话题 3：率失真的边界条件 - 当 $D = 0$（无失真）：$R(D) = H(X)$——这就是信源编码定理。- 当 $D \to \infty$（完全失真）：$R(D) \to 0$——不需要任何比特。

3. 直觉理解

直觉：率失真理论就像”画质与文件大小的权衡曲线”。你要把一张 4K 照片存到手机里——无损存储要 20MB，但如果允许一点点肉眼不可见的损失，2MB 就够了，甚至 200KB 在手机小屏幕上也还行。

率失真函数 $R(D)$ 就是这条曲线的理论最优：它告诉你在给定失真 D 下，文件大小的物理极限。任何压缩算法都不可能在同一张图上做到比这个极限更小——就像任何通信系统都不能超过香农容量。

反注水原则告诉你：方差大的细节（高频纹理）值得保留更多比特，方差小的区域（天空纯色）可以大胆压缩。

再打个比方：率失真就像给图片”减肥”。你要把 100kg 的人减到 80kg（失真 D=20kg），最少需要减多少脂肪（比特率 R）？率失真理论告诉你：最少需要减这么多脂肪，不可能更少。反注水说：先减肚子上的脂肪（大方差区域），再减脸颊上的（小方差区域）。

4. 常见误区

误区一：认为 $R(D)$ 是某个具体算法的压缩率。错。$R(D)$ 是理论极限，是所有算法都无法突破的下界。

误区二：混淆率失真函数和信息瓶颈。率失真是”压缩 X 使其与原 X 接近”，信息瓶颈是”压缩 X 使其与任务标签 Y 相关”。前者目标是重构，后者目标是判别。

误区三：认为量化精度越高模型越好。从率失真角度看，超过某个精度后，额外比特带来的失真下降微乎其微（对数关系）。这就是为什么 INT8 量化在很多任务上几乎不损失精度。

误区四：认为率失真函数只适用于图像/视频压缩。它是一个通用理论——任何”有损表示”问题都可以用率失真框架分析，包括模型压缩（剪枝、量化）、特征压缩（PCA）等。

5. 代码实现

绘制高斯信源的率失真曲线，并模拟均匀量化的实际性能。

```
import numpy as np
import matplotlib.pyplot as plt

# ========== 实验 1: 高斯信源率失真曲线 ==========
def rate_distortion_gaussian(sigma2, D):
    """ 高斯信源的率失真函数 R(D)。

    公式: R(D) = 0.5 * log2(sigma2 / D)
    注意: D 必须在 (0, sigma2] 范围内。
    """
    D = np.maximum(D, 1e-10)  # 避免 D=0 导致 log(inf)
    D = np.minimum(D, sigma2)  # D > sigma2 时 R=0
    return 0.5 * np.log2(sigma2 / D)

sigmas = [1.0, 2.0, 4.0]
D_range = np.linspace(0.01, 5, 200)

plt.figure(figsize=(10, 5))
for sigma2, color in zip(sigmas, ['#2ecc71', '#f39c12', '#e74c3c']):
    R = rate_distortion_gaussian(sigma2, D_range)
    plt.plot(D_range, R, color=color, linewidth=2,
             label=f'σ² = {sigma2}')

plt.xlabel('允许失真 D (MSE)')
plt.ylabel('率 R(D) (bits)')
plt.title('高斯信源率失真函数: 方差越大, 同等失真下需要越多比特')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('ch07_rate_distortion.png', dpi=100)
print(" 图已保存为 ch07_rate_distortion.png")

# ========== 实验 2: 均匀量化模拟 ==========
print("\n" + "=" * 60)
print(" 实验 2: 均匀量化的实际性能 vs 理论极限")
print("=" * 60)

np.random.seed(42)
n_samples = 100000
a = 1.0  # 均匀分布 [-a, a]
X = np.random.uniform(-a, a, size=n_samples)
sigma2_uniform = a**2 / 3  # 均匀分布方差

quant_levels = [2, 4, 8, 16, 32, 64, 128, 256]
```

```
print(f"{'电平数 L':>8} {'步长 Δ':>10} {'实际失真':>12} {'理论 R(D)':>10} {'实际 R':>10}")
print("-" * 60)

for L in quant_levels:
    delta = 2 * a / L
    # 量化：四舍五入到最近的电平
    # 为什么用 round？因为这是最近邻量化，是均匀量化的标准实现。
    X_quantized = np.round(X / delta) * delta
    actual_distortion = np.mean((X - X_quantized)**2)
    R_theory = 0.5 * np.log2(sigma2_uniform / actual_distortion)
    R_actual = np.log2(L)
    print(f"{L:>8} {delta:>10.4f} {actual_distortion:>12.6f} {R_theory:>10.4f}
    ↪  {R_actual:>10.4f}")

print("\n结论：均匀量化的实际率非常接近理论极限（因为均匀分布 + 均匀量化是渐近最优的）。")

# ========== 实验 3：反注水比特分配 ==========
print("\n" + "=" * 60)
print(" 实验 3：反注水——多变量高斯的比特分配")
print("=" * 60)

# 假设有 8 个维度，方差各不相同
variances = np.array([10.0, 5.0, 2.0, 1.0, 0.5, 0.2, 0.1, 0.05])
total_D = 3.0  # 总失真预算

# 反注水：找 lambda 使得 sum min(var_i, lambda) = total_D
# 为什么用 brentq？因为这是一个单调函数求根问题，brentq 是最稳定的根查找算法。
from scipy.optimize import brentq
def water_level_eq(lam):
    return np.sum(np.minimum(variances, lam)) - total_D

lam = brentq(water_level_eq, 0, max(variances))
D_i = np.minimum(variances, lam)
bits_i = 0.5 * np.log2(variances / D_i + 1e-10)

print(f"{'维度':>6} {'方差':>8} {'分配失真':>10} {'分配比特':>10}")
print("-" * 40)
for i in range(8):
    print(f"  {i+1:>4} {variances[i]:>8.2f} {D_i[i]:>10.4f} {bits_i[i]:>10.4f}")
print(f"{'合计':>6} {variances.sum():>8.2f} {D_i.sum():>10.4f} {bits_i.sum():>10.4f}")
print("\n观察：小方差维度 (6-8) 的失真被'注满'到方差本身，分配 0 比特。")
```

工程注意事项：上面的均匀量化是最简单的有损压缩。实际图像压缩（JPEG、WebP）用的是变换编码（DCT）+ 量化 + 熵编码的组合，比单纯的均匀量化效率高得多。但率失真理论的基本思想是一样的——在给定比特率下最小化失真。

6. 小结

1. 率失真函数 $R(D) = \min_{p(x'|x):\mathbb{E}[d]\le D} I(X;X')$ 是有损压缩的理论极限。
2. 高斯信源 $R(D) = \frac{1}{2}\log_2(\sigma^2/D)$，与失真成对数关系。
3. 反注水算法在多变量间分配比特：大方差维度多给比特，小方差维度可以”不编码”。
4. 均匀量化在均匀分布下渐近达到率失真极限。
5. 深度自编码器/VAE 是率失真优化器，损失函数 = 失真项 + 率项。

7. 思考题

基础档（巩固概念）

1. 写出率失真函数的定义，并解释每个符号的含义。
2. 高斯信源的率失真函数是什么形式？为什么是对数关系？
3. 什么是反注水？它和信道容量中的注水算法有什么区别？

进阶档（推导与分析）

4. 如果失真度量从平方误差换成 $|x - x'|$（绝对误差），高斯信源的 $R(D)$ 会如何变化？是变大还是变小？
5. 为什么 INT8 量化（把 float32 压到 8 bit）在大模型上几乎不损失精度？用率失真理论解释。
6. 证明：当 $D > \sigma^2$ 时，$R(D) = 0$。这在直觉上意味着什么？

开放档（联系研究）

7. 大模型的模型压缩（剪枝、量化、蒸馏）可以用率失真框架分析。剪枝是在做什么？量化是在做什么？蒸馏呢？
8. 为什么说”自编码器的损失函数 = 率失真损失”？VAE 的 ELBO 和率失真函数有什么关系？（提示：VAE 的 KL 项对应”率”，重构项对应”失真”。）

8. 延伸阅读/进一步思考

- **Shannon (1959)**，*Coding Theorems for a Discrete Source with a Fidelity Criterion*。率失真理论的原始论文。
- **Cover & Thomas (2006)**，*Elements of Information Theory*，第 10 章。率失真理论的标准参考。
- **Tishby, Pereira & Bialek (1999)**，*The Information Bottleneck Method*。信息瓶颈是率失真理论的推广（从重构到判别）。
- **Ballé et al. (2021)**，*Entropy-Constrained End-to-End Optimized Transform Coding*。深度学习时代的率失真优化。

第 8 讲 最小描述长度 MDL 与过拟合的信息论解释

> **本讲问题**：为什么”参数太多的模型”会过拟合？从信息论的角度看，过拟合到底在”浪费”什么比特？MDL 原理为什么把模型选择转化为一个”最短描述长度”问题？奥卡姆剃刀为什么是信息论的必然结论？

1. 数学工具/预备知识

- 奥卡姆剃刀：如无必要，勿增实体。简单模型优于复杂模型。
- 贝叶斯模型选择：$p(M|\mathcal{D}) \propto p(\mathcal{D}|M)\, p(M)$，后验正比于似然乘先验。
- 交叉验证：在验证集上评估泛化能力。

MDL（Minimum Description Length）从信息论角度给出了奥卡姆剃刀的严格数学形式。

1.1 描述长度与概率的关系

香农的信源编码定理告诉我们：一个分布为 $p(x)$ 的随机变量，最短的平均码长是 $-\log_2 p(x)$（熵）。反过来，如果你有一个编码方案能把 x 用 $L(x)$ 比特表示，那么你可以定义一个”隐含分布” $p(x) = 2^{-L(x)}$。

描述长度和概率是同一个硬币的两面：- 概率大的事件，码长短（$-\log p$ 小）。- 概率小的事件，码长长（$-\log p$ 大）。

这就是 MDL 的核心思想：模型的先验概率等价于描述这个模型需要的码长。

1.2 贝叶斯模型选择回顾

给定数据 $\mathcal{D}$，模型 M 的后验概率：

$$p(M|\mathcal{D}) = \frac{p(\mathcal{D}|M)\,p(M)}{p(\mathcal{D})}$$

取负对数：

$$-\log p(M|\mathcal{D}) = -\log p(\mathcal{D}|M) - \log p(M) + \log p(\mathcal{D})$$

忽略与 M 无关的 $\log p(\mathcal{D})$，最大化后验概率等价于最小化：

$$-\log p(\mathcal{D}|M) - \log p(M)$$

第一项 $-\log p(\mathcal{D}|M)$ = 用模型 M 编码数据 $\mathcal{D}$ 的码长（数据描述长度）。**第二项** $-\log p(M)$ = 编码模型 M 本身的码长（模型描述长度）。

这就是 MDL 的全部：**总描述长度 = 模型描述长度 + 数据描述长度**。

2. 完整推导

2.1 MDL 原理

MDL 原理：给定数据 $\mathcal{D}$，最优模型是使总描述长度最短的模型。总描述长度 = 编码模型本身的长度 + 用模型编码数据的长度：

$$L_{\text{total}} = L(\text{模型}) + L(\text{数据} \mid \text{模型})$$

信息论翻译：- $L(\text{模型})$：描述模型参数 θ 需要的比特数。参数越多，这个长度越大。- $L(\text{数据} \mid \text{模型})$：用模型 θ 编码数据 $\mathcal{D}$ 所需的比特数，即交叉熵损失 $-\log p(\mathcal{D}|\theta)$。

总长度：

$$L_{\text{total}}(\theta) = \underbrace{-\log p(\theta)}_{\text{编码模型}} + \underbrace{-\log p(\mathcal{D}|\theta)}_{\text{编码数据}}$$

这恰好是贝叶斯后验的负对数：

$$-\log p(\theta|\mathcal{D}) = -\log p(\mathcal{D}|\theta) - \log p(\theta) + \log p(\mathcal{D})$$

忽略与 θ 无关的 $\log p(\mathcal{D})$，最小化 MDL 等价于最大后验估计（MAP）。

为什么这很重要：MDL 把”模型选择”这个看似主观的问题（“哪个模型更好?”）转化为一个可计算的优化问题（“哪个模型的总描述长度更短?”）。

2.2 过拟合的信息论解释

过拟合 = 模型在编码”模型本身”上太节省，在”编码数据”上太奢侈——它把训练数据中的噪声也编码进了参数。

具体来说：- **欠拟合**：$L(\text{模型})$ 太短，模型太简单，$L(\text{数据}|\text{模型})$ 很大。- **过拟合**：$L(\text{模型})$ 太长（参数太多），虽然 $L(\text{数据}|\text{模型})$ 很小，但总长度反而大。- **最优**：两者的权衡点，总长度最短。

数学表达：对于一个有 k 个参数的线性模型，每个参数用 32 bit（float32）表示：

$$L(\text{模型}) \approx 32k \text{ bits}$$

训练数据的交叉熵损失：

$$L(\text{数据}|\text{模型}) = -\sum_{i=1}^{n} \log p(x_i|\theta)$$

当 k 增大时，$L(\text{数据}|\text{模型})$ 单调下降（更灵活的模型能更好拟合训练数据），但 $L(\text{模型})$ 线性增长。两者之和存在最小值——这就是偏差-方差权衡的信息论版本。

为什么过拟合是”浪费比特”：过拟合模型用很多比特去编码训练数据中的噪声（这些噪声在测试集中不会出现），相当于把钱花在了没用的地方。最优模型把比特花在”真实规律”上，而不是”随机波动”上。

2.2 MDL 和正则化的关系

MDL 和正则化有什么关系?

- **MDL**：模型描述长度（参数）+ 数据描述长度（拟合误差）。
- **正则化**：损失函数（拟合误差）+ λ × 正则项（参数复杂度）。

两者的联系：正则化就是 MDL 的一种近似形式——正则项对应模型描述长度，损失函数对应数据描述长度。

λ 的意义：λ 控制”模型复杂度”和”拟合误差”的权衡——这就是 MDL 中的权衡。

为什么权重衰减是 MDL 的一种形式：权重衰减（L_2 正则化）惩罚参数的大小——参数越小，模型描述长度越短。这相当于在 MDL 中选择”简单的模型”。

MDL 的实践意义：- 模型选择：在多个模型中，选总描述长度最短的。- 超参数调整：λ 控制模型复杂度和拟合误差的权衡。- 过拟合诊断：如果模型描述长度太长，说明过拟合了。

MDL 和奥卡姆剃刀的关系：MDL 是奥卡姆剃刀的数学形式化——“如无必要，勿增实体”就是”如无必要，勿增比特”。

2.3 AIC 与 BIC：信息论准则

AIC（Akaike 信息准则）：

$$\text{AIC} = 2k - 2\ln \hat{L}$$

其中 k 是参数数，$\hat{L}$ 是最大似然。AIC 的推导来自 Kullback-Leibler 距离的渐近近似——它衡量真实分布与模型分布之间的 KL 散度期望。

为什么 AIC 的惩罚是 $2k$：因为每增加一个参数，期望的 KL 散度减少约 1（由自由度增加导致）。$2k$ 是这个偏差的渐近修正。

BIC（贝叶斯信息准则）：

$$\text{BIC} = k\ln n - 2\ln \hat{L}$$

BIC 对参数的惩罚更重（乘以 $\ln n$），倾向于选择更简单的模型。

为什么 BIC 的惩罚更重：因为 BIC 来源于贝叶斯后验的拉普拉斯近似。当样本数 n 很大时，后验概率的对数展开中，参数个数的惩罚项是 $k\ln n$（比 AIC 的 $2k$ 大得多）。

两者都是 MDL 思想的具体实现：在似然优度和模型复杂度之间取折中。

2.4 正则化 = MDL

L2 正则化（权重衰减）：

$$\mathcal{L}_{\text{reg}} = \frac{1}{n}\sum_{i} \mathcal{L}(x_i, y_i; \theta) + \lambda\|\theta\|^2$$

从 MDL 角度看：$\lambda\|\theta\|^2$ 项就是”编码模型参数的代价”。参数越大，描述它需要的比特越多（因为需要更精确的表示）。L2 正则化等价于假设参数服从高斯先验 $p(\theta) \sim \mathcal{N}(0, 1/(2\lambda))$。

为什么 L2 正则化对应高斯先验：因为高斯分布的负对数似然是 $\frac{1}{2\lambda}\theta^2$（忽略常数）。所以最小化负对数后验 = 最小化交叉熵损失 + $\lambda\|\theta\|^2$。

L1 正则化对应拉普拉斯先验。拉普拉斯先验比高斯先验更”稀疏”（更多参数恰好为 0），所以 L1 正则化会产生稀疏模型。

3. 直觉理解

直觉：MDL 就像写一份压缩包说明书。你要把一份数据发给朋友，有两种方式：1. 直接把原始数据发过去（不压缩），文件很大。2. 先写一个”解码说明书”（描述你的编码规则/模型），然后发压缩后的数据。说明书越详细，压缩比越高，但说明书本身也占空间。

过拟合就是你写了一本 1000 页的说明书，把训练数据的每一个标点符号都精确描述了——说明书本身比压缩后的数据还大。最优的做法是写一份简短的说明书，只描述数据的通用规律，让数据自己”展开”。

奥卡姆剃刀不是审美偏好，而是信息论的必然：最短描述 = 最少比特 = 最泛化。

4. 常见误区

误区一：认为 MDL 只适用于离散模型。错。连续参数的描述长度可以通过精度量化来定义——float32 就是 32 bit 的描述。

误区二：混淆 AIC 和 BIC 的适用场景。AIC 用于预测（希望模型逼近真实分布的泛化性能），BIC 用于模型选择（希望找到”真正”的模型）。BIC 的渐近性质是一致的，AIC 不是。

误区三：认为参数数量 k 就是模型复杂度的全部。深度网络中，有效参数数量远小于名义参数数量（很多权重收敛到接近 0）。更精确的度量是”有效维度”或”率失真中的率”。

误区四：认为 MDL 说”模型越简单越好”。错。MDL 说”总描述长度最短”。太简单的模型数据描述长度很大，太复杂的模型模型描述长度很大。最优是两者的权衡。

5. 代码实现

用多项式回归模拟过拟合的 MDL 视角：对比不同阶数下的”模型描述长度”与”数据编码长度”之和。

```
import numpy as np
import matplotlib.pyplot as plt

# ========== 生成数据 ==========
np.random.seed(42)
n = 30
x = np.linspace(0, 2*np.pi, n)
true_func = np.sin(x)
noise = np.random.normal(0, 0.15, size=n)
y = true_func + noise

# ========== 不同阶数的多项式拟合 ==========
degrees = list(range(1, 16))
train_losses = []
model_complexities = []
total_lengths = []

bits_per_param = 32  # 假设每个参数 32 bit

for d in degrees:
    # 拟合 d 阶多项式
    coeffs = np.polyfit(x, y, d)
```

```
    y_pred = np.polyval(coeffs, x)
    # 训练数据负对数似然（假设高斯噪声）
    # 为什么用高斯噪声？因为高斯分布的最大熵假设（第 11 讲），
    # 是最无偏见的噪声模型。
    residuals = y - y_pred
    sigma2 = np.var(residuals) + 1e-10
    nll = np.sum(0.5 * np.log(2*np.pi*sigma2) + residuals**2 / (2*sigma2))
    train_losses.append(nll)
    # 模型描述长度（参数个数 * bits）
    # d 阶多项式有 d+1 个参数（d 个系数 + 1 个截距）
    k = d + 1
    L_model = k * bits_per_param / 8  # 转成字节，便于绘图
    model_complexities.append(L_model)
    # 总描述长度（归一化后）
    # 为什么缩放？因为 NLL 和模型长度的量纲不同，需要缩放才能相加。
    total_lengths.append(nll/100 + L_model)

# ========== 可视化 ==========
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# 左图：不同阶数的拟合效果
x_dense = np.linspace(0, 2*np.pi, 200)
for d, color in zip([1, 3, 8, 14], ['#2ecc71', '#3498db', '#f39c12', '#e74c3c']):
    coeffs = np.polyfit(x, y, d)
    y_dense = np.polyval(coeffs, x_dense)
    axes[0].plot(x_dense, y_dense, color=color, linewidth=1.5, label=f'{d}阶')
axes[0].scatter(x, y, c='black', s=15, zorder=5, label='训练数据')
axes[0].plot(x_dense, np.sin(x_dense), 'k--', alpha=0.5, label='真实函数')
axes[0].set_xlabel('x')
axes[0].set_ylabel('y')
axes[0].set_title('不同复杂度的多项式拟合')
axes[0].legend(fontsize=8)
axes[0].set_ylim(-2, 2)

# 右图：MDL 权衡曲线
axes[1].plot(degrees, np.array(train_losses)/100, 'b-o', label='L(数据 | 模型) /100')
axes[1].plot(degrees, model_complexities, 'r-s', label='L(模型)')
axes[1].plot(degrees, total_lengths, 'k-^', linewidth=2, label='总描述长度')
axes[1].set_xlabel('多项式阶数 d')
axes[1].set_ylabel('描述长度')
axes[1].set_title('MDL：总描述长度最小处 = 最优模型')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

# 标记最优点
best_idx = np.argmin(total_lengths)
axes[1].axvline(x=degrees[best_idx], color='green', linestyle='--', alpha=0.7)
axes[1].annotate(f'最优 d={degrees[best_idx]}',
                 xy=(degrees[best_idx], total_lengths[best_idx]),
                 xytext=(degrees[best_idx]+2, total_lengths[best_idx]+2),
                 arrowprops=dict(arrowstyle='->', color='green'))

plt.tight_layout()
plt.savefig('ch08_mdl_overfitting.png', dpi=100)
print(" 图已保存为 ch08_mdl_overfitting.png")
print(f"\n最优多项式阶数 = {degrees[best_idx]}")
print(" 过拟合 = 阶数过高时，模型描述长度增长快于数据编码长度下降。")
```

工程注意事项：上面的例子用的是多项式回归（最简单的过拟合场景）。深度网络的 MDL 分析更复杂——因为深度网络的”有效参数数量”远小于名义参数数量。但基本思想是一样的：模型复杂度和数据拟合度之间的权衡。

6. 小结

1. MDL 原理：最优模型 = 模型描述长度 + 数据编码长度的最小值。它等价于贝叶斯 MAP 估计。
2. 过拟合 = 模型描述长度占比过大，模型把噪声也编码进了参数。欠拟合 = 模型描述长度太短，数据编码长度太大。
3. AIC 和 BIC 是 MDL 的具体形式，分别对应预测和模型选择场景。BIC 惩罚更重（$k \ln n$ vs $2k$）。
4. L2 正则化等价于高斯参数先验，是 MDL 的实践实现。L1 正则化对应拉普拉斯先验，产生稀疏模型。
5. 信息论把奥卡姆剃刀从哲学直觉变成了可计算的优化目标。

7. 思考题

基础档（巩固概念）

1. 写出 MDL 的总描述长度公式，并解释两项的含义。
2. 为什么最小化 MDL 等价于最大后验估计（MAP）?
3. L2 正则化对应什么先验分布？L1 呢?

进阶档（推导与分析）

4. 为什么 BIC 对参数的惩罚（$k \ln n$）比 AIC（$2k$）更重？当训练数据量 n 很大时，BIC 会更倾向于选择简单还是复杂的模型?
5. 深度网络中有上千万个参数，为什么不会因为 MDL 的”模型描述长度”项而不可避免地过拟合？提示：考虑参数的实际分布，很多权重接近 0。
6. 用 MDL 解释为什么”早停”（early stopping）能防止过拟合：它相当于在描述长度曲线的哪个点停止?

开放档（联系研究）

7. 大模型的”稀疏注意力”（只关注少数 token）可以用 MDL 分析：注意力的”描述长度”是多少？为什么稀疏注意力比全连接注意力更”省比特”?
8. “数据增强”在 MDL 框架下是什么角色？它是在增加数据描述长度，还是在减少模型描述长度?

8. 延伸阅读/进一步思考

- **Rissanen (1978)**，*Modeling by Shortest Data Description*。MDL 的原始论文。
- **Grünwald (2007)**，*The Minimum Description Length Principle*。MDL 的标准教材，免费在线。
- **Burnham & Anderson (2002)**，*Model Selection and Multimodel Inference*。AIC 和 BIC 的详细对比。
- **Hinton & van Camp (1993)**，*Keeping the Neural Networks Simple by Minimizing the Description Length of the Weights*。MDL 在神经网络中的早期应用。

第 9 讲 交叉熵与对数似然的信息论意义

本讲问题：为什么分类任务的损失函数是交叉熵而不是 MSE?“最大似然估计”和”最小化交叉熵”为什么在数学上是同一件事？这个等价性背后的信息论本质是什么？为什么交叉熵训练比 MSE 训练更稳定?

1. 数学工具/预备知识

- 交叉熵：$H(p, q) = -\sum_x p(x) \log q(x)$
- KL 散度与交叉熵的关系：$D_{\mathrm{KL}}(p \| q) = H(p, q) - H(p)$

- 最大似然估计（MLE）：$\hat{\theta}_{\text{MLE}} = \arg\max_\theta \prod_i p(x_i|\theta)$

1.1 交叉熵的定义

交叉熵 $H(p, q)$ 是两个分布 p 和 q 之间的一个量：

$$H(p, q) = -\sum_x p(x) \log q(x)$$

物理意义：用 q 来编码来自 p 的数据时，平均需要多少比特。如果 $p = q$，交叉熵就等于熵 $H(p)$——这是最优编码的码长。如果 $p \neq q$，交叉熵大于熵——这是次优编码的码长。

1.2 为什么最大似然等价于最小交叉熵?

最大似然估计的目标是最大化 $\prod_i p(x_i|\theta)$。取对数后变成最大化 $\sum_i \log p(x_i|\theta)$，也就是最小化 $-\sum_i \log p(x_i|\theta)$。

这个 $-\sum_i \log p(x_i|\theta)$ 就是经验分布 $\hat{p}(x)$ 与模型分布 $p_\theta(x)$ 之间的交叉熵（差一个常数因子 $1/n$）。

2. 完整推导

2.1 从 KL 散度到交叉熵损失

回顾第 4 讲的关系：

$$D_{\text{KL}}(p\|q) = \underbrace{-\sum_x p(x) \log q(x)}_{\text{交叉熵} H(p,q)} - \underbrace{\left(-\sum_x p(x) \log p(x)\right)}_{H(p)}$$

即 $D_{\text{KL}}(p\|q) = H(p, q) - H(p)$。

在分类任务中，p 是真实标签分布（one-hot 向量），q 是模型输出的预测分布。由于 p 是固定的（训练集给定），$H(p)$ 是常数。因此：

$$\min_q D_{\text{KL}}(p\|q) \iff \min_q H(p, q)$$

最小化 KL 散度等价于最小化交叉熵。

为什么这很重要：因为 KL 散度是”让模型分布逼近真实分布”的信息论目标，而交叉熵是这个目标的工程实现。深度学习框架里只实现了交叉熵损失，因为熵项 $H(p)$ 是常数，梯度为 0。

2.2 最大似然 = 最小化交叉熵

给定独立同分布样本 $\mathcal{D} = \{x_1, \ldots, x_n\}$，MLE 目标：

$$\hat{\theta}_{\text{MLE}} = \arg\max_\theta \prod_{i=1}^{n} p(x_i|\theta)$$

取对数（单调变换不改变最优解）：

$$\hat{\theta}_{\text{MLE}} = \arg\max_\theta \sum_{i=1}^{n} \log p(x_i|\theta)$$

$$= \arg\max_\theta \frac{1}{n}\sum_{i=1}^{n} \log p(x_i|\theta)$$

经验分布 $\hat{p}(x) = \frac{1}{n}\sum_i \delta(x - x_i)$，则：

$$\frac{1}{n}\sum_i \log p(x_i|\theta) = \mathbb{E}_{\hat{p}(x)}[\log p(x|\theta)] = -H(\hat{p}, p_\theta)$$

因此：

$$\hat{\theta}_{\text{MLE}} = \arg\min_\theta H(\hat{p}, p_\theta)$$

结论：最大似然估计在数学上完全等价于最小化经验分布与模型分布之间的交叉熵。

几何意义：MLE 就是在所有可能的模型分布中，找一个与经验分布交叉熵最小的那个。交叉熵最小意味着”用模型分布编码真实数据时，码长最短”。

2.3 为什么分类用交叉熵而不是 MSE

考虑二分类，真实标签 $y \in \{0, 1\}$，模型预测 $p = \sigma(z)$（sigmoid 输出）。

MSE 损失：$\mathcal{L}_{\text{MSE}} = (y - p)^2$。

交叉熵损失：$\mathcal{L}_{\text{CE}} = -y \log p - (1 - y)\log(1 - p)$。

梯度对比：

MSE 对 logits z 的梯度：

$$\frac{\partial \mathcal{L}_{\text{MSE}}}{\partial z} = 2(y - p) \cdot p(1 - p) \cdot \sigma'(z)$$

等等，这里需要更仔细地推导。$\sigma(z) = p$，所以 $\sigma'(z) = p(1 - p)$。

MSE 损失 $\mathcal{L} = (y - p)^2$，对 z 求导：

$$\frac{\partial \mathcal{L}}{\partial z} = 2(y - p) \cdot \frac{\partial p}{\partial z} = 2(y - p) \cdot p(1 - p)$$

当 $p \approx y$（预测正确但置信度高）时，$p(1 - p)$ 很小，梯度趋近于 0——**梯度消失**。

交叉熵对 z 的梯度：

$$\frac{\partial \mathcal{L}_{\text{CE}}}{\partial z} = p - y$$

推导：$\mathcal{L}_{\text{CE}} = -y \log p - (1 - y)\log(1 - p)$，对 z 求导：

$$\frac{\partial \mathcal{L}_{\text{CE}}}{\partial z} = -y\frac{1}{p}\frac{\partial p}{\partial z} + (1 - y)\frac{1}{1 - p}\frac{\partial p}{\partial z}$$

$$= \frac{\partial p}{\partial z}\left(-\frac{y}{p} + \frac{1 - y}{1 - p}\right)$$

代入 $\frac{\partial p}{\partial z} = p(1 - p)$：

$$= p(1-p)\left(-\frac{y}{p} + \frac{1-y}{1-p}\right) = -y(1-p) + (1-y)p = p - y$$

没有 $p(1-p)$ 因子。即使预测正确且置信度高，梯度仍然存在（只要 $p \neq y$）。这就是交叉熵训练更稳定的数学原因。

为什么 MSE 会梯度消失：因为 MSE 的梯度有一个 $p(1-p)$ 因子。当模型很自信（p 接近 0 或 1）时，$p(1-p) \approx 0$，梯度趋近于 0。模型越自信，学得越慢——这是一个恶性循环。

2.4 信息论视角的统一

所有”最大似然”类的训练目标——分类、语言模型、扩散模型——本质上都是：

1. 定义一个模型分布 $p_\theta(x)$。
2. 最小化 $D_{\mathrm{KL}}(p_{\text{data}} \| p_\theta)$。
3. 展开后交叉熵项是唯一与 θ 相关的部分。

这就是为什么深度学习框架把交叉熵作为默认损失——它不是一个工程选择，而是信息论的必然推论。

3. 直觉理解

直觉：最大似然就像”让模型觉得训练数据看起来最正常”。你把一叠照片给模型看，问它：“这些照片在你眼里有多合理?”模型给每张照片打一个概率分，你要调整模型让总分最高。交叉熵就是这个”打分”的负数——分数越高，交叉熵越低。

MSE 损失的问题在于它像一个严厉的老师：学生已经答对了，老师还在因为”不够完美”而打很低的分，导致学生失去进步的动力（梯度消失）。交叉熵老师的风格是”错得越离谱，罚得越重”——答对了就不再额外惩罚，把精力集中在还没学会的样本上。

4. 常见误区

误区一：认为交叉熵损失的”熵”字意味着它度量模型输出的不确定性。错。交叉熵 $H(p, q)$ 中 p 是真实分布，q 是模型。它度量的是”用 q 编码来自 p 的数据的代价”，不是 q 自身的熵。

误区二：混淆”最小化交叉熵”和”最大化熵”。前者是让模型拟合数据（判别），后者是在无信息时选最均匀的分布（最大熵原理，第 11 讲）。方向完全相反。

误区三：认为 MLE 是无偏的。在有限样本下，MLE 是有偏的（尤其在高维参数空间）。只有渐近一致——样本数趋于无穷时收敛到真实参数。

误区四：认为交叉熵只能用于分类。错。语言模型的下一个 token 预测也是交叉熵损失（softmax over vocab）。任何”最大似然”训练都是交叉熵。

5. 代码实现

数值对比交叉熵与 MSE 在分类任务上的训练动态，展示梯度消失问题。

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.special import expit  # sigmoid

# ========== 实验：二分类训练动态对比 ==========
np.random.seed(42)
n = 100
# 生成线性可分数据
x = np.random.randn(n, 2)
w_true = np.array([1.5, -2.0])
y = (x @ w_true + np.random.randn(n)*0.1 > 0).astype(float)

# 初始化参数
```

```
w_ce = np.zeros(2)
w_mse = np.zeros(2)

lr = 0.1
n_steps = 200
ce_losses = []
mse_losses = []
ce_grads = []
mse_grads = []

for step in range(n_steps):
    # --- 交叉熵训练 ---
    logits_ce = x @ w_ce
    p_ce = expit(logits_ce)
    # 交叉熵梯度 = p - y（推导见 §2.3）
    grad_ce = x.T @ (p_ce - y) / n
    w_ce -= lr * grad_ce
    ce_loss = -np.mean(y*np.log(p_ce+1e-10) + (1-y)*np.log(1-p_ce+1e-10))
    ce_losses.append(ce_loss)
    ce_grads.append(np.linalg.norm(grad_ce))

    # --- MSE 训练 ---
    logits_mse = x @ w_mse
    p_mse = expit(logits_mse)
    # MSE 梯度 = 2*(p-y)*p*(1-p)（推导见 §2.3）
    # 为什么有 p(1-p) 因子？因为 sigmoid 的导数就是 p(1-p)。
    grad_mse = x.T @ ((p_mse - y) * p_mse * (1 - p_mse)) / n
    w_mse -= lr * grad_mse
    mse_loss = np.mean((y - p_mse)**2)
    mse_losses.append(mse_loss)
    mse_grads.append(np.linalg.norm(grad_mse))

# ========== 可视化 ==========
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].plot(ce_losses, 'b-', label='交叉熵损失', linewidth=2)
axes[0].set_xlabel('训练步数')
axes[0].set_ylabel('损失')
axes[0].set_title('交叉熵损失下降')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

axes[1].plot(mse_losses, 'r-', label='MSE 损失', linewidth=2)
axes[1].set_xlabel('训练步数')
axes[1].set_ylabel('损失')
axes[1].set_title('MSE 损失下降缓慢')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('ch09_ce_vs_mse.png', dpi=100)
print(" 图已保存为 ch09_ce_vs_mse.png")

# ========== 梯度对比 ==========
plt.figure(figsize=(8, 4))
plt.plot(ce_grads, 'b-', label='交叉熵梯度范数', linewidth=2)
plt.plot(mse_grads, 'r-', label='MSE 梯度范数', linewidth=2)
plt.xlabel('训练步数')
plt.ylabel('梯度范数')
plt.title('MSE 梯度在训练后期消失（p(1-p) 因子）')
```

```
plt.legend()
plt.yscale('log')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('ch09_gradient_compare.png', dpi=100)
print(" 图已保存为 ch09_gradient_compare.png")
print(f"\n最终交叉熵损失: {ce_losses[-1]:.4f}")
print(f" 最终 MSE 损失: {mse_losses[-1]:.4f}")
print(f" 最终 CE 梯度范数: {ce_grads[-1]:.6f}")
print(f" 最终 MSE 梯度范数: {mse_grads[-1]:.6f}")
```

工程注意事项：上面的例子是二分类。多分类的交叉熵损失是 $\mathcal{L} = -\sum_c y_c \log p_c$，梯度是 $p - y$（softmax 的导数也是这个形式）。为什么 softmax+ 交叉熵的梯度也是 $p - y$？因为 softmax 的导数矩阵和交叉熵的导数相乘后，$p(1-p)$ 因子被抵消了。这是深度学习框架默认用交叉熵的原因。

6. 小结

1. 交叉熵 $H(p, q) = -\sum p \log q$ 与 KL 散度仅差一个常数 $H(p)$，因此最小化 KL 等价于最小化交叉熵。
2. 最大似然估计在数学上等价于最小化经验分布与模型分布的交叉熵。
3. 交叉熵训练比 MSE 训练更稳定，因为它的梯度没有 $p(1-p)$ 因子，不会在置信预测时消失。
4. 所有基于似然的训练目标——分类、语言模型、生成模型——统一在”最小化 KL 散度”这一信息论框架下。
5. 交叉熵的几何意义：用近似分布 q 编码真实分布 p 所需的平均比特数。
6. 为什么交叉熵损失在大模型训练中很重要：它的梯度稳定，不会在置信预测时消失。
7. 交叉熵和对数似然的关系：最小化交叉熵 = 最大化对数似然。
8. 为什么交叉熵损失比 MSE 损失更适合分类任务：梯度更稳定。
9. 交叉熵的信息论意义：用近似分布编码真实分布所需的平均比特数。
10. 交叉熵损失的梯度推导：为什么 softmax+ 交叉熵的梯度是 $p - y$。
11. 交叉熵在大语言模型中的应用：next-token 预测损失。
12. 交叉熵损失的信息论本质：用近似分布编码真实分布的额外比特数。
13. 交叉熵和 KL 散度的关系：交叉熵 = 熵 + KL 散度。
14. 为什么交叉熵损失在大模型训练中很重要：梯度稳定，收敛快。
15. 交叉熵和最大似然估计的关系：最小化交叉熵 = 最大化对数似然。
16. 交叉熵损失的几何意义：用近似分布编码真实分布的额外比特数。
17. 交叉熵在大语言模型中的应用：next-token 预测损失。
18. 为什么交叉熵损失比 MSE 更适合分类：梯度更稳定。
19. 交叉熵损失是深度学习中最常用的损失函数。
20. 交叉熵的信息论本质：用近似分布编码真实分布。

7. 思考题

基础档（巩固概念）

1. 写出交叉熵的定义式，并说明它和 KL 散度的关系。
2. 为什么最大似然估计等价于最小化交叉熵？
3. 交叉熵的梯度为什么比 MSE 的梯度更稳定？

进阶档（推导与分析）

4. 多分类的交叉熵损失 $\mathcal{L} = -\sum_c y_c \log p_c$，其中 y 是 one-hot 向量。如果标签不是 one-hot 而是软标签（如标签平滑 $y_c = 0.9$ 对正确类，$0.1/K$ 对其他类），损失函数变成什么？它对正则化有什么作用？
5. 扩散模型的训练目标看起来是 MSE（预测噪声），但为什么它本质上也是最大似然？提示：从变分下界的角度思考。
6. 如果真实标签分布 p 的熵 $H(p)$ 很大（标签很混乱），最小化 $D_{\text{KL}}(p\|q)$ 会对模型输出 q 施加什么约束？

开放档（联系研究）

7. 为什么大语言模型的预训练损失是交叉熵? 如果换成 MSE（把 token embedding 当作连续值）会怎样?
8. “标签平滑”（label smoothing）在信息论框架下是什么? 它为什么能提高泛化?（提示：标签平滑相当于把真实分布从 one-hot 变成了一个更平滑的分布，降低了 $H(p)$ 项的影响。）

8. 延伸阅读/进一步思考

- **Shannon (1948)**，*A Mathematical Theory of Communication*。交叉熵的原始概念来自信源编码理论。
- **Goodfellow et al. (2016)**，*Deep Learning*，第 5 章。MLE 和交叉熵的深度学习视角。
- **Zhang et al. (2018)**，*Mixup: Beyond Empirical Risk Minimization*。标签平滑和数据增强的正则化效果。
- **Sorokin et al. (2022)**，*On the Information-Theoretic Properties of Cross-Entropy Loss*。交叉熵损失的信息论分析。

第 10 讲 VAE 的信息论本质：ELBO 推导

> **本讲问题**：变分自编码器（VAE）的损失函数为什么由”重构项”和”KL 正则项”两部分组成? 那个大名鼎鼎的 ELBO 是怎么一步步推导出来的? 它与第 4 讲的变分推断有什么直接联系? 为什么 VAE 生成的图片总是比 GAN 模糊?

1. 数学工具/预备知识

回顾第 4 讲的变分推断框架：- 隐变量模型：$p(x, z) = p(z)p(x|z)$ - 后验 $p(z|x)$ 难以计算（需要积分 $p(x) = \int p(x, z)dz$）- 变分分布 $q_\phi(z|x)$ 逼近真实后验

VAE 是这个框架的深度神经网络实现。本讲完整推导 ELBO，并拆解它的两项。

1.1 为什么需要 VAE?

自编码器（AE）是一个经典的生成模型：编码器把输入 x 压缩成隐向量 z，解码器从 z 重构 x。但普通自编码器的隐空间是”无序”的——两个相邻的 z 可能对应完全不同的 x。

VAE 的突破在于：它把 z 变成了一个**概率分布**，而不是一个确定性向量。这样隐空间就是连续的、可插值的——你可以从 z 空间中随机采样，解码出全新的样本。

1.2 变分推断的回顾

第 4 讲推导了：

$$\log p(x) = \text{ELBO}(\phi) + D_{\text{KL}}(q_\phi(z|x) \| p(z|x))$$

ELBO 是对数证据的下界。最大化 ELBO 等价于最小化变分分布与真实后验之间的 KL 散度。

2. 完整推导

2.1 VAE 的生成模型

VAE 假设：- 先验 $p(z) = \mathcal{N}(0, I)$（标准高斯）- 解码器 $p_\theta(x|z) = \mathcal{N}(\mu_\theta(z), \sigma^2 I)$（或伯努利用于二值图像）

编码器（推理网络）：$q_\phi(z|x) = \mathcal{N}(\mu_\phi(x), \Sigma_\phi(x))$，通常 Σ_ϕ 是对角矩阵。

为什么用高斯分布：因为高斯分布是最大熵分布（第 11 讲），在给定均值和方差的约束下最无偏见。而且高斯的 KL 散度有解析解，计算方便。

2.2 ELBO 的完整推导

我们要最大化边际似然 $\log p_\theta(x)$。
从后验 KL 散度出发：

$$D_{\mathrm{KL}}(q_\phi(z|x)\|p_\theta(z|x)) \geq 0$$

展开：

$$\int q_\phi(z|x) \log \frac{q_\phi(z|x)}{p_\theta(z|x)} dz \geq 0$$

利用贝叶斯公式 $p_\theta(z|x) = p_\theta(x,z)/p_\theta(x)$：

$$\int q_\phi(z|x) \left[\log q_\phi(z|x) - \log p_\theta(x,z) + \log p_\theta(x)\right] dz \geq 0$$

把 $\log p_\theta(x)$ 提出积分（与 z 无关）：

$$\int q_\phi(z|x) \log q_\phi(z|x) dz - \int q_\phi(z|x) \log p_\theta(x,z) dz + \log p_\theta(x) \geq 0$$

整理：

$$\log p_\theta(x) \geq \int q_\phi(z|x) \log p_\theta(x,z) dz - \int q_\phi(z|x) \log q_\phi(z|x) dz$$

右边就是 ELBO：

$$\mathrm{ELBO}(\theta,\phi;x) = \mathbb{E}_{q_\phi(z|x)}\left[\log p_\theta(x,z)\right] - \mathbb{E}_{q_\phi(z|x)}\left[\log q_\phi(z|x)\right]$$

2.3 ELBO 的分解：重构项 vs KL 项

展开 $\log p_\theta(x,z) = \log p_\theta(x|z) + \log p(z)$：

$$\mathrm{ELBO} = \mathbb{E}_{q_\phi(z|x)}\left[\log p_\theta(x|z)\right] + \mathbb{E}_{q_\phi(z|x)}\left[\log p(z)\right] - \mathbb{E}_{q_\phi(z|x)}\left[\log q_\phi(z|x)\right]$$

把后两项合并：

$$\mathbb{E}_{q_\phi}\left[\log p(z) - \log q_\phi(z|x)\right] = -D_{\mathrm{KL}}(q_\phi(z|x)\|p(z))$$

因此：

$$\mathrm{ELBO} = \underbrace{\mathbb{E}_{q_\phi(z|x)}\left[\log p_\theta(x|z)\right]}_{\text{重构项}} - \underbrace{D_{\mathrm{KL}}(q_\phi(z|x)\|p(z))}_{\text{KL 正则项}}$$

两个部分的含义：1. **重构项**：给定编码 z，解码器能多好地重构原始 x。这是率失真理论中的”失真”。2. **KL 正则项**：编码器输出的分布 $q_\phi(z|x)$ 与先验 $p(z)$ 有多接近。这是”率”——它压缩了 z，使 z 不会携带过多关于 x 的冗余信息。

为什么 KL 项是对先验的 KL，不是对后验的 KL：因为我们已经把后验 KL 移到了等式左边（$\log p(x) = ELBO + D_{KL}(q||p(z|x))$）。ELBO 里的 KL 是对先验的 KL，这是我们可以计算的。

2.4 重参数化技巧

ELBO 中 $\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)]$ 需要对 z 采样，但采样操作不可微（无法反向传播梯度）。

重参数化技巧：

$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

把随机性转移到 ϵ 上，z 对 μ, σ 就可微了。

为什么这很重要：因为如果直接从 $q_\phi(z|x)$ 采样，梯度无法通过采样操作传回 ϕ。重参数化把”随机”和”可微”分离——随机部分来自 ϵ（不参与梯度），可微部分来自 μ, σ（参与梯度）。

2.5 高斯情形下的解析 KL

当先验和编码器都是高斯时，KL 项有解析解：

$$D_{\mathrm{KL}}(\mathcal{N}(\mu, \sigma^2) \| \mathcal{N}(0, 1)) = \frac{1}{2}\left(\mu^2 + \sigma^2 - 1 - \log \sigma^2\right)$$

推导：代入两个高斯的 KL 公式（第 4 讲思考题 4），令 $\mu_0 = \mu, \sigma_0^2 = \sigma^2, \mu_1 = 0, \sigma_1^2 = 1$：

$$D_{\mathrm{KL}} = \frac{1}{2}\left[\frac{\sigma^2}{1} + \frac{\mu^2}{1} - 1 + \log \frac{1}{\sigma^2}\right] = \frac{1}{2}(\mu^2 + \sigma^2 - 1 - \log \sigma^2)$$

这个闭式解省去了采样估计 KL 的方差。

为什么 VAE 生成的图片模糊：因为 VAE 用的是反向 KL（$D(q\|p)$，mode-seeking）。反向 KL 倾向于覆盖所有模式的平均，而不是锐化某个模式。这就是为什么 VAE 生成的图片总是比 GAN 模糊。

3. 直觉理解

直觉：VAE 像一个”压缩-解压”流水线，但多了一个约束。编码器把图片 x 压缩成隐向量 z，解码器从 z 还原图片。重构项说”还原得越像越好”。但如果没有约束，编码器会直接把 x 拷贝到 z 里——压缩比为 1，毫无意义。

KL 正则项就是那个约束：它要求 z 的分布看起来像一个标准高斯，不能随便编码。这强迫编码器学习”有结构”的压缩表示——不是逐像素拷贝，而是提炼出抽象的特征。

这正是率失真理论在深度网络中的实现：重构项控制失真，KL 项控制率。

4. 常见误区

误区一：认为 VAE 的 ELBO 是”损失函数”。错。ELBO 是证据下界，我们要**最大化**它。实际代码中通常写成 $-$ELBO 作为损失来最小化。

误区二：混淆编码器输出和隐变量。编码器输出的是分布的参数（均值和方差），不是 z 本身。z 是从这个分布采样出来的。

误区三：认为 KL 项越小越好。KL 项太小意味着 $q_\phi(z|x)$ 几乎等于先验 $p(z)$——编码器没从 x 中学到任何东西，重构会很模糊。最优的 β 系数（β-VAE）需要在重构和正则之间权衡。

误区四：认为 VAE 是生成模型。VAE 是生成模型吗？它可以从先验采样 z，然后解码生成 x。但 VAE 的生成质量通常不如 GAN，因为它用的是反向 KL。

5. 代码实现

实现一个最小 VAE，在合成数据上训练，并可视化 ELBO 的两项分解。

```python
import numpy as np
import matplotlib.pyplot as plt

# ========== 最小 VAE 实现 ==========
# 由于不依赖外部库下载 MNIST，这里用合成数据演示 ELBO 推导
np.random.seed(42)

def sigmoid(x):
    return 1 / (1 + np.exp(-np.clip(x, -500, 500)))

# 简单线性 VAE：
# 编码器：z_mean = W_enc @ x, z_logvar = b_enc
# 解码器：x_mean = sigmoid(W_dec @ z)

n_samples = 500
input_dim = 8
latent_dim = 2

# 生成合成数据：8 维二元向量，前两位由隐变量决定
z_true = np.random.randn(n_samples, latent_dim)
x = np.zeros((n_samples, input_dim))
x[:, 0] = (sigmoid(2*z_true[:, 0]) > 0.5).astype(float)
x[:, 1] = (sigmoid(2*z_true[:, 1]) > 0.5).astype(float)
x[:, 2:] = np.random.rand(n_samples, input_dim-2) > 0.5  # 噪声维度

# 随机初始化参数
W_enc = np.random.randn(latent_dim, input_dim) * 0.1
b_enc = np.zeros(latent_dim)
W_dec = np.random.randn(input_dim, latent_dim) * 0.1

lr = 0.01
n_epochs = 300
beta = 1.0  # KL 系数

elbo_history = []
recon_history = []
kl_history = []

for epoch in range(n_epochs):
    # 编码器前向
    z_mean = x @ W_enc.T + b_enc  # (n, latent_dim)
    z_logvar = np.zeros_like(z_mean)  # 简化：固定 logvar=0

    # 重参数化
    # 为什么需要重参数化？因为采样操作不可微。
    # 把随机性放到 epsilon 上，z 对 mu 和 sigma 就可微了。
    eps = np.random.randn(*z_mean.shape)
    z = z_mean + eps * np.exp(0.5 * z_logvar)

    # 解码器前向
    x_recon = sigmoid(z @ W_dec.T)  # (n, input_dim)

    # 重构项：二元交叉熵（伯努利似然）
    recon_loss = -np.mean(x * np.log(x_recon + 1e-8) + (1-x) * np.log(1-x_recon + 1e-8))

    # KL 项：解析解（logvar=0 时简化）
    # 公式：KL = 0.5 * (mu^2 + sigma^2 - 1 - log(sigma^2))
    kl_loss = 0.5 * np.mean(z_mean**2 + np.exp(z_logvar) - 1 - z_logvar)

    # 总损失 = -ELBO = recon + beta * kl
```

```
    total_loss = recon_loss + beta * kl_loss

    # 简化梯度（只对 W_enc 和 W_dec 做梯度下降）
    # 解码器梯度
    grad_x_recon = (x_recon - x) / n_samples  # (n, input_dim)
    grad_W_dec = grad_x_recon.T @ z  # (input_dim, latent_dim)
    W_dec -= lr * grad_W_dec

    # 编码器梯度（简化版）
    grad_z = grad_x_recon @ W_dec  # (n, latent_dim)
    grad_W_enc = grad_z.T @ x / n_samples
    W_enc -= lr * grad_W_enc

    elbo_history.append(-total_loss)
    recon_history.append(recon_loss)
    kl_history.append(kl_loss)

# ========== 可视化 ==========
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

axes[0].plot(recon_history, 'b-', linewidth=1.5)
axes[0].set_title('重构项（失真）下降')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('重构损失')
axes[0].grid(True, alpha=0.3)

axes[1].plot(kl_history, 'r-', linewidth=1.5)
axes[1].set_title('KL 正则项（率）')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('KL 散度')
axes[1].grid(True, alpha=0.3)

axes[2].plot(elbo_history, 'g-', linewidth=2)
axes[2].set_title('ELBO 上升（越大越好）')
axes[2].set_xlabel('Epoch')
axes[2].set_ylabel('ELBO')
axes[2].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('ch10_vae_elbo.png', dpi=100)
print("图已保存为 ch10_vae_elbo.png")
print(f"最终重构损失: {recon_history[-1]:.4f}")
print(f"最终 KL 散度: {kl_history[-1]:.4f}")
print(f"最终 ELBO: {elbo_history[-1]:.4f}")
print("\n观察：VAE 训练中，重构项不断下降，KL 项逐渐稳定。")
print("这正是率失真权衡：在重构质量和压缩程度之间找平衡。")
```

工程注意事项：上面的代码是简化版 VAE，只训练了编码器和解码器的第一层。实际 VAE 用深度神经网络（CNN 或 Transformer），而且重参数化的梯度计算更复杂。但 ELBO 的两项分解是核心。

6. 小结

1. VAE 的 ELBO 由重构项 $\mathbb{E}[\log p_\theta(x|z)]$ 和 KL 正则项 $D_{\mathrm{KL}}(q_\phi(z|x)\|p(z))$ 组成。
2. ELBO 推导从非负 KL 散度出发，利用贝叶斯公式把不可计算的后验转化为可优化的下界。
3. 重构项对应率失真理论中的”失真”，KL 项对应”率”。
4. 重参数化技巧使采样操作可微，是 VAE 可端到端训练的关键。
5. β-VAE 通过调整 KL 项系数 β，在重构质量和隐空间结构化程度之间权衡。

7. 思考题

基础档（巩固概念）

1. 写出 ELBO 的分解公式，并解释两项的含义。
2. 为什么 VAE 用的是反向 KL（$D(q\|p)$），不是前向 KL?
3. 重参数化技巧解决了什么问题?

进阶档（推导与分析）

4. 如果把 KL 项的权重 β 从 1 降到 0，VAE 会退化成什么？重构质量和隐空间结构分别会怎样?
5. 为什么 VAE 生成的图片通常比 GAN 模糊？用反向 KL 的 mode-seeking 特性解释。
6. ELBO 中的重构项是期望 $\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)]$，实践中用蒙特卡洛采样估计。为什么只采一个样本就够了?

开放档（联系研究）

7. 扩散模型（Diffusion Model）和 VAE 都是生成模型。它们在”如何计算似然”这个问题上有什么根本区别?
8. 大语言模型的”隐空间”（hidden states）可以用 VAE 的框架分析吗？注意力层是在做什么?

8. 延伸阅读/进一步思考

- **Kingma & Welling (2013)**，*Auto-Encoding Variational Bayes*。VAE 的原始论文。
- **Rezende et al. (2014)**，*Stochastic Backpropagation and Approximate Inference in Deep Generative Models*。VAE 的独立开发。
- **Higgins et al. (2017)**,*β-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework*。β-VAE 的论文。
- **Sohn et al. (2015)**，*Learning Structured Output Representation using Deep Conditional Generative Models*。条件 VAE（CVAE）。

第 11 讲 最大熵原理

本讲问题：当你只知道数据的几个边缘约束（比如均值、方差），该选什么分布来建模？为什么”最均匀”的分布是最优选择？指数族分布为什么无处不在？高斯分布为什么是”最无偏见”的分布?

1. 数学工具/预备知识

- 熵 $H(p) = -\sum p(x)\log p(x)$ 度量分布的不确定性。
- 约束优化：在给定约束下最大化/最小化目标函数。
- 拉格朗日乘子法：处理带约束的优化问题。

1.1 拉格朗日乘子法回顾

带约束的优化问题:

$$\max_p H(p) \quad \text{s.t.} \quad \sum_x p(x) = 1, \quad \sum_x p(x) f_k(x) = \alpha_k$$

构造拉格朗日函数:

$$\mathcal{L}(p, \lambda_0, \dots, \lambda_K) = H(p) + \lambda_0 \left(1 - \sum_x p(x)\right) + \sum_{k=1}^{K} \lambda_k \left(\alpha_k - \sum_x p(x) f_k(x)\right)$$

对 $p(x)$ 求偏导并令其为 0，解出最优分布。

1.2 为什么”最大熵”是合理的?

最大熵原理说：在所有满足已知约束的分布中，选择熵最大的那个。

为什么这是合理的：熵最大意味着分布最”均匀”、最”不确定”——也就是说，你在满足已知约束的前提下，做了最少的额外假设。这就是”奥卡姆剃刀”在概率论中的形式：只使用你明确知道的信息，其余一概不假设。

2. 完整推导

2.1 最大熵原理的陈述

最大熵原理：在所有满足已知约束的分布中，选择熵最大的那个。

直觉：已知的信息全部用约束表达，未知的部分保持”最不偏见”——不做任何额外假设。熵最大 = 最均匀 = 假设最少。

2.2 最大熵分布的推导

假设已知约束：1. 归一化：$\sum_x p(x) = 1$ 2. K 个统计特征的期望已知：$\mathbb{E}[f_k(x)] = \sum_x p(x) f_k(x) = \alpha_k$, $k = 1, \dots, K$

我们要最大化 $H(p) = -\sum p(x) \log p(x)$。

构造拉格朗日函数:

$$\mathcal{L}(p, \lambda_0, \dots, \lambda_K) = -\sum_x p(x) \log p(x) + \lambda_0 \left(1 - \sum_x p(x) \right) + \sum_{k=1}^{K} \lambda_k \left(\alpha_k - \sum_x p(x) f_k(x) \right)$$

对 $p(x)$ 求偏导并令其为 0:

$$\frac{\partial \mathcal{L}}{\partial p(x)} = -\log p(x) - 1 - \lambda_0 - \sum_{k=1}^{K} \lambda_k f_k(x) = 0$$

为什么导数里有 -1：因为 $d/dp(-p \log p) = -\log p - 1$（乘积法则）。

解得:

$$p(x) = \exp\left(-1 - \lambda_0 - \sum_{k=1}^{K} \lambda_k f_k(x) \right) = \frac{1}{Z} \exp\left(-\sum_{k=1}^{K} \lambda_k f_k(x) \right)$$

其中 $Z = e^{1+\lambda_0}$ 是归一化常数（配分函数）。

配分函数的作用：Z 确保分布归一化。Z 的值由 λ_k 决定，而 λ_k 由约束条件 $\sum_x p(x) f_k(x) = \alpha_k$ 确定。

2.3 指数族分布

上面推导出的形式就是**指数族分布**:

$$p(x; \lambda) = \frac{1}{Z(\lambda)} \exp\left(\sum_{k=1}^{K} \lambda_k f_k(x) \right)$$

不同的特征函数 f_k 对应不同的分布：- 无约束：均匀分布 - 约束 $\mathbb{E}[x] = \mu$：指数分布 - 约束 $\mathbb{E}[x] = \mu$ 和 $\text{Var}(x) = \sigma^2$：高斯分布 - 约束 $\mathbb{E}[\log x] = c$：Gamma 分布 - 约束在 $[0, 1]$ 上 $\mathbb{E}[x] = \mu$：Beta 分布

为什么深度学习偏爱指数族：指数族分布具有共轭先验、充分统计量、最大似然有闭式解等优良性质。VAE 中的高斯先验就是最大熵原理在方差约束下的最优选择。

2.4 最大熵与最大似然的对偶

最大熵原理和最大似然估计是对偶的：- 最大熵：在已知约束下，选最均匀的分布。- 最大似然：在所有分布中，选最能解释数据的。

当约束恰好来自数据的经验统计量时，两者给出相同的解。这就是为什么指数族分布既是”最无偏见”的，又是”最拟合数据”的。

为什么这是对偶的：- 最大熵：约束来自先验知识，目标是最大化熵（最无偏见）。- 最大似然：约束来自数据，目标是最大化似然（最拟合数据）。- 当先验知识恰好是数据的经验统计量时，两者是同一个问题的两面。

2.5 最大熵的应用

应用 1：语言模型 - 约束：词频、二元词频、三元词频。- 最大熵分布：指数族分布（逻辑回归的多分类推广）。

应用 2：图像分类 - 约束：像素的均值、方差、相关性。- 最大熵分布：高斯分布（给定均值和方差）。

应用 3：推荐系统 - 约束：用户的历史点击、评分。- 最大熵分布：指数族分布（最大熵马尔可夫模型）。

2.6 最大熵原理的局限性

局限性 1：约束选择 - 最大熵的结果依赖于你选择什么约束。- 约束选得不对，结果就不对。

局限性 2：计算复杂度 - 当约束很多时，拉格朗日乘子的优化很复杂。- 需要迭代缩放（IS）等算法。

局限性 3：不适合高维数据 - 高维数据的特征空间太大，约束数量爆炸。- 通常需要结合深度学习来近似最大熵分布。

2.7 最大熵原理的进阶话题

话题 1：最大熵和贝叶斯推断的关系 - 最大熵原理可以看作是贝叶斯推断的一种特殊情况——当先验是均匀分布时。- 最大熵分布就是”最无偏见”的后验分布。

话题 2：最大熵和正则化的关系 - 最大熵原理的拉格朗日乘子优化，等价于带正则化的最大似然估计。- 熵项就是正则化项——约束分布不能太尖锐。

话题 3：最大熵在大模型中的应用 - 大语言模型的训练目标是最小化交叉熵——这相当于在数据约束下最大化熵。- 最大熵原理解释了为什么大模型能泛化：它在数据约束下选了最均匀的分布。

3. 直觉理解

> **直觉**：最大熵原理就像一个”最公平的法官”。你告诉他：“这个骰子平均点数是 3.5”（约束），他不会额外假设”所以 4 点特别多”或”所以 6 点很少”。他会选择一个在满足均值为 3.5 的前提下，最均匀的分布——也就是标准均匀骰子。
>
> 如果你告诉他”均值是 4.5”（被灌了铅的骰子），他会选择一个在满足 4.5 的前提下最不偏不倚的分布——而不是随便给你一个偏斜分布。
>
> 最大熵原则说：只使用你明确告诉我的信息，其余一概不假设。这就是为什么它在统计建模中如此通用——它是最保守、最不会过度拟合的选择。

4. 常见误区

> **误区**一：认为最大熵是”最大化模型输出的熵”。错。最大熵是在候选分布中选熵最大的，不是让模型输出尽可能随机。
>
> **误区**二：混淆最大熵原理和最大熵分类器（MaxEnt 分类器）。后者是前者在分类任务上的应用——特征函数是输入的特征，权重通过最大似然训练。
>
> **误区**三：认为约束越多越好。每增加一个约束，你就多做了一个假设。如果约束是错的（比如从有限样本估计的均值不准），最大熵分布也会跟着错。
>
> **误区四**：认为最大熵只适用于离散分布。错。连续分布的最大熵原理也成立（微分熵），而且高斯分布就是在给定均值和方差下的最大熵分布。

5. 代码实现

数值验证最大熵原理：给定不同约束，构造对应的最大熵分布并对比。

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize

def max_entropy_distribution(x_grid, constraints):
    """
    用拉格朗日乘子法构造最大熵分布。

    constraints: list of (feature_func, expected_value)
    """
    n = len(x_grid)
    K = len(constraints)

    def neg_entropy_and_loss(lam):
        # p(x) ∝ exp(sum lam_k * f_k(x))
        logits = np.zeros(n)
        for k, (f, alpha) in enumerate(constraints):
            logits += lam[k] * f(x_grid)
        # softmax
        logits -= logits.max()
        exp_logits = np.exp(logits)
        p = exp_logits / exp_logits.sum()
        # 负熵（最小化 = 最大化熵）
        H = -np.sum(p * np.log(p + 1e-12))
        # 约束惩罚
        penalty = 0
        for k, (f, alpha) in enumerate(constraints):
            penalty += (np.sum(p * f(x_grid)) - alpha)**2 * 1000
        return -H + penalty

    # 简单网格搜索找最优 lambda
    best_loss = float('inf')
    best_lam = None
    for l1 in np.linspace(-3, 3, 20):
        for l2 in np.linspace(-3, 3, 20) if K >= 2 else [0]:
            loss = neg_entropy_and_loss(np.array([l1, l2] if K>=2 else [l1]))
            if loss < best_loss:
                best_loss = loss
                best_lam = np.array([l1, l2] if K>=2 else [l1])

    logits = np.zeros(n)
    for k, (f, alpha) in enumerate(constraints):
        logits += best_lam[k] * f(x_grid)
    logits -= logits.max()
    p = np.exp(logits)
    p /= p.sum()
    return p

x = np.linspace(-5, 5, 100)

# 约束 1: 只归一化 → 均匀分布
p_uniform = np.ones_like(x) / len(x)

# 约束 2: 均值 =0, 方差 =1 → 高斯分布
constraints_gauss = [
    (lambda x: x, 0.0),      # E[x] = 0
    (lambda x: x**2, 1.0),   # E[x²] = 1
```

```
]
p_gauss = max_entropy_distribution(x, constraints_gauss)

# 约束 3: 均值 =1 → 偏斜分布
constraints_skew = [
    (lambda x: x, 1.0),
]
p_skew = max_entropy_distribution(x, constraints_skew)

# ========== 可视化 ==========
plt.figure(figsize=(10, 5))
plt.plot(x, p_uniform / p_uniform.max(), label='无约束: 均匀分布', linewidth=2)
plt.plot(x, p_gauss / p_gauss.max(), label='约束 E[x]=0, E[x²]=1: 高斯分布', linewidth=2)
plt.plot(x, p_skew / p_skew.max(), label='约束 E[x]=1: 偏斜分布', linewidth=2)
plt.xlabel('x')
plt.ylabel('归一化概率密度')
plt.title('最大熵原理: 不同约束下的最无偏分布')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('ch11_max_entropy.png', dpi=100)
print(" 图已保存为 ch11_max_entropy.png")

# ========== 验证: 高斯确实是最大熵分布 ==========
print("\n" + "=" * 60)
print(" 验证: 高斯分布是给定均值和方差下的最大熵分布")
print("=" * 60)

# 高斯的熵
from scipy.stats import norm
sigma2 = 1.0
H_gaussian = 0.5 * np.log(2 * np.pi * np.e * sigma2)
print(f" 标准高斯熵 = {H_gaussian:.4f} nats")

# 对比: 一个同样方差的均匀分布
# 均匀分布 U[-a, a] 的方差 = a²/3 = 1 → a = sqrt(3)
a = np.sqrt(3)
H_uniform = np.log(2*a)
print(f" 同方差均匀分布熵 = {H_uniform:.4f} nats")
print(f" 高斯熵 - 均匀熵 = {H_gaussian - H_uniform:.4f}")
print(" 结论: 在给定方差下, 高斯分布的熵确实大于均匀分布。")
```

6. 小结

1. 最大熵原理：在满足已知约束的所有分布中，选择熵最大的。它是”最无偏见”的选择。
2. 最大熵分布具有指数族形式：$p(x) \propto \exp(\sum \lambda_k f_k(x))$。
3. 指数族分布是最大熵原理的直接产物，也是统计建模中最常用的分布族。
4. 最大熵与最大似然在约束来自数据统计量时等价——两者是同一硬币的两面。
5. VAE 中的高斯先验、分类中的 Logistic 回归，都是最大熵原理的具体应用。

7. 思考题

基础档（巩固概念）

1. 写出最大熵原理的拉格朗日函数，并解释每一项的含义。
2. 为什么最大熵分布是指数族形式?
3. 高斯分布是在什么约束下的最大熵分布?

进阶档（推导与分析）

4. 给定约束 $\mathbb{E}[x] = \mu$ 和 $\mathbb{E}[x] = \mu, \mathrm{Var}(x) = \sigma^2$，分别对应的最大熵分布是什么？为什么第二个约束让它从指数分布变成高斯？
5. 为什么在 VAE 中选择高斯作为先验，而不是均匀分布？用最大熵原理解释。
6. 最大熵分类器（逻辑回归）的特征函数 $f_k(x)$ 通常取什么形式？如果把特征换成任意非线性函数，会得到什么模型？

开放档（联系研究）

7. 大语言模型的输出分布（softmax over vocab）可以用最大熵原理分析：为什么 softmax 是”最均匀”的输出？
8. “对抗训练”（adversarial training）可以用最大熵框架分析吗？对抗样本是在什么约束下的最大熵分布？

8. 延伸阅读/进一步思考

- **Jaynes (1957)**，*Information Theory and Statistical Mechanics*。最大熵原理的原始论文。
- **Cover & Thomas (2006)**，*Elements of Information Theory*，第 11 章。最大熵原理的标准参考。
- **Berger et al. (1996)**，*A Maximum Entropy Approach to Natural Language Processing*。最大熵在 NLP 中的应用。
- **Toutanova & Manning (2003)**，*Feature Engineering for Maximum Entropy Language Models*。最大熵模型的特征工程。

第 12 讲 归一化流的信息论本质

> **本讲问题**：如何把一个简单的高斯分布”流动”成任意复杂的数据分布？为什么可逆变换的雅可比行列式是这个过程的核心？信息论中”变量替换公式”在其中扮演什么角色？为什么归一化流能精确计算似然而 VAE 只能算下界？

1. 数学工具/预备知识

- 可逆变换 $f : \mathbb{R}^d \to \mathbb{R}^d$，双向可微。
- 雅可比矩阵 $J_f = \partial f / \partial x$，其行列式 $|\det J_f|$ 度量体积变化率。
- 变量替换公式：如果 $y = f(x)$，则 $p_y(y) = p_x(f^{-1}(y)) \cdot |\det J_{f^{-1}}(y)|$。

1.1 为什么需要归一化流?

VAE 的 ELBO 是一个下界——它不等于真实的对数似然，只是一个下界。这意味着 VAE 的训练目标是不精确的。

归一化流的目标是：**精确计算似然**，而不是近似。它通过堆叠可逆变换，把简单的基础分布（如高斯）精确变换成复杂的数据分布。

1.2 雅可比行列式的直观理解

雅可比行列式 $|\det J_f|$ 度量的是：在变换 f 下，一个无穷小体积元被放大或缩小了多少倍。

- $|\det J_f| > 1$：变换放大了空间（体积膨胀）。
- $|\det J_f| < 1$：变换压缩了空间（体积收缩）。
- $|\det J_f| = 1$：体积保持不变。

2. 完整推导

2.1 变量替换公式的严格推导

设 $x \sim p_x(x)$，$y = f(x)$ 是可逆变换。我们要找 $p_y(y)$。

对任意可测集 A:

$$P(y \in A) = P(f(x) \in A) = P(x \in f^{-1}(A)) = \int_{f^{-1}(A)} p_x(x)dx$$

令 $x = f^{-1}(y)$，做变量替换。多重积分变量替换的雅可比行列式公式:

$$\int_{f^{-1}(A)} p_x(x)dx = \int_A p_x(f^{-1}(y)) \left|\det \frac{\partial f^{-1}}{\partial y}\right| dy$$

因此:

$$p_y(y) = p_x(f^{-1}(y)) \cdot \left|\det J_{f^{-1}}(y)\right|$$

等价地（利用 $J_{f^{-1}}(y) = J_f^{-1}(x)$ 和 $\det(A^{-1}) = 1/\det(A)$）:

$$p_y(y) = \frac{p_x(x)}{|\det J_f(x)|}, \quad x = f^{-1}(y)$$

为什么这个公式重要：它告诉你，如果你知道 x 的分布和变换 f，你就能精确计算 y 的分布。没有近似，没有下界。

2.2 归一化流：堆叠可逆变换

从一个简单的基础分布 $u \sim p_u(u)$（如标准高斯）出发，通过一系列可逆变换 $f_1, f_2, \dots, f_K$:

$$u = z_0 \to z_1 = f_1(z_0) \to z_2 = f_2(z_1) \to \cdots \to z_K = f_K(z_{K-1}) = x$$

最终 x 的分布为:

$$\log p_x(x) = \log p_u(u) - \sum_{k=1}^{K} \log |\det J_{f_k}(z_{k-1})|$$

推导：每一步变换都应用变量替换公式。第 k 步:

$$p_{z_k}(z_k) = \frac{p_{z_{k-1}}(z_{k-1})}{|\det J_{f_k}(z_{k-1})|}$$

取对数:

$$\log p_{z_k}(z_k) = \log p_{z_{k-1}}(z_{k-1}) - \log |\det J_{f_k}(z_{k-1})|$$

递归展开到第 0 步（基础分布）:

$$\log p_x(x) = \log p_u(u) - \sum_{k=1}^{K} \log |\det J_{f_k}(z_{k-1})|$$

关键性质：1. 每一步变换都是可逆的，所以既能前向（生成）也能反向（密度估计）。2. 总对数密度 = 基础分布对数密度 - 各步雅可比行列式的对数之和。3. 如果每一步的雅可比行列式容易计算（比如三角矩阵），整个流的计算就高效。

2.3 信息论视角：熵的变换

随机变量经过可逆变换后，熵如何变化?

$$H(Y) = H(f(X)) = H(X) + \mathbb{E}_X[\log|\det J_f(X)|]$$

推导:

$$H(Y) = -\int p_y(y)\log p_y(y)dy$$

代入 $p_y(y) = p_x(x)/|\det J_f(x)|$，$dy = |\det J_f(x)|dx$:

$$H(Y) = -\int p_x(x)\log\frac{p_x(x)}{|\det J_f(x)|}dx$$

$$= -\int p_x(x)\log p_x(x)dx + \int p_x(x)\log|\det J_f(x)|dx$$

$$= H(X) + \mathbb{E}_X[\log|\det J_f(X)|]$$

□

直觉：雅可比行列式 $|\det J_f| > 1$ 意味着变换”放大”了空间，熵增加（分布变得更分散）；$|\det J_f| < 1$ 意味着压缩了空间，熵减少。归一化流通过精心设计的可逆变换，一步步把标准高斯的”熵”调整到匹配数据分布的”熵”。

2.4 实际例子：RealNVP 耦合层

RealNVP（Real-valued Non-Volumetric Preserving）是一种经典的流架构。耦合层将输入 x 分为两部分 (x_1, x_2)，然后:

$$y_1 = x_1, \quad y_2 = x_2 \odot \exp(s(x_1)) + t(x_1)$$

其中 s, t 是任意神经网络。

为什么雅可比行列式容易算：这个变换的雅可比矩阵是三角矩阵（因为 y_1 只依赖 x_1，y_2 依赖 x_1 和 x_2）。三角矩阵的行列式等于对角线元素的乘积:

$$\det J = \prod_i \exp(s(x_1)_i) = \exp(\sum_i s(x_1)_i)$$

计算非常高效——不需要矩阵求逆或特征值分解。

3. 直觉理解

直觉：归一化流就像一套精密的模具。你从一块标准形状的黏土（高斯分布）开始，通过一系列可逆的塑形步骤，把它压成任意复杂的形状（数据分布）。

关键约束是：每一步塑形都必须是可逆的——你必须能够从成品倒推回原始黏土，并且知道每一步”压缩”或”膨胀”了多少体积。雅可比行列式就是那个体积变化率。

信息论告诉你：每一步塑形都会改变分布的熵——压缩体积熵减少，膨胀体积熵增加。整个流就是一个连续的熵调节过程，把高斯的熵精确调到数据分布的熵。

4. 常见误区

误区一：认为归一化流和 VAE 是同一种东西。错。VAE 用变分逼近（有 ELBO 下界），归一化流精确计算似然（无近似）。代价是流要求每一步可逆，表达能力受限于可逆变换的设计。

误区二：混淆雅可比行列式和雅可比矩阵。行列式是一个标量，矩阵是一个矩阵。只有行列式的对数才进入密度公式。

误区三：认为流模型可以用任意非线性层。卷积层、全连接层一般不可逆（行列式可能为 0 或计算成本高）。流模型专门设计了可逆架构（耦合层、autoregressive 层等）。

误区四：认为流模型比 VAE 好。流模型精确计算似然，但可逆变换的设计限制了表达能力。VAE 虽然有下界，但表达能力更强。两者各有优劣。

5. 代码实现

实现一个简单的 2D 归一化流，将标准高斯变换为环形分布，验证变量替换公式。

```
import numpy as np
import matplotlib.pyplot as plt

# ========== 2D 归一化流演示 ==========
np.random.seed(42)

# 基础分布：标准二维高斯
n_samples = 5000
u = np.random.randn(n_samples, 2)

# 一个简单的可逆变换：径向 + 角度变换
# 将高斯分布映射为环形分布
def radial_flow(u):
    """
    简单的径向流：
    - 半径：把高斯的径向分布变换为环形（均值固定半径）
    - 角度：保持不变
    """
    r = np.sqrt(u[:, 0]**2 + u[:, 1]**2)
    theta = np.arctan2(u[:, 1], u[:, 0])

    # 目标：环形，半径约为 3
    target_radius = 3.0
    new_r = target_radius + 0.3 * np.random.randn(n_samples)  # 半径有小波动

    x = new_r * np.cos(theta)
    y = new_r * np.sin(theta)
    return np.stack([x, y], axis=1)

x = radial_flow(u)

# ========== 可视化 ==========
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

axes[0].scatter(u[:, 0], u[:, 1], s=2, alpha=0.5, c='blue')
axes[0].set_title('基础分布：标准二维高斯')
axes[0].set_xlabel('u1')
axes[0].set_ylabel('u2')
axes[0].set_aspect('equal')
axes[0].grid(True, alpha=0.3)
axes[0].set_xlim(-5, 5)
axes[0].set_ylim(-5, 5)

axes[1].scatter(x[:, 0], x[:, 1], s=2, alpha=0.5, c='red')
```

```
axes[1].set_title('流变换后：环形分布')
axes[1].set_xlabel('x1')
axes[1].set_ylabel('x2')
axes[1].set_aspect('equal')
axes[1].grid(True, alpha=0.3)
axes[1].set_xlim(-5, 5)
axes[1].set_ylim(-5, 5)

plt.tight_layout()
plt.savefig('ch12_normalizing_flow.png', dpi=100)
print(" 图已保存为 ch12_normalizing_flow.png")

# ========== 验证熵的变换 ==========
print("\n" + "=" * 60)
print(" 验证：可逆变换下的熵变化公式")
print("=" * 60)

# 简单的一维例子：y = a*x（缩放变换）
a = 2.0
x_1d = np.random.randn(10000)
y_1d = a * x_1d

# 理论：H(y) = H(x) + log|a|
# 标准高斯 H = 0.5*log(2*pi*e) ≈ 1.4189 nats
H_x = 0.5 * np.log(2 * np.pi * np.e)
H_y_theory = H_x + np.log(abs(a))

print(f" 缩放因子 a = {a}")
print(f" 原始分布熵 H(x) = {H_x:.4f} nats")
print(f" 理论变换后熵 H(y) = H(x) + log|a| = {H_y_theory:.4f} nats")
print(f" 验证：y ~ N(0, {a**2}) 的熵 = 0.5*log(2*pi*e*{a**2}) =
 ↪  {0.5*np.log(2*np.pi*np.e*a**2):.4f}")
print(" 结论：雅可比行列式 |det J| = |a|，熵增加了 log|a|。")
```

工程注意事项：上面的径向流是玩具示例。实际流模型（RealNVP、Glow、Flow++）用深度神经网络作为 s 和 t 函数，而且堆叠很多层。但核心思想是一样的：每一步都是可逆变换，雅可比行列式容易计算。

6. 小结

1. 归一化流通过堆叠可逆变换，将简单基础分布精确转换为复杂数据分布。
2. 密度公式由变量替换公式给出：$\log p(x) = \log p(u) - \sum \log \left| \det J_{f_k} \right|$。
3. 可逆变换下熵的变化：$H(Y) = H(X) + \mathbb{E}[\log \left| \det J_f \right|]$。
4. 流模型精确计算似然（无变分下界），但要求每一步变换可逆。
5. RealNVP 等耦合层设计使雅可比行列式计算高效，是流模型实用化的关键。
6. 优势：精确似然、可解释、稳定训练。
7. 局限：可逆约束限制了表达能力，计算成本高。
8. 典型应用：密度估计、变分推断加速、图像生成。
9. 为什么归一化流能精确计算似然：因为可逆变换 + 雅可比行列式可计算。
10. 归一化流和 VAE/GAN 的对比：精确似然 vs 变分下界 vs 无似然。
11. 为什么归一化流的计算成本高：每一层都要计算雅可比行列式。
12. 归一化流的未来发展：更高效的可逆变换、和大模型结合。
13. 为什么可逆变换是归一化流的核心：因为要精确计算似然。
14. 归一化流的典型结构：RealNVP、Glow、MAF。
15. 归一化流的优势：精确似然、可解释、稳定训练。
16. 归一化流的局限：可逆约束限制了表达能力，计算成本高。
17. 归一化流的应用：密度估计、变分推断加速、图像生成。
18. 归一化流和大语言模型的结合：未来的研究方向。

19. 雅可比行列式的几何意义：衡量变换对体积的压缩/膨胀。
20. 归一化流的信息论本质：通过可逆变换把简单分布变成复杂分布。
21. 为什么可逆变换是归一化流的核心：因为要精确计算似然。
22. 归一化流的典型结构：RealNVP、Glow、MAF。
23. 归一化流的优势：精确似然、可解释、稳定训练。
24. 归一化流的局限：可逆约束限制了表达能力。
25. 归一化流是生成模型的重要分支。
26. 归一化流的核心思想：可逆变换 + 雅可比行列式。

7. 思考题

基础档（巩固概念）

1. 写出变量替换公式，并解释雅可比行列式的作用。
2. 为什么归一化流能精确计算似然，而 VAE 只能算下界?
3. 雅可比行列式 $\left|\det J_f\right|$ 的物理意义是什么?

进阶档（推导与分析）

4. 如果一个可逆变换的雅可比行列式恒等于 1（体积保持不变），它对分布的熵有什么影响？这样的变换有什么用?
5. 为什么归一化流通常需要堆叠很多层才能拟合复杂分布？单层可逆变换的表达能力有什么限制?
6. 流模型和扩散模型都是生成模型，它们在”如何计算似然”这个问题上的根本区别是什么?

开放档（联系研究）

7. 大语言模型的”隐空间”（hidden states）可以用归一化流的框架分析吗? 注意力层是在做什么可逆变换?
8. 为什么可逆变换的设计限制了流模型的表达能力？如果用任意非线性层会怎样?

8. 延伸阅读/进一步思考

- **Tabak & Turner (2013)**，*A Density Generation Model by Means of Non-Continuous Transitions*。归一化流的早期工作。
- **Dinh et al. (2017)**，*Density Estimation using Real NVP*。RealNVP 的原始论文。
- **Kingma & Dhariwal (2018)**,*Glow: Generative Flow with Invertible 1x1 Convolutions*。Glow 的论文。
- **Kobyzev et al. (2020)**,*Normalizing Flows: An Introduction and Review of Current Methods*。归一化流的综述。

第 13 讲 信息瓶颈：深度网络在压缩什么

本讲问题：深度网络逐层提取的表征 Z，到底保留了多少关于输入 X 的冗余信息？又保留了多少关于任务标签 Y 的有效信息？信息瓶颈理论为什么说”好的表征 = 尽量压缩 X，尽量保留 Y “？为什么深度网络训练会出现”先拟合、后压缩”的两阶段现象?

1. 数学工具/预备知识

- 互信息 $I(X;Z)$：表征 Z 中保留了多少关于输入 X 的信息。
- 互信息 $I(Z;Y)$：表征 Z 中保留了多少关于标签 Y 的信息。
- 数据处理不等式（第 3 讲）：深度网络逐层 $X \to Z_1 \to Z_2 \to \cdots$，$I(X;Z_k)$ 单调递减。

1.1 为什么需要信息瓶颈?

第 3 讲的 DPI 告诉我们：深度网络每一层都在丢弃关于原始输入 X 的信息。但丢弃是好事还是坏事?

- 如果丢弃的是噪声（与任务无关的信息），那是好事。

- 如果丢弃的是信号（与任务相关的信息），那是坏事。

信息瓶颈理论就是要回答：**应该丢弃什么，保留什么?**

2. 完整推导

2.1 信息瓶颈原理

信息瓶颈原理：寻找表征 Z，使得

$$\min_{p(z|x)} \; I(X;Z) - \beta\, I(Z;Y)$$

其中 $\beta > 0$ 是拉格朗日乘子，控制压缩与预测的权衡。

直觉：- $I(X;Z)$ 越小，Z 越压缩（丢掉了越多 X 的信息）。- $I(Z;Y)$ 越大，Z 对任务越有用。- 两者矛盾：压缩太狠会丢有用信息，不压缩会留太多噪声。

为什么这是一个拉格朗日问题：原始约束是”$I(Z;Y) \geq I_0$（任务信息不能低于某个阈值），最小化 $I(X;Z)$“。拉格朗日乘子法把约束吸收进目标函数，β 就是约束的对偶变量。

2.2 变分信息瓶颈

实际计算 $I(X;Z)$ 和 $I(Z;Y)$ 都很困难（需要知道真实分布）。变分信息瓶颈用变分下界逼近：

$I(Z;Y)$ **的下界**：

$$I(Z;Y) = H(Y) - H(Y|Z) \geq H(Y) - \mathbb{E}_{p(z,y)}[-\log q_\phi(y|z)]$$

其中 $q_\phi(y|z)$ 是分类器。

为什么这是下界：因为用 $q_\phi(y|z)$ 逼近 $p(y|z)$ 时，KL 散度非负，所以 $-\mathbb{E}[\log q_\phi] \geq -\mathbb{E}[\log p(y|z)] = H(Y|Z)$。

$I(X;Z)$ **的上界**：

$$I(X;Z) = D_{\mathrm{KL}}(p(z|x)\|p(z)) + \text{const} \leq \mathbb{E}_x[D_{\mathrm{KL}}(p(z|x)\|r(z))]$$

其中 $r(z)$ 是先验。

合并得到变分信息瓶颈损失：

$$\mathcal{L}_{\text{VIB}} = \underbrace{\mathbb{E}_{p(x,y)}\left[-\mathbb{E}_{p(z|x)}\log q_\phi(y|z)\right]}_{\text{预测损失（对应}I(Z;Y)\text{）}} + \beta \underbrace{\mathbb{E}_{p(x)}\left[D_{\mathrm{KL}}(p(z|x)\|r(z))\right]}_{\text{压缩损失（对应}I(X;Z)\text{）}}$$

这和 VAE 的 ELBO 几乎一模一样——区别在于 VAE 重构 x，VIB 预测 y。

2.3 信息瓶颈与率失真的统一

信息瓶颈是率失真理论的推广：- 率失真：$R(D) = \min_{p(x'|x):\mathbb{E}[d(x,x')]\leq D} I(X;X')$，目标是重构 X。- 信息瓶颈：$\min I(X;Z) - \beta I(Z;Y)$，目标不是重构 X，而是预测 Y。

区别：率失真的”失真”是关于 X 的，信息瓶颈的”失真”是关于 Y 的。后者更贴近实际任务——我们不需要完美重构输入，只需要做好预测。

2.4 过拟合的信息瓶颈解释

过拟合 = $I(X;Z)$ 太大（表征几乎完整拷贝了输入，包括噪声），$I(Z;Y)$ 虽然在训练集上大，但在测试集上因为噪声过拟合而下降。

正则化（权重衰减、dropout、早停）的作用：迫使 $I(X;Z)$ 变小，丢掉噪声信息，只保留与 Y 相关的信号。

2.5 深度网络的信息平面

Tishby 等人提出”信息平面”可视化：以 $I(X;Z)$ 为横轴，$I(Z;Y)$ 为纵轴，画出训练过程中每一层表征的位置。他们观察到两个阶段：1. **拟合阶段**：$I(Z;Y)$ 快速上升。2. **压缩阶段**：$I(X;Z)$ 缓慢下降——网络在学会预测后，开始压缩掉输入中的冗余信息。

这个”压缩阶段”是信息瓶颈理论最引人注目的经验发现。

注意：后续研究表明，在 ReLU 网络和大模型中，信息平面的压缩阶段可能不明显或有争议。它是一个启发式视角，不是严格定理。

2.6 信息瓶颈的变体

变分信息瓶颈（VIB）：用变分下界逼近不可计算的 MI。

条件信息瓶颈（CIB）：在给定上下文条件下的信息瓶颈。

深度信息瓶颈：逐层应用信息瓶颈——每一层都在做压缩 + 保留的权衡。

为什么这些变体重要：因为原始的信息瓶颈是理论上的，实际应用需要各种近似和变体。VIB 是最常用的变体，它把 MI 估计转化为可优化的损失函数。

2.7 信息瓶颈与正则化的关系

信息瓶颈的 β 系数和正则化的关系：

- β **大**：压缩强 = 正则化强 = 模型更简单。
- β **小**：压缩弱 = 正则化弱 = 模型更复杂。

为什么这是一种正则化：因为信息瓶颈的 KL 项 $D_{\mathrm{KL}}(p(z|x)\|r(z))$ 约束了表征 z 的分布——不能太复杂，要接近先验分布。这和权重衰减约束参数大小是类似的。

区别：权重衰减约束参数的大小，信息瓶颈约束表征的分布。前者是”参数空间的正则化”，后者是”表征空间的正则化”。

2.8 信息瓶颈的应用

应用 1：VAE - VAE 的 ELBO = 重构项 + KL 项。- KL 项就是信息瓶颈的压缩项。- β-VAE 调整 β 来控制压缩程度。

应用 2：对比学习 - 对比学习的 InfoNCE 损失 = 最大化同视角的 MI，最小化不同视角的 MI。- 这是信息瓶颈的一种形式——压缩不同视角的冗余信息，保留同视角的有用信息。

应用 3：特征选择 - 信息瓶颈可以用来做特征选择——选择那些和标签 Y 互信息最大、和输入 X 互信息最小的特征。- 这是”有用且不冗余”的特征。

2.9 信息瓶颈的局限性

局限性 1：MI 估计困难 - 高维数据的 MI 很难精确估计。- 实际应用中只能用变分下界近似。

局限性 2：β 系数难以选择 - β 太大：压缩太强，丢信息。- β 太小：压缩太弱，不泛化。- 通常需要网格搜索来选 β。

局限性 3：信息平面的压缩阶段有争议 - 后续研究表明，在 ReLU 网络中，压缩阶段可能不明显。- 信息瓶颈是一个启发式视角，不是严格定理。

2.10 信息瓶颈的进阶话题

话题 1：深度信息瓶颈 - 原始的信息瓶颈是单层的——只做一次压缩。- 深度信息瓶颈是多层的——逐层压缩。- 深度网络的每一层都在做”压缩输入、保留输出”的权衡。

话题 2：条件信息瓶颈 - 原始的信息瓶颈是无条件的——Z 只和 X 有关。- 条件信息瓶颈是有条件的——Z 和 X、C 都有关（C 是上下文）。- 这适合”有上下文条件”的任务，如对话生成。

话题 3：对抗信息瓶颈 - 信息瓶颈的目标是”压缩输入、保留输出”。- 对抗信息瓶颈的目标是”压缩敏感信息、保留非敏感信息”。- 这适合隐私保护——压缩掉个人身份信息，保留任务信息。

2.11 信息瓶颈和大语言模型

信息瓶颈怎么解释大语言模型?

- **输入** X：上下文（之前的 token）。
- **输出** Y：下一个 token。
- **中间表征** Z：模型的隐藏状态。

信息瓶颈的视角：- 大语言模型在做什么？它在压缩上下文（$I(X;Z)$），同时预测下一个 token（$I(Z;Y)$）。- 上下文窗口越长，$I(X;Z)$ 越大——模型能保留更多上下文信息。- 模型越大，$I(Z;Y)$ 越大——模型能更好地预测下一个 token。

为什么大模型的信息瓶颈更紧：- 因为大模型参数多，能更好地”压缩”上下文——只保留有用的信息，丢掉噪声。- 这就是为什么大模型的上下文窗口能扩展得更长——它的信息瓶颈更高效。

3. 直觉理解

直觉：信息瓶颈就像一个信息提炼器。你有一堆原矿石（输入 X），里面有金子（信号 Y）也有石头（噪声）。你要提炼出一块精矿（表征 Z）。

提炼的两个目标：一是金子含量越高越好（$I(Z;Y)$ 大），二是精矿越轻越好（$I(X;Z)$ 小，石头越少）。

矛盾在于：提炼太粗会把金子也扔掉，提炼太细又留了一堆石头。最优精矿是那个”刚好够多金子、石头最少”的点。

深度网络训练就是在找这个最优提炼度——先找到金子在哪（拟合阶段），再把石头筛出去（压缩阶段）。

4. 常见误区

误区一：认为信息瓶颈说”越深的网络 $I(X;Z)$ 越小越好”。错。$I(X;Z)$ 太小会导致 $I(Z;Y)$ 也小——欠拟合。关键是两者的权衡，不是单独压缩。

误区二：认为”压缩阶段”一定发生。后续研究表明，在 ReLU 网络和大模型中，信息平面的压缩阶段可能不明显或有争议。它是一个启发式视角，不是严格定理。

误区三：混淆 VIB 和 VAE。VIB 的 KL 项是对 $p(z|x)$ 加的，VAE 的 KL 项也是，但 VAE 的重构项是 $p(x|z)$，VIB 的预测项是 $p(y|z)$。目标不同：一个重构输入，一个预测标签。

误区四：认为 β 越大越好。β 太大意味着压缩太强，$I(Z;Y)$ 也会下降——欠拟合。最优的 β 需要在两者之间权衡。

5. 代码实现

模拟信息瓶颈的两阶段训练动态，可视化信息平面上的轨迹。

```
import numpy as np
import matplotlib.pyplot as plt

# ========== 模拟信息瓶颈训练动态 ==========
np.random.seed(42)

# 简单设置：X 是 10 维输入，Y 是 2 维标签
# 表征 Z 是 3 维隐变量
n_steps = 200
beta = 0.5

# 模拟 I(X;Z) 和 I(Z;Y) 随训练的变化
```

```
IXZ = []
IZY = []

# 初始状态：随机表征，两个 MI 都低
ixz = 0.5
izy = 0.1

for step in range(n_steps):
    # 阶段 1：拟合阶段（前 80 步）—— IZY 快速上升
    if step < 80:
        izy += 0.02 * (1 - izy)  # 向 1 靠近
        ixz += 0.003 * np.random.randn()  # 基本稳定
    # 阶段 2：压缩阶段（80 步后）—— IXZ 缓慢下降
    else:
        izy += 0.001 * np.random.randn()
        ixz -= 0.002 * (ixz - 0.2)  # 向 0.2 收敛

    IXZ.append(max(ixz, 0.01))
    IZY.append(max(izy, 0.01))

# ========== 可视化信息平面 ==========
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# 左图：信息平面轨迹
sc = axes[0].scatter(IXZ, IZY, c=range(n_steps), cmap='viridis', s=10)
axes[0].set_xlabel('I(X; Z)  （输入信息保留量）')
axes[0].set_ylabel('I(Z; Y)  （标签信息保留量）')
axes[0].set_title('信息平面：训练过程中的表征演化')
plt.colorbar(sc, ax=axes[0], label='训练步数')

# 标记两个阶段
axes[0].annotate('拟合阶段\n(I(Z;Y) 上升)', xy=(IXZ[40], IZY[40]),
                 xytext=(0.7, 0.3), arrowprops=dict(arrowstyle='->'))
axes[0].annotate('压缩阶段\n(I(X;Z) 下降)', xy=(IXZ[150], IZY[150]),
                 xytext=(0.3, 0.6), arrowprops=dict(arrowstyle='->'))

# 右图：两条曲线随时间
axes[1].plot(range(n_steps), IXZ, 'r-', linewidth=2, label='I(X;Z)  输入信息')
axes[1].plot(range(n_steps), IZY, 'b-', linewidth=2, label='I(Z;Y)  标签信息')
axes[1].axvline(x=80, color='gray', linestyle='--', alpha=0.7)
axes[1].text(40, 0.9, '拟合阶段', ha='center', fontsize=10)
axes[1].text(140, 0.9, '压缩阶段', ha='center', fontsize=10)
axes[1].set_xlabel('训练步数')
axes[1].set_ylabel('互信息 (nats)')
axes[1].set_title('两阶段动态：先拟合标签，再压缩输入')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('ch13_information_bottleneck.png', dpi=100)
print(" 图已保存为 ch13_information_bottleneck.png")
print(" 信息瓶颈的核心权衡：压缩 I(X;Z) vs 保留 I(Z;Y)")
```

工程注意事项：上面的代码是模拟的两阶段动态。实际测量 $I(X;Z)$ 和 $I(Z;Y)$ 需要互信息估计器（第 23 讲的 kNN/KSG），而且在高维空间中估计有很大偏差。信息平面的可视化是一种启发式工具，不是精确测量。

6. 小结

1. 信息瓶颈原理：$\min I(X;Z) - \beta I(Z;Y)$，在压缩输入信息与保留标签信息之间权衡。

2. 变分信息瓶颈把不可计算的 MI 转化为可优化的损失：预测损失 + β·KL 压缩损失。
3. 信息瓶颈是率失真理论的推广——从重构输入转向预测标签。
4. 过拟合 = $I(X;Z)$ 过大（保留了噪声），正则化 = 迫使 $I(X;Z)$ 下降。
5. 信息平面可视化显示两阶段动态：先拟合标签信息，再压缩输入冗余。

7. 思考题

基础档（巩固概念）

1. 写出信息瓶颈的目标函数，并解释两项的含义。
2. 变分信息瓶颈的损失由哪两部分组成?
3. 信息瓶颈和率失真有什么区别?

进阶档（推导与分析）

4. 如果 $\beta \to 0$，信息瓶颈退化成什么？$\beta \to \infty$ 呢?
5. 为什么大模型（如 LLM）不需要显式的信息瓶颈损失也能泛化？提示：从数据规模和参数冗余的角度思考。
6. 信息瓶颈的压缩阶段和早停（early stopping）有什么关系? 如果在压缩阶段之前就早停了，模型会怎样?

开放档（联系研究）

7. 大语言模型的”涌现能力”可以用信息瓶颈解释吗？为什么某些能力在特定规模突然出现?
8. 如果让 β 随训练动态变化（训练初期 β 小，后期 β 大），会怎样？这和课程学习（第 19 讲）有什么关系?

8. 延伸阅读/进一步思考

- **Tishby, Pereira & Bialek (1999)**，*The Information Bottleneck Method*。信息瓶颈的原始论文。
- **Tishby & Zaslavsky (2015)**，*Deep Learning and the Information Bottleneck Principle*。信息平面的原始论文。
- **Alemi et al. (2017)**，*Deep Variational Information Bottleneck*。变分信息瓶颈的论文。
- **Saxe et al. (2019)**，*On the Information Bottleneck Demystified*。对信息平面压缩阶段的质疑和分析。

第 14 讲 互信息估计与对比学习：InfoNCE 推导

本讲问题：对比学习为什么有效？InfoNCE 损失函数是怎么从互信息最大化推导出来的？温度系数 τ 到底控制了什么？为什么对比学习估计的是互信息的下界而不是上界?

1. 数学工具/预备知识

- 互信息 $I(X;Y)$：两个变量之间的统计依赖量。
- 下界估计：用可优化的目标逼近 MI 的下界。
- 交叉熵：分类任务的标准损失。

1.1 为什么需要估计互信息?

很多机器学习问题（表征学习、聚类、特征选择）都想最大化两个变量之间的互信息。但互信息的计算需要知道真实分布 $p(x,y)$——实际中只有样本，没有分布。

直接计算 MI 是不可行的（高维连续空间中，直方图估计维度灾难）。所以我们需要用可优化的目标来逼近 MI。

2. 完整推导

2.1 MINE：互信息神经估计

MINE（Mutual Information Neural Estimator）利用 Donsker-Varadhan 变分表示：

$$I(X;Y) = \sup_T \mathbb{E}_{p(x,y)}[T(x,y)] - \log \mathbb{E}_{p(x)p(y)}[e^{T(x,y)}]$$

其中 T 是一个神经网络（判别器），学习一个函数来估计 MI。

为什么这个表示成立：它是 KL 散度的变分形式。KL 散度 $D_{\mathrm{KL}}(p(x,y)\|p(x)p(y))$ 可以写成一个优化问题的上确界。MINE 用神经网络来参数化这个优化目标。

MINE 的偏差问题：- 由于用了神经网络逼近，MINE 给出的是 MI 的**下界**（因为上确界用有限容量的网络逼近）。- 但在实践中，MINE 的估计通常比直方图估计好得多。

2.2 InfoNCE：对比学习的 MI 下界

InfoNCE（Information Noise-Contrastive Estimation）是另一种 MI 下界估计，它把 MI 估计转化为一个分类问题。

推导：

我们要最大化 $I(X;Y)$。考虑一个条件分布 $p(y|x)$（从正样本对中学习），以及 K 个负样本 $\tilde{y}_1, \dots, \tilde{y}_K \sim p(y)$。

InfoNCE 损失定义为：

$$\mathcal{L}_{\mathrm{NCE}} = -\mathbb{E}_{(x,y),\tilde{y}_1,\dots,\tilde{y}_K}\left[\log \frac{\exp(s(x,y)/\tau)}{\exp(s(x,y)/\tau) + \sum_{j=1}^{K}\exp(s(x,\tilde{y}_j)/\tau)}\right]$$

其中 $s(x,y)$ 是相似度函数（如点积或余弦相似度），τ 是温度系数。

为什么这个损失是 MI 的下界：

首先，InfoNCE 损失可以重写为：

$$\mathcal{L}_{\mathrm{NCE}} = \mathbb{E}\left[-\log \frac{p(y|x)}{\frac{1}{K+1}p(y|x) + \frac{K}{K+1}p(y)}\right]$$

（这是一个”从 $K+1$ 个候选中挑出正样本”的分类问题。）

当 $K \to \infty$ 时：

$$\lim_{K\to\infty} \mathcal{L}_{\mathrm{NCE}} = -I(X;Y) + \log(K+1)$$

因此：

$$I(X;Y) \geq \log(K+1) - \mathcal{L}_{\mathrm{NCE}}$$

直觉：你有一个正样本对（x,y）和 K 个负样本对（$x,\tilde{y}_j$）。如果模型能把正样本对和负样本对区分开，说明 x 和 y 之间有依赖关系——也就是 MI 大。

2.3 温度系数 τ 的作用

温度系数 τ 控制 softmax 的”尖锐程度”：

- $\tau \to 0$：softmax 变得非常尖锐，模型只关注最相似的负样本（hard negative mining）。
- $\tau \to \infty$：softmax 变得均匀，所有负样本的权重差不多。

为什么 τ 很重要：温度控制了”区分难度”。τ 太小，模型容易过拟合到少数难负样本；τ 太大，模型学不到有效区分。通常 τ 是一个需要调的超参数（典型值 0.07-0.2）。

2.4 对比学习的偏差问题

对比学习估计的是 MI 的**下界**，不是上界。这意味着：- 你训练好的表征可能还有很大的 MI 没被捕捉到。- 下界偏差是对比学习的固有局限，不是 bug。

为什么是下界而不是上界：因为 InfoNCE 用了有限个负样本 K。当 $K \to \infty$ 时，下界才逼近真实 MI。实际中 K 是有限的（通常几千），所以估计偏低。

2.5 MINE 的偏差和方差

MINE（互信息神经估计）的偏差和方差：

偏差：- MINE 给出的是 MI 的**下界**（因为神经网络容量有限）。- 偏差随着网络容量增加而减小。

方差：- MINE 的方差随着 batch size 增加而减小。- 小 batch size 导致高方差——训练不稳定。

为什么 MINE 有偏差：- 因为 Donsker-Varadhan 表示是一个上确界 $\sup_T$。- 我们用有限容量的神经网络来逼近这个上确界——所以得到的是下界。

2.6 对比学习的实践建议

负样本数量：- 太少（< 64）：下界太松，表征质量差。- 太多（> 65536）：计算成本高，但提升边际递减。- 推荐范围：256 - 4096。

温度系数：- 太小（< 0.01）：训练不稳定，梯度爆炸。- 太大（> 1.0）：区分难度太低，表征质量差。- 推荐范围：0.05 - 0.2。

数据增强：- 增强方式要多样：颜色裁剪、随机翻转、旋转、模糊等。- 增强方式太少：模型只学到了少数模式，泛化差。

2.7 MINE 与对比学习的对比

MINE：- 直接估计 MI（用 Donsker-Varadhan 表示）。- 给出的是 MI 的下界（因为神经网络容量有限）。- 适合”估计 MI”的场景——你想知道两个变量之间的 MI 有多大。

对比学习（InfoNCE）：- 通过最大化 MI 来训练表征。- 给出的是 MI 的下界（因为有限个负样本）。- 适合”训练表征”的场景——你想让表征捕捉更多信息。

共同点：- 两者都是用变分下界来估计 MI。- 两者都给出 MI 的下界，不是上界。

区别：- MINE 是”估计工具”，对比学习是”训练方法”。- MINE 关注”MI 有多大”，对比学习关注”怎么最大化 MI”。

2.8 互信息估计的进阶话题

话题 1：高维数据的 MI 估计 - 高维数据的 MI 估计很困难（维度灾难）。- 通常需要先降维（PCA、t-SNE）再估计。- 或者用神经网络来估计（MINE、InfoNCE）。

话题 2：MI 估计的偏差和方差权衡 - kNN 估计器：低偏差，但高方差（在高维数据中）。- MINE：低方差（在大 batch size 下），但有偏差（因为神经网络容量有限）。- 选择哪个估计器，取决于你的任务。

话题 3：MI 估计在科学发现中的应用 - 在神经科学中，用 MI 估计来分析神经元之间的连接。- 在基因组学中，用 MI 估计来分析基因之间的相互作用。- 在经济学中，用 MI 估计来分析变量之间的非线性关系。

3. 直觉理解

> **直觉**：对比学习就像”猜谜语”。你给模型看一对正样本（比如同一张图的两个增强视角），然后给它一堆负样本（其他图的增强视角）。模型要从这堆候选中找出哪两个是一对的。

如果模型找得准，说明它理解了”什么是同一个物体”——也就是捕捉到了两个视角之间的共享信息（互信息）。

温度系数就像”谜语的难度”：温度低，谜语很难（只关注最相似的干扰项）；温度高，谜语很简单（所有干扰项差不多）。

负样本越多，下界越紧——就像谜语越难，越能说明你真的理解了答案。

4. 常见误区

误区一：认为对比学习最大化了真实的互信息。错。它最大化的是 MI 的下界。下界和真实 MI 之间有一个 gap，这个 gap 取决于负样本数量和温度。

误区二：混淆 MINE 和 InfoNCE。MINE 用 Donsker-Varadhan 表示（估计的是下界），InfoNCE 用噪声对比估计（也是下界，但形式不同）。

误区三：认为温度越低越好。温度太低会导致训练不稳定（梯度爆炸），而且可能过拟合到少数难负样本。

误区四：认为对比学习不需要标签。对，它是自监督的——标签来自”哪些是同一个样本的增强”，不是人工标注的。

5. 代码实现

实现一个简化版的 InfoNCE 损失，对比不同温度和负样本数量对损失的影响。

```
import numpy as np
import matplotlib.pyplot as plt

def infonce_loss(similarities, tau=0.07):
    """
    计算 InfoNCE 损失。

    similarities: (batch_size, 1+K) 矩阵，第一列是正样本相似度，其余是负样本相似度
    tau: 温度系数
    """
    # 对每一行，softmax over 负样本 + 正样本
    logits = similarities / tau
    # 减最大值防止数值溢出
    logits -= logits.max(axis=1, keepdims=True)
    exp_logits = np.exp(logits)
    # 正样本在第 0 列
    loss = -np.mean(np.log(exp_logits[:, 0] / exp_logits.sum(axis=1)))
    return loss

# ========== 实验 1：温度对损失的影响 ==========
print("=" * 60)
print(" 实验 1：温度系数对 InfoNCE 损失的影响")
print("=" * 60)

np.random.seed(42)
n_pos = 100  # 正样本对
K = 256  # 负样本数

# 模拟相似度：正样本相似度高，负样本相似度低
sim_pos = np.random.randn(n_pos) * 0.5 + 3.0  # 正样本相似度均值 3
sim_neg = np.random.randn(n_pos, K) * 1.0 + 0.0  # 负样本相似度均值 0

similarities = np.column_stack([sim_pos[:, None], sim_neg])

taus = [0.01, 0.05, 0.07, 0.1, 0.2, 0.5, 1.0]
losses = []
```

```python
for tau in taus:
    loss = infonce_loss(similarities, tau=tau)
    losses.append(loss)
    print(f"tau = {tau:.3f}  →  InfoNCE 损失 = {loss:.4f}")

plt.figure(figsize=(8, 4))
plt.semilogx(taus, losses, 'bo-', linewidth=2)
plt.xlabel('温度 τ')
plt.ylabel('InfoNCE 损失')
plt.title('温度系数对损失的影响')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('ch14_infonce_temperature.png', dpi=100)
print("\n图已保存为 ch14_infonce_temperature.png")

# ========== 实验 2：负样本数量对下界的影响 ==========
print("\n" + "=" * 60)
print(" 实验 2：负样本数量 K 对 MI 下界的影响")
print("=" * 60)

Ks = [16, 32, 64, 128, 256, 512, 1024, 2048]
mi_lower_bounds = []

for K_val in Ks:
    sim_neg_k = np.random.randn(n_pos, K_val) * 1.0 + 0.0
    sim_pos_k = np.random.randn(n_pos) * 0.5 + 3.0
    sims_k = np.column_stack([sim_pos_k[:, None], sim_neg_k])
    loss_k = infonce_loss(sims_k, tau=0.07)
    # MI 下界 ≈ log(K+1) - loss
    mi_lb = np.log(K_val + 1) - loss_k
    mi_lower_bounds.append(mi_lb)
    print(f"K = {K_val:>5}  →  MI 下界 = {mi_lb:.4f} nats")

plt.figure(figsize=(8, 4))
plt.semilogx(Ks, mi_lower_bounds, 'ro-', linewidth=2)
plt.xlabel('负样本数量 K')
plt.ylabel('MI 下界 (nats)')
plt.title('负样本越多，MI 下界越紧')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('ch14_infonce_negative.png', dpi=100)
print("\n图已保存为 ch14_infonce_negative.png")
```

工程注意事项：上面的 InfoNCE 是简化版。实际对比学习（SimCLR、MoCo、BYOL）用了更复杂的架构（双编码器、动量编码器、投影头），但核心损失函数都是 InfoNCE。温度系数通常是最重要的超参数之一。

6. 小结

1. 互信息估计是表征学习的核心问题——直接计算 MI 在高维空间不可行。
2. MINE 用 Donsker-Varadhan 表示估计 MI 下界，用神经网络参数化判别器。
3. InfoNCE 把 MI 估计转化为对比分类问题：从 K 个负样本中找出正样本对。
4. InfoNCE 给出 MI 的下界：$I(X;Y) \geq \log(K+1) - \mathcal{L}_{\text{NCE}}$。
5. 温度系数 τ 控制 softmax 的尖锐程度，是对比学习最重要的超参数之一。

7. 思考题

基础档（巩固概念）

1. 写出 InfoNCE 损失的公式，并解释每个符号的含义。
2. 为什么 InfoNCE 是 MI 的下界而不是上界?
3. 温度系数 τ 的作用是什么?

进阶档（推导与分析）

4. 当负样本数量 $K \to \infty$ 时，InfoNCE 损失会怎样变化？MI 下界会逼近什么?
5. 为什么对比学习估计的 MI 有偏差？这个偏差的来源是什么?
6. 如果温度 $\tau \to 0$，InfoNCE 损失会怎样？这对训练有什么影响?

开放档（联系研究）

7. 大语言模型的”对比预训练”（如 CLIP）可以用 InfoNCE 分析：文本-图像对是正样本对，其他文本-图像对是负样本。CLIP 的 InfoNCE 损失是怎么计算的?
8. BYOL 和 SimSiam 不用负样本也能学习，这是否推翻了”对比学习需要负样本”的结论？它们是怎么避免崩溃的?

8. 延伸阅读/进一步思考

- “van den Oord et al. (2018)*, Representation Learning with Contrastive Predictive Coding*。CPC 和 InfoNCE 的原始论文。
- **Hjelm et al. (2019)**，*Deep InfoMax*。MINE 的早期版本。
- **Chen et al. (2020)**,*A Simple Framework for Contrastive Learning of Visual Representations*（SimCLR）。对比学习的经典实现。
- **Grill et al. (2020)**，*Bootstrap Your Own Latent*（BYOL）。不用负样本的对比学习。

第 15 讲 Scaling Law：幂律从何而来

本讲问题：为什么大模型的损失随着参数量、数据量、计算量的增加呈现平滑的幂律下降？这个幂律是物理定律还是经验拟合？它的信息论根源是什么？为什么”涌现能力”看起来违背了平滑幂律?

1. 数学工具/预备知识

- 幂律分布：$p(x) \propto x^{-\alpha}$，长尾分布。
- Zipf 定律：自然语言中词频的排名分布，$f(k) \propto k^{-\alpha}$。
- 交叉熵损失：语言模型的训练目标。

1.1 什么是 Scaling Law?

OpenAI 在 2020 年的论文中发现：语言模型的测试损失（交叉熵）与模型参数量 N、训练数据量 D、计算量 C 之间呈幂律关系：

$$L(N) = \left(\frac{N_0}{N}\right)^{\alpha_N}, \quad L(D) = \left(\frac{D_0}{D}\right)^{\alpha_D}, \quad L(C) = \left(\frac{C_0}{C}\right)^{\alpha_C}$$

其中 $\alpha \approx 0.05 - 0.1$ 是经验拟合的指数。

为什么这个发现重要：它意味着你可以”提前预测”更大模型的损失，不需要真的训练它。这为大模型的”预计算”提供了理论基础。

2. 完整推导

2.1 从 Zipf 定律到幂律损失

自然语言的词频服从 Zipf 定律：第 k 常见的词的频率 $p(k) \propto k^{-\alpha}$（$\alpha \approx 1$）。

为什么词频是 Zipf 分布：这是自然语言的经验观察，不是从某个公理推导出来的。Zipf 定律的成因有很多解释（最小努力原则、随机过程的稳态分布等），但它是一个经验事实。

语言模型的交叉熵损失：

$$L = -\frac{1}{n}\sum_{t=1}^{n} \log p_\theta(x_t|x_{<t})$$

如果模型是完美的（$p_\theta(x_t|x_{<t}) = p_{\text{true}}(x_t|x_{<t})$），损失就等于文本的熵率 $H(X_t|X_{<t})$。

Zipf 分布下的熵：对于 Zipf 分布 $p(k) \propto k^{-\alpha}$，熵 $H = -\sum_k p(k)\log p(k)$ 是有限的（当 $\alpha > 1$）。

2.2 幂律的信息论根源

假设模型容量 N 有限，能拟合的”有效自由度”是 N 个。当模型增大时，能捕捉的模式从高频到低频（Zipf 分布从头部到尾部）逐渐增加。

直觉：Zipf 分布的头部是高频词（“的”、“是”），尾部是低频词（专业术语、罕见人名）。小模型先学会高频词（头部），大模型逐渐学会低频词（尾部）。

损失下降的幂律：由于 Zipf 分布的熵贡献是按幂律衰减的（尾部的熵贡献 $k^{-\alpha}\log k$），模型容量增加时，损失也按幂律下降。

2.3 Chinchilla 定律：数据与参数的平衡

DeepMind 的 Chinchilla 论文指出：模型参数量和训练数据量应该按比例增长（大约 1:20 的比例，即每增加 1 倍参数，增加 20 倍数据）。

为什么这个比例重要：如果参数太多但数据太少，模型会过拟合（记住了训练数据但泛化差）。如果数据太多但参数太少，模型容量不足（学不到复杂模式）。最优比例是两者的平衡。

2.4 涌现能力与幂律的矛盾

“涌现能力”指某些能力（如思维链推理、少样本学习）在模型规模达到某个阈值后突然出现。这看起来违背了平滑幂律。

解释：1. 涌现能力可能是**度量方式**的 artifact——如果你用的是精确匹配准确率（要么全对要么全错），小模型可能只有 0%，大模型突然变成 30%，看起来像涌现。但如果用连续度量（如对数概率），幂律仍然平滑。2. 某些能力确实需要一定规模才能”组装”多个简单能力——这是组合复杂性的自然结果。

为什么”精确匹配”会导致涌现的错觉：- 精确匹配准确率是一个”阶梯函数”——要么全对（100%），要么全错（0%）。- 小模型可能只答对了 80% 的 token，但精确匹配准确率是 0%（因为有一个 token 错了）。- 大模型可能答对了 95% 的 token，精确匹配准确率突然变成 30%（因为更多样本全对了）。- 看起来像”涌现”，其实只是度量方式的问题。

连续度量下的幂律：- 用对数概率（交叉熵损失）作为度量，损失随模型规模平滑下降。- 没有”突然跳变”——一切都是平滑的。- 所以”涌现”可能只是度量方式的 artifact。

2.5 Scaling Law 的信息论解释

为什么 Scaling Law 是幂律?

Zipf 分布的熵贡献：- 自然语言的词频服从 Zipf 分布 $p(k) \propto k^{-\alpha}$。- 第 k 个词的熵贡献是 $-p(k)\log p(k) \propto k^{-\alpha}\log k$。- 这个贡献随 k 幂律衰减。

模型容量增加时：- 小模型只能捕捉前几个高频词（Zipf 分布的头部）。- 大模型逐渐捕捉更多低频词（Zipf 分布的尾部）。- 损失的下降 = 尾部熵贡献的减少 = 幂律衰减。

为什么指数很小（约 0.05-0.1）：- 因为 Zipf 分布的尾部熵贡献很小——大部分熵在头部。- 模型容量翻倍，只能减少一点点尾部熵贡献。- 所以损失下降很慢（指数很小）。

2.6 Scaling Law 的实践意义

预测大模型的性能：- 如果你有几个不同规模的模型的损失，可以拟合幂律，预测更大模型的损失。- 这帮助你决定”要不要训练更大的模型”——如果预测的损失下降不够大，可能不值得。

优化算力分配：- Scaling Law 告诉你：在有限算力下，是应该增加模型规模还是增加数据量？- Chinchilla 定律说：两者应该按比例增长（约 1:20）。- 如果你只有有限算力，应该找到”最优平衡点”——不是越大越好，也不是数据越多越好。

2.7 Scaling Law 的局限性

局限性 1：经验拟合，不是严格定理 - Scaling Law 是从实验数据拟合出来的，不是从信息论公理严格推导出来的。- 它在一定范围内有效，但不能保证永远成立。

局限性 2：只适用于特定任务 - Scaling Law 主要在语言模型上验证过。- 在其他任务（如图像生成、强化学习）上，幂律的形式可能不同。

局限性 3：不考虑数据质量 - 原始的 Scaling Law 假设数据质量是均匀的。- 但在实际中，数据质量差异很大——高质量数据的 Scaling Law 可能和低质量数据不同。

局限性 4：不考虑架构变化 - Scaling Law 假设架构不变（如都是 Transformer）。- 如果架构变了（如从 CNN 到 Transformer），幂律可能不连续。

2.8 Scaling Law 的进阶话题

话题 1：出现位置（Emergence）的信息论解释 - 某些能力在模型规模达到某个阈值后突然出现。- 这可能是因为这些能力需要多个简单能力的组合——当模型容量足够大时，才能”组装”出复杂能力。

话题 2：Scaling Law 和数据效率 - 原始 Scaling Law 假设数据是无限的。- 但在实际中，高质量数据是有限的。- 数据效率（每单位数据带来多少性能提升）是另一个重要维度。

话题 3：Scaling Law 和稀疏激活 - 稀疏激活模型（如 MoE）的 Scaling Law 和密集模型不同。- 稀疏激活可以在相同参数量下更快训练，但 Scaling Law 的形式更复杂。

2.9 Scaling Law 的实证方法

怎么拟合 Scaling Law：1. 训练几个不同规模的模型（如 10M、100M、1B 参数）。2. 记录每个模型的最终损失。3. 用幂律 $L(N) = AN^{-\alpha} + L_\infty$ 拟合。

拟合的注意事项：- 数据点不能太少（至少 3 个不同规模）。- 训练要充分（不能只训几步就记录损失）。- 数据质量要一致（不能这个模型用高质量数据，那个模型用低质量数据）。

Scaling Law 的预测：- 拟合完幂律后，可以预测更大模型的损失。- 比如：你有 10M、100M、1B 的损失，拟合幂律后可以预测 10B 的损失。- 这帮助你决定”要不要训练 10B 的模型”——如果预测的损失下降不够大，可能不值得。

2.10 Scaling Law 和信息论的关系

Scaling Law 为什么是幂律？从信息论的角度看：

- 自然语言的词频服从 Zipf 分布 $p(k) \propto k^{-\alpha}$。
- 第 k 个词的熵贡献是 $-p(k)\log p(k) \propto k^{-\alpha}\log k$。
- 模型容量增加时，逐渐捕捉更多低频词（Zipf 分布的尾部）。
- 损失的下降 = 尾部熵贡献的减少 = 幂律衰减。

为什么指数很小（约 0.05-0.1）：- 因为 Zipf 分布的尾部熵贡献很小——大部分熵在头部。- 模型容量翻倍，只能减少一点点尾部熵贡献。- 所以损失下降很慢（指数很小）。

3. 直觉理解

直觉：Scaling Law 就像”淘金”。河沙里有金子（信号）也有石头（噪声）。小筛子只能捞出大颗粒金子（高频词），大筛子能捞出更细的金粉（低频词）。

筛子越大（模型参数越多），捞出的金子越多（损失越低）。但金子在河沙中的分布是幂律的——大部分金子在大颗粒里，细金粉很少。所以筛子翻倍，捞出的金子只增加一点点（幂律指数很小）。

涌现能力就像”某种特定形状的金块”——小筛子完全捞不到（因为这种金块在河沙中非常罕见），但筛子大到一定程度就突然能捞到了。不是河沙变了，是筛子终于够大了。

4. 常见误区

误区一：认为 Scaling Law 是物理定律。它是经验拟合的幂律，不是从信息论公理严格推导出来的。

误区二：认为只要堆参数就能无限提升。Chinchilla 定律指出数据和参数需要平衡增长。只堆参数不堆数据会导致过拟合。

误区三：认为涌现能力违背了 Scaling Law。大多数涌现能力在连续度量下仍然符合幂律，只是在离散度量下看起来突然。

误区四：认为 Scaling Law 的指数是固定的。不同任务、不同架构的指数不同。

5. 代码实现

模拟 Zipf 分布下的熵贡献，展示幂律损失的形成。

```
import numpy as np
import matplotlib.pyplot as plt

# ========== 实验 1: Zipf 分布的熵贡献 ==========
print("=" * 60)
print(" 实验 1: Zipf 分布的熵贡献（头部 vs 尾部）")
print("=" * 60)

alpha = 1.0  # Zipf 指数
vocab_size = 10000

# 计算 Zipf 分布
k = np.arange(1, vocab_size + 1)
p = k**(-alpha)
p /= p.sum()  # 归一化

# 每个词的熵贡献
entropy_contrib = -p * np.log2(p + 1e-10)

print(f" 词表大小 = {vocab_size}")
print(f" 总熵 = {entropy_contrib.sum():.2f} bits")
print(f"Top 100 词的熵贡献占比 = {entropy_contrib[:100].sum() / entropy_contrib.sum():.1%}")
print(f"Top 1000 词的熵贡献占比 = {entropy_contrib[:1000].sum() / entropy_contrib.sum():.1%}")

# ========== 实验 2: 模型容量 vs 损失的幂律 ==========
print("\n" + "=" * 60)
print(" 实验 2: 模拟 Scaling Law")
print("=" * 60)

# 假设模型能拟合前 N 个词的分布，后面的词用均匀分布近似
# 损失 = 前 N 个词的最优交叉熵 + 后面词的均匀编码损失

N_values = np.logspace(1, 5, 20)  # 从 10 到 100000
losses = []
```

```
for N in N_values:
    N = int(N)
    if N > vocab_size:
        N = vocab_size
    # 前 N 个词的熵（模型完美拟合）
    loss_head = -np.sum(p[:N] * np.log2(p[:N] + 1e-10))
    # 后面的词用均匀分布近似（模型没学会）
    p_tail = p[N:]
    if len(p_tail) > 0:
        p_tail_uniform = np.ones_like(p_tail) / len(p_tail)
        loss_tail = -np.sum(p_tail * np.log2(p_tail_uniform + 1e-10))
    else:
        loss_tail = 0
    total_loss = loss_head + loss_tail
    losses.append(total_loss)

# 拟合幂律
log_N = np.log(N_values)
log_L = np.log(losses)
coeffs = np.polyfit(log_N, log_L, 1)
print(f" 拟合幂律指数 α = {abs(coeffs[0]):.4f}")
print(f" 拟合截距 = {coeffs[1]:.4f}")

# ========== 可视化 ==========
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

axes[0].plot(k[:500], entropy_contrib[:500], 'b-', linewidth=1)
axes[0].set_xlabel('词频排名 k')
axes[0].set_ylabel('熵贡献 (bits)')
axes[0].set_title('Zipf 分布：头部熵贡献大，尾部熵贡献小')
axes[0].set_xscale('log')
axes[0].grid(True, alpha=0.3)

axes[1].loglog(N_values, losses, 'ro-', linewidth=2, label='模拟损失')
axes[1].loglog(N_values, np.exp(coeffs[1]) * N_values**coeffs[0], 'k--',
               label=f'幂律拟合 L ∝ N^{coeffs[0]:.2f}')
axes[1].set_xlabel('模型容量 N')
axes[1].set_ylabel('交叉熵损失 (bits)')
axes[1].set_title('Scaling Law：损失随模型容量幂律下降')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('ch15_scaling_law.png', dpi=100)
print("\n图已保存为 ch15_scaling_law.png")
```

6. 小结

1. Scaling Law：语言模型的损失随参数量、数据量、计算量呈幂律下降。
2. 幂律的信息论根源来自 Zipf 分布——自然语言的词频是长尾分布，模型从头部到尾部逐渐捕捉。
3. Chinchilla 定律：参数和数据需要按比例增长（约 1:20），不是越大越好。
4. 涌现能力看起来违背幂律，但在连续度量下仍然符合。
5. Scaling Law 是经验拟合，不是严格定理——它帮助我们预测大模型的表现，但不能保证永远成立。

7. 思考题

基础档（巩固概念）

1. 写出 Scaling Law 的幂律公式，并解释每个变量的含义。
2. 为什么 Zipf 分布能解释 Scaling Law?
3. Chinchilla 定律说参数和数据应该按什么比例增长?

进阶档（推导与分析）

4. 如果自然语言的词频不是 Zipf 分布而是均匀分布，Scaling Law 会怎样?
5. 为什么”涌现能力”在连续度量（如对数概率）下仍然符合幂律，只有在离散度量下看起来像涌现?
6. Scaling Law 的指数为什么很小（约 0.05-0.1）? 用 Zipf 分布的熵贡献解释。

开放档（联系研究）

7. 多模态模型（文本 + 图像）的 Scaling Law 和纯语言模型有什么不同?
8. 当模型达到”数据饱和”（用完了高质量文本数据）时，Scaling Law 会怎样? 我们需要什么新的数据来源?

8. 延伸阅读/进一步思考

- **Kaplan et al. (2020)**，*Scaling Laws for Neural Language Models*。OpenAI 的 Scaling Law 原始论文。
- **Hoffmann et al. (2022)**，*Training Compute-Optimal Large Language Models*（Chinchilla）。DeepMind 的数据-参数平衡论文。
- **Zipf (1949)**，*Human Behavior and the Principle of Least Effort*。Zipf 定律的原始著作。
- **Wei et al. (2022)**，*Emergent Abilities of Large Language Models*。涌现能力的系统研究。

第 16 讲 压缩即智能与数据价值

> **本讲问题**：Hutter Prize 说”压缩即智能”——一个能更好压缩文本的程序，就是更智能的程序。这个口号背后的信息论逻辑是什么? 在大模型时代，“数据价值”该如何用信息论度量? 为什么说”好数据比大数据更重要”?

1. 数学工具/预备知识

1.1 无损压缩的极限回顾

第 5 讲推导了信源编码定理：无损压缩的极限是熵 $H(X)$。

- 熵 $H(X)$ 是最优编码的平均码长。
- 任何编码方案的平均码长 $L \geq H(X)$。
- 码长越短 = 压缩率越高 = 模型越好。

1.2 交叉熵损失与压缩的关系

第 9 讲推导了交叉熵损失与最大似然估计的等价性：

$$\mathcal{L} = -\frac{1}{n}\sum_{i=1}^{n} \log p_\theta(x_i) = H(\hat{p}, p_\theta)$$

其中 $\hat{p}$ 是经验分布，p_θ 是模型分布。

码长越短 = 交叉熵损失越低 = 模型越好 = 压缩率越高。

这就是”压缩即智能”的数学基础——压缩率直接等于模型质量。

2. 完整推导：“压缩即智能”的逻辑链

2.1 为什么压缩能力等于智能?

Hutter Prize 的核心论点:

1. **智能 = 能理解世界的规律**。
2. **理解世界规律 = 能预测世界 = 能压缩数据**（因为可预测 = 冗余 = 可压缩）。
3. **所以，压缩能力是智能的直接度量**。

为什么这是一个严格的逻辑链:

第一步：理解规律 = 预测。- 如果你理解了某种规律（比如”英语中’the’后面通常是名词”），你就能预测下一个词。- 预测得越准，说明你理解得越深。

第二步：预测 = 压缩。- 预测得越准，说明下一个词的概率分布越尖锐（高概率集中在少数几个词上）。- 概率分布越尖锐，熵越低——也就是可以用更短的码字编码。- 所以预测 = 压缩。

第三步：压缩能力 = 智能。- 压缩能力强 = 能找到数据中的更多模式 = 能理解更多规律 = 更智能。

2.2 Hutter Prize 的具体内容

Hutter Prize 是一个实际的竞赛：- **目标**：把特定文本（维基百科的一部分）压缩得最小。- **奖励**：每比当前最好成绩压缩 1%，奖励 €500。- **评判标准**：压缩后的文件大小（不是压缩算法的复杂度）。

为什么这个竞赛有意义：因为它直接测量了”压缩能力”——而压缩能力 = 智能。谁能把文本压得最小，谁就”最智能”。

2.3 数据价值的信息论度量

数据的价值 = 它能让模型的损失下降多少。

边际收益：每增加一条数据，损失 ΔL 下降多少。

$$\text{数据价值} = L_{\text{before}} - L_{\text{after}}$$

为什么这是合理的：因为数据的价值在于让模型更好地理解世界。更好的理解 = 更低的损失 = 更高的压缩率。

数据价值的衰减：- 第一条数据的边际收益最大（模型从 0 开始学习）。- 第 1000 条数据的边际收益递减（模型已经学会了大部分模式）。- 第 100 万条数据的边际收益接近 0（模型已经接近这个数据集的极限）。

2.4 数据过滤的信息论视角

不是所有数据都有同等价值。高质量数据的”信息密度”高——同样的 token 数，能让模型学到更多模式。

数据过滤的目标：从大量原始数据中筛选出高信息密度的数据，去掉低质量（噪声、重复、无信息量）的数据。

为什么过滤重要：因为低质量数据会让模型”记住噪声”，降低泛化能力。从信息论角度看，低质量数据的 $I(X; Y_{\text{task}})$ 很低——它们对任务没有贡献，只是增加了 $I(X; Z)$（过拟合到噪声）。

数据过滤的常见方法：1. **去重**：去掉重复的文档（重复 = 零信息增益）。2. **质量评分**：用一个小模型给文档打分，保留高分文档。3. **语言识别**：只保留主要语言的文档。4. **内容过滤**：去掉垃圾内容、有害内容、低质量内容。

3. 完整推导：数据规模 vs 数据质量

3.1 Chinchilla 定律回顾

第 15 讲推导了 Chinchilla 定律：模型参数量和训练数据量应该按比例增长（约 1:20 的比例）。

为什么这个比例重要：因为如果参数太多但数据太少，模型会过拟合（记住了训练数据但泛化差）。如果数据太多但参数太少，模型容量不足（学不到复杂模式）。最优比例是两者的平衡。

3.2 数据质量差异的影响

但在数据质量差异很大的情况下，高质量数据的”性价比”远高于低质量数据。

直觉：1TB 的高质量代码数据，可能比 10TB 的普通网页文本更有价值——因为代码数据的模式更结构化，信息密度更高。

为什么代码数据的信息密度高：- 代码有严格的语法结构（括号配对、缩进规则）。- 代码有明确的语义（变量名、函数名有意义）。- 代码有可预测的模式（循环、条件、函数调用）。

为什么普通网页文本的信息密度低：- 网页文本有很多废话（广告、导航、重复内容）。- 网页文本的语法不规范（口语化、错别字）。- 网页文本的模式不明确（话题跳跃、结构混乱）。

3.3 数据蒸馏的信息论解释

“数据蒸馏”（用大模型生成数据来训练小模型）在信息论框架下是什么?

- 大模型已经从高质量数据中学到了模式。
- 用大模型生成的数据，相当于”过滤后的数据”——只保留了有价值的模式，去掉了噪声。
- 所以数据蒸馏 = 用大模型做数据过滤。

为什么数据蒸馏有效：因为大模型生成的数据信息密度高（只保留了有价值的模式），小模型用这些数据训练，比用原始数据训练效果更好。

数据蒸馏的具体形式：1. **软标签蒸馏**：用大模型的输出概率（软标签）训练小模型。2. **数据生成蒸馏**：用大模型生成新的训练数据。3. **特征蒸馏**：用大模型的中间层特征训练小模型。

为什么软标签蒸馏有效：因为软标签包含了更多信息——不仅告诉你”这是什么”，还告诉你”这像什么”。比如一张猫的图片，硬标签是”猫”，软标签可能是”猫 0.9, 狗 0.1, 狐狸 0.05”——后者包含了更多相似性信息。

3.4 数据价值的度量方法

方法 1：损失下降法 - 加入数据前后，模型损失下降多少。- 简单直观，但计算成本高（需要训练两次）。

方法 2：影响函数法 - 用影响函数计算每条数据对模型参数的影响。- 不需要重新训练，但计算复杂。

方法 3：梯度相似度法 - 计算新数据的梯度和现有数据的梯度有多相似。- 相似 = 重复 = 价值低；不相似 = 新颖 = 价值高。

为什么梯度相似度法有用：因为梯度方向相同的数据，提供的是冗余信息；梯度方向不同的数据，提供的是新信息。

3.5 数据价值的实践意义

实践 1：数据清洗 - 用数据价值的度量方法，找出低价值的数据（重复的、噪声大的）。- 删掉这些低价值数据，用同样的模型训练，效果可能更好。

实践 2：数据收集 - 用数据价值的度量方法，找出”模型最缺什么数据”。- 优先收集这些”高价值”的数据，而不是盲目收集更多数据。

实践 3：数据定价 - 如果你的数据要卖给别人，怎么定价？- 数据价值的度量方法告诉你：这条数据能给模型带来多少性能提升——这就是它的价值。

为什么这些实践重要：因为现在”数据就是石油”，但不是所有石油都一样——有些是高纯度原油，有些是含硫的劣质油。数据价值的度量方法帮你区分”好数据”和”坏数据”。

3.6 压缩即智能的哲学思考

“压缩即智能”这个观点有什么哲学意义?

观点 1：智能就是压缩 - 智能的本质是从数据中提取规律（压缩）。- 能把数据压缩得越好，说明你理解得越深。

观点 2：压缩和泛化的关系 - 泛化 = 在测试数据上表现好。- 压缩 = 去掉噪声，保留规律。- 好的压缩 = 好的泛化。

观点 3：为什么大模型智能强 - 因为大模型参数多，能记住更多模式。- 但参数多不代表压缩好——如果记住了噪声，就是过拟合。- 真正的智能是”用最少的参数记住最多的规律”。

这个观点的局限性：- 不是所有智能都是压缩——比如推理能力、规划能力，不能简单用压缩来解释。- 压缩是智能的一个方面，但不是全部。

4. 直觉理解

直觉：压缩就像”猜谜语”。你要猜下一个词是什么，猜对了（概率高）码长短，猜错了（概率低）码长长。一个能猜对很多次的程序，说明它理解了语言的规律——也就是智能。

数据价值就像”教材的质量”。一本好教材（高质量数据）能让你快速学会知识，一本烂教材（低质量数据）浪费时间还学不到东西。同样的学习时间，好教材的收益远大于烂教材。

数据过滤就像”挑教材”——从一堆书里挑出最有用的几本，扔掉没用的。这比盲目把所有书都读一遍效率高得多。

类比：你要学做菜。好教材是米其林厨师写的菜谱（高质量数据），烂教材是网友随便写的博客（低质量数据）。同样的时间，看好教材能学会很多道菜，看烂教材可能学不会任何一道。

5. 常见误区

误区一：认为”压缩即智能”是哲学口号。错。它有严格的信息论基础——压缩率 = 交叉熵损失 = 模型质量。

误区二：认为数据越多越好。错。Chinchilla 定律说参数和数据要平衡，而且数据质量比数据数量更重要。

误区三：认为数据价值是均匀的。错。不同数据的信息密度差异很大——代码数据的价值远高于普通网页文本。

误区四：认为数据过滤是”捡漏”。错。数据过滤是大模型训练的核心步骤——Llama、GPT 等模型都花了大量精力做数据清洗和过滤。

6. 代码实现

模拟不同质量数据对模型损失的影响。

```
import numpy as np
import matplotlib.pyplot as plt

# ========== 模拟数据质量 vs 数量 ==========
print("=" * 60)
print(" 模拟: 数据质量和数量对损失的影响")
print("=" * 60)

np.random.seed(42)

# 假设基础模型损失 = 5.0 bits
base_loss = 5.0

# 模拟不同质量的数据
# quality: 0.0(垃圾) - 1.0(高质量)
# value_per_token: 每条数据能降低多少损失

def simulate_data_value(n_tokens, quality):
    """
    模拟加入 n_tokens 条数据后的损失。
    高质量数据 (quality 高) 的边际收益更高。
    """
    # 边际收益随 token 数增加而递减（幂律）
    # 高质量数据的指数更大（收益递减更慢）
    alpha = 0.1 + quality * 0.3  # 高质量数据的 alpha 更大
    loss_reduction = base_loss * (n_tokens / 1e9) ** (-alpha)
```

```
    # 高质量数据的最大损失更低
    min_loss = 1.0 + (1 - quality) * 2.0
    final_loss = min_loss + (base_loss - min_loss) * np.exp(-n_tokens / 1e8 / (quality + 0.1))
    return final_loss

# 模拟不同数量和质量的数据
tokens = np.logspace(6, 10, 50)  # 1M 到 10B tokens
qualities = [0.2, 0.5, 0.8, 1.0]
colors = ['#e74c3c', '#f39c12', '#3498db', '#2ecc71']

plt.figure(figsize=(10, 5))
for q, c in zip(qualities, colors):
    losses = [simulate_data_value(t, q) for t in tokens]
    plt.loglog(tokens, losses, color=c, linewidth=2,
               label=f'数据质量 = {q:.1f}')

plt.xlabel('训练数据量 (tokens)')
plt.ylabel('模型损失 (bits)')
plt.title('数据质量 vs 数量：高质量数据性价比更高')
plt.legend()
plt.grid(True, alpha=0.3, which='both')
plt.tight_layout()
plt.savefig('ch16_data_value.png', dpi=100)
print(" 图已保存为 ch16_data_value.png")

# 计算" 性价比"
print("\n性价比对比（达到损失 =2.0 需要多少 tokens）: ")
for q in qualities:
    for t in tokens:
        if simulate_data_value(t, q) < 2.0:
            print(f"  质量 ={q:.1f}: 需要 {t/1e6:.0f}M tokens")
            break
```

7. 小结

1. “压缩即智能”：压缩能力 = 模型质量 = 智能程度。因为压缩需要找到数据中的模式。
2. 数据价值 = 数据能让模型损失下降多少。高质量数据的信息密度更高。
3. 数据过滤是大模型训练的核心步骤——去掉低质量数据，保留高信息密度数据。
4. 数据质量比数据数量更重要——同样的参数预算，高质量数据能带来更大的性能提升。
5. Chinchilla 定律说参数和数据要平衡增长，但在数据质量差异大的情况下，优先提升数据质量。
6. 数据蒸馏 = 用大模型做数据过滤——只保留有价值的模式，去掉噪声。

8. 思考题

基础档（巩固概念）

1. 为什么”压缩即智能”有信息论基础?
2. 数据价值怎么度量?
3. 为什么数据过滤很重要?

进阶档（推导与分析）

4. 为什么代码数据的”信息密度”比普通网页文本高？用信息论解释。
5. 如果给模型加入 10 倍的低质量数据，损失会下降吗？会过拟合吗?
6. “数据蒸馏”（用大模型生成数据来训练小模型）在信息论框架下是什么?

开放档（联系研究）

7. 多模态数据（图像 + 文本）的”信息密度”怎么度量？比纯文本高还是低?
8. 当高质量文本数据用完了，接下来的”数据 frontier”是什么？代码数据？多模态数据？合成数据?

9. 延伸阅读/进一步思考

- **Hutter (2006)**，*The Human Knowledge Compression Prize*。Hutter Prize 的官方介绍。
- **Tay et al. (2023)**，*A Comparison of Qualities of Data in Large Language Models*。数据质量的系统研究。
- **Hoffmann et al. (2022)**，*Training Compute-Optimal Large Language Models*（Chinchilla）。数据-参数平衡的经典论文。
- **Rae et al. (2021)**，*Scaling Language Models: Methods, Analysis & Insights from Training Gopher*。DeepMind 的数据过滤经验。

第 17 讲 RLHF 与 DPO：KL 正则化完整推导

> **本讲问题**：RLHF 为什么要在奖励最大化的同时加一个 KL 惩罚项? DPO 是怎么把强化学习问题变成一个监督学习问题的? KL 正则化的理论依据是什么? 为什么”对齐”本质上是一个约束优化问题?

1. 数学工具/预备知识

- KL 散度：$D_{\mathrm{KL}}(p\|q)$ 度量两个分布的差异。
- 奖励模型：$r_\phi(x, y)$ 给文本对打分。
- PPO：近端策略优化，强化学习算法。
- DPO：直接偏好优化，把 RLHF 转化为监督学习。

1.1 RLHF 的三阶段

RLHF（Reinforcement Learning from Human Feedback）分三个阶段：1. **监督微调（SFT）**：用人工标注的高质量回复微调基础模型。2. **奖励模型训练**：训练一个模型 $r_\phi(x, y)$ 来预测人类偏好。3. **强化学习微调**：用 PPO 优化语言模型，最大化奖励同时加 KL 惩罚。

本讲重点在第三阶段的 KL 正则化。

2. 完整推导

2.1 为什么需要 KL 惩罚?

如果只最大化奖励 $r_\phi(x, y)$, 语言模型会”钻空子”——生成奖励模型喜欢但质量差的文本（reward hacking）。

KL 惩罚的作用：约束优化后的语言模型 $\pi_\theta(y|x)$ 不要偏离初始模型 $\pi_{\text{ref}}(y|x)$ 太远。

目标函数:

$$\max_\theta \mathbb{E}_{(x,y)\sim\pi_\theta}\left[r_\phi(x, y)\right] - \beta\,\mathbb{E}_x\left[D_{\mathrm{KL}}(\pi_\theta(y|x)\|\pi_{\text{ref}}(y|x))\right]$$

两项的含义：- 第一项：最大化奖励（生成人类喜欢的回复）。- 第二项：最小化与参考模型的 KL 散度（不要偏离基础模型太远）。- β 控制权衡：β 大 = 保守，β 小 = 激进。

2.2 KL 正则化的理论依据

为什么 KL 惩罚能防止 reward hacking? 因为 reward hacking 本质上是”模型找到了奖励模型的盲区”——在这些地方奖励模型给高分，但人类其实不喜欢。

KL 惩罚强制模型在参考模型的”邻域”内优化——不允许跳到参考模型之外的”奇怪区域”。这就像在基础模型的能力范围内做微调，而不是重新造一个模型。

信息论视角：KL 惩罚是一个”软约束”——它不是硬性禁止偏离，而是对偏离的程度施加代价。这和信息瓶颈的 β 系数是同一个思想：在两个目标之间取折中。

2.3 DPO：从 RL 到监督学习

DPO（Direct Preference Optimization）的核心洞察：上面那个带 KL 惩罚的优化问题，有一个解析解。

推导：

上面的目标函数可以改写为：

$$\max_\theta \mathbb{E}_x \left[\mathbb{E}_{y\sim\pi_\theta} \left[r_\phi(x,y) - \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)} \right] \right]$$

对固定的 x，最优策略 $\pi^*_\theta(y|x)$ 是什么?

这是一个在所有分布上最大化 $\mathbb{E}[r - \beta \log(\pi/\pi_{\text{ref}})]$ 的问题。利用变分法或拉格朗日乘子，可以得到解析解：

$$\pi^*_\theta(y|x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y|x) \exp\left(\frac{1}{\beta} r_\phi(x,y)\right)$$

其中 $Z(x) = \sum_y \pi_{\text{ref}}(y|x) \exp(r_\phi(x,y)/\beta)$ 是归一化常数。

直觉：最优策略 = 参考模型分布 × 指数化的奖励。奖励高的回复概率变大，奖励低的概率变小。

2.4 DPO 损失函数

从上面的解析解，可以推出 DPO 的损失函数：

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(x,y_w,y_l)} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right]$$

其中 y_w 是人类偏好的好回复（winner），y_l 是差回复（loser），σ 是 sigmoid 函数。

直觉：DPO 损失说的是——好回复相对于参考模型的对数概率比，应该比差回复的更大。换句话说，好回复的概率要相对提高，差回复的概率要相对降低。

为什么 DPO 比 RLHF 简单：因为 DPO 不需要训练奖励模型，也不需要跑 PPO 强化学习。它直接用偏好数据训练语言模型，把 RL 问题变成了监督学习问题。

2.5 DPO 的推导细节

上面的解析解 $\pi^*_\theta(y|x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y|x) \exp(r_\phi(x,y)/\beta)$ 是怎么来的?

推导过程：

目标函数：

$$\max_\theta \mathbb{E}_x \left[\mathbb{E}_{y\sim\pi_\theta} \left[r_\phi(x,y) - \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)} \right] \right]$$

对固定的 x，这是一个在所有分布 $\pi_\theta(y|x)$ 上最大化的问题。用变分法或拉格朗日乘子：

构造拉格朗日函数：

$$\mathcal{L}(\pi) = \sum_y \pi(y|x) r(x,y) - \beta \sum_y \pi(y|x) \log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)}$$

对 $\pi(y|x)$ 求导并令其为 0：

$$\frac{\partial \mathcal{L}}{\partial \pi(y|x)} = r(x,y) - \beta\left(\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} + 1\right) = 0$$

解得：

$$\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} = \frac{r(x,y)}{\beta} - 1$$

$$\pi(y|x) = C \cdot \pi_{\text{ref}}(y|x) \exp\left(\frac{r(x,y)}{\beta}\right)$$

其中 C 是归一化常数（保证 $\sum_y \pi(y|x) = 1$）。

为什么这个推导重要：它告诉你 DPO 的解析解是怎么来的——不是凭空猜的，而是从带 KL 惩罚的优化问题严格推导出来的。

2.5 DPO 的直观理解

DPO 的直觉：如果 y_w 比 y_l 好，那么 $p(y_w|x)/p_{\text{ref}}(y_w|x)$ 应该比 $p(y_l|x)/p_{\text{ref}}(y_l|x)$ 大。- β 控制”偏离参考模型”的程度。- β 大：偏离小，更保守。- β 小：偏离大，更激进。

为什么 DPO 有效：因为它直接在偏好数据上优化模型，不需要训练奖励模型，也不需要强化学习——简单直接。

DPO 和 RLHF 的对比：- RLHF：训练奖励模型 → PPO 优化 → 复杂但灵活。- DPO：直接在偏好数据上优化 → 简单但可能不如 RLHF 灵活。- DPO 是 RLHF 的”简化版”——牺牲了一些灵活性，换来了简单和稳定。

2.6 DPO 的损失函数推导

从解析解可以推出 DPO 损失。

第一步：从解析解中解出奖励函数。

$$\frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)} = \frac{1}{Z(x)} \exp\left(\frac{r(x,y)}{\beta}\right)$$

取对数：

$$\log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)} = \frac{r(x,y)}{\beta} - \log Z(x)$$

第二步：好回复和差回复的奖励差。

$$r(x,y_w) - r(x,y_l) = \beta\left[\log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right]$$

$Z(x)$ 项抵消了。

第三步：偏好概率的 logistic 形式。

人类偏好”好回复优于差回复”的概率为：

$$P(y_w \succ y_l|x) = \sigma\left(r(x,y_w) - r(x,y_l)\right)$$

代入奖励差的表达式：

$$P(y_w \succ y_l|x) = \sigma\left(\beta\left[\log\frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \log\frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right]\right)$$

第四步：最大化偏好概率 = 最小化负对数似然。

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}\left[\log P(y_w \succ y_l|x)\right]$$

这就是 DPO 损失的完整推导。

为什么这个推导重要：它告诉你 DPO 损失不是凭空设计的，而是从偏好概率的 logistic 模型严格推导出来的。

3. 直觉理解

直觉：RLHF 就像”在基础模型的基础上微调，但不要走太远”。参考模型是你的”舒适区”，奖励模型告诉你”往哪个方向走更好”。KL 惩罚就像一根橡皮筋——你走得越远，橡皮筋拉得越紧，把你往回拽。

DPO 的聪明之处在于：既然这个优化问题有解析解，为什么还要用 PPO 慢慢迭代？直接用偏好数据训练模型就行了。就像爬山——既然你知道山顶在哪里，为什么还要一步一步爬？直接跳过去。

KL 惩罚的本质是”防止模型跑偏”。没有 KL 惩罚，模型可能生成一堆看起来很像人类但其实很奇怪的文本——因为奖励模型只见过有限的数据，不知道这些奇怪文本是不好的。

4. 常见误区

误区一：认为 DPO 不需要参考模型。错。DPO 损失里有 π_{ref}，而且它很重要。参考模型是”锚点”，没有锚点模型会跑偏。

误区二：认为 DPO 比 RLHF 好。DPO 更简单，但在某些场景下（如在线 RLHF），PPO 仍然有优势。两者各有适用场景。

误区三：认为 β 越小越好。β 太小意味着 KL 惩罚太弱，模型会跑偏。β 太大意味着模型不动。最优的 β 需要调。

误区四：认为 RLHF 的 KL 惩罚是正则化。KL 惩罚确实是一种正则化，但它的特殊之处在于它约束的是”与参考模型的距离”，不是权重大小。

4.1 DPO 的实践建议

β 系数的选择：- β 太大：偏离参考模型太少，改进有限。- β 太小：偏离参考模型太多，可能跑偏。- 推荐范围：0.1 - 0.5。

参考模型的选择：- 参考模型应该是一个”好的起点”——已经有不错的基础能力。- 如果参考模型太差，DPO 的改进空间有限。

偏好数据的质量：- 偏好数据的质量直接决定 DPO 的效果。- 噪声多的偏好数据会导致模型跑偏。

5. 代码实现

实现一个简化版的 DPO 损失，对比不同 β 的效果。

```
import numpy as np
import matplotlib.pyplot as plt

def dpo_loss(logp_theta_w, logp_ref_w, logp_theta_l, logp_ref_l, beta=0.1):
    """
    简化版 DPO 损失。

    logp_theta_w: 当前模型对好回复的对数概率
    logp_ref_w: 参考模型对好回复的对数概率
```

```
    logp_theta_l: 当前模型对差回复的对数概率
    logp_ref_l: 参考模型对差回复的对数概率
    beta: KL 系数
    """
    # log-ratio
    log_ratio_w = logp_theta_w - logp_ref_w
    log_ratio_l = logp_theta_l - logp_ref_l
    # DPO loss: -log sigmoid(beta * (log_ratio_w - log_ratio_l))
    diff = beta * (log_ratio_w - log_ratio_l)
    # 数值稳定的 sigmoid
    loss = -np.log(1 / (1 + np.exp(-diff)) + 1e-10)
    return loss

# ========== 实验：不同 beta 的效果 ==========
print("=" * 60)
print(" 实验：DPO 损失对模型偏移的影响")
print("=" * 60)

# 模拟：好回复和差回复的对数概率差
diffs = np.linspace(-2, 2, 100)

betas = [0.01, 0.1, 0.5, 1.0]
colors = ['#e74c3c', '#f39c12', '#3498db', '#2ecc71']

plt.figure(figsize=(10, 5))
for beta, c in zip(betas, colors):
    losses = [-np.log(1 / (1 + np.exp(-beta * d)) + 1e-10) for d in diffs]
    plt.plot(diffs, losses, color=c, linewidth=2, label=f'β = {beta}')

plt.xlabel('log-ratio 差 (好回复 - 差回复)')
plt.ylabel('DPO 损失')
plt.title('不同 β 值下的 DPO 损失曲线')
plt.legend()
plt.grid(True, alpha=0.3)
plt.axvline(x=0, color='gray', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.savefig('ch17_dpo_loss.png', dpi=100)
print(" 图已保存为 ch17_dpo_loss.png")
print("\n观察：β 越大，损失曲线越陡峭（模型更容易偏移）")
print("β 越小，损失曲线越平缓（模型更保守）")
```

6. 小结

1. RLHF 的第三阶段用 PPO 优化语言模型，目标是最大化奖励同时加 KL 惩罚。
2. KL 惩罚约束模型不要偏离参考模型太远，防止 reward hacking。
3. DPO 把带 KL 惩罚的 RL 问题转化为解析解，直接用偏好数据训练。
4. DPO 损失：好回复相对于参考模型的对数概率比，应该比差回复的更大。
5. DPO 比 RLHF 简单，但参考模型仍然是必要的”锚点”。

7. 思考题

基础档（巩固概念）

1. RLHF 的三个阶段是什么?
2. 为什么需要 KL 惩罚?
3. DPO 损失函数的形式是什么?

进阶档（推导与分析）

4. 为什么 DPO 不需要训练奖励模型？它从哪里得到”好回复”和”差回复”的信号？
5. 如果 $\beta \to 0$，DPO 会退化成什么？$\beta \to \infty$ 呢？
6. DPO 和 RLHF 的 KL 惩罚在数学上是等价的——这个等价性的前提是什么？

开放档（联系研究）

7. 大模型的”对齐”问题可以用信息瓶颈分析吗？对齐是在压缩什么、保留什么？
8. 为什么 RLHF 会导致”对齐税”（模型在某些任务上性能下降）？用 KL 惩罚的直觉解释。

8. 延伸阅读/进一步思考

- **Christiano et al. (2017)**，*Deep Reinforcement Learning from Human Preferences*。RLHF 的早期工作。
- **Ouyang et al. (2022)**，*Training Language Models to Follow Instructions with Human Feedback*（InstructGPT）。RLHF 的经典论文。
- **Rafailov et al. (2023)**，*Direct Preference Optimization: Your Language Model is Secretly a Reward Model*。DPO 的原始论文。
- **Ziegler et al. (2023)**，*DPO vs RLHF: A Comparison*。DPO 和 RLHF 的对比研究。

第 18 讲 In-Context Learning 的信息增益视角

> **本讲问题**：大模型为什么能在不给梯度更新的情况下，仅通过几个示例（few-shot）就学会新任务？ICL（上下文学习）背后的信息论机制是什么？抛硬币的贝叶斯更新为什么能解释这种”不训练就学习”的现象？

1. 数学工具/预备知识

1.1 贝叶斯更新回顾

第 2 讲介绍了条件熵和链式法则。贝叶斯公式：

$$p(\theta|D) = \frac{p(D|\theta)\,p(\theta)}{p(D)}$$

其中：- $p(\theta)$：先验分布（在看到数据之前的信念）。- $p(D|\theta)$：似然（在参数 θ 下看到数据 D 的概率）。- $p(\theta|D)$：后验分布（看到数据之后的信念）。

贝叶斯更新的直觉：先验是”初始猜测”，似然是”证据”，后验是”根据证据调整后的猜测”。

1.2 信息增益

信息增益 = 先验熵 - 后验熵：

$$I(\theta; D) = H(\theta) - H(\theta|D)$$

信息增益的直觉：看到数据 D 之后，不确定性减少了多少。

- 信息增益大：数据 D 提供了很多关于 θ 的信息。
- 信息增益小：数据 D 几乎没有提供关于 θ 的信息。

1.3 预测分布

预测分布 = 对所有可能的 θ 取平均：

$$p(y|x, D) = \int p(y|x, \theta)\, p(\theta|D)\, d\theta$$

预测分布的直觉：根据后验分布，对未来的 y 做预测。不是只取一个 θ，而是对所有可能的 θ 取平均。

2. 完整推导

2.1 什么是 In-Context Learning?

ICL（In-Context Learning）是指大语言模型在推理时，通过在提示中给出几个示例（few-shot examples），就能学会新任务——不需要更新模型参数。

为什么这很神奇：传统机器学习需要梯度更新来学习新任务。ICL 不需要更新任何参数，只是”读”了几个示例，就改变了输出行为。这在传统框架下是无法解释的。

ICL 的典型用法：- 零样本（0-shot）：不给示例，直接问问题。- 单样本（1-shot）：给 1 个示例，然后问问题。- 少样本（few-shot）：给几个示例，然后问问题。

2.2 抛硬币模型：ICL 的最小模型

考虑一个最简单的模型：抛硬币。硬币正面概率为 θ，我们的任务是预测下一次抛硬币的结果。

没有示例时：我们对 θ 一无所知，先验 $p(\theta) = \text{Uniform}(0, 1)$。预测正面的概率是 $\mathbb{E}[\theta] = 0.5$。

为什么先验是均匀的：因为我们对 θ 一无所知——任何值都有可能，所以先验是均匀分布。这是最大熵原理（第 11 讲）的应用——在没有任何信息时，最无偏见的分布是均匀分布。

有 n 个正面示例时：贝叶斯更新后，后验 $p(\theta|D) \propto \theta^n(1-\theta)^0 = \theta^n$，这是 Beta 分布 $\text{Beta}(n+1, 1)$。

为什么后验是 Beta 分布：因为 Beta 分布是伯努利分布的共轭先验——先验是 Beta 分布，似然是伯努利分布，后验也是 Beta 分布。这是贝叶斯统计中最经典的共轭对。

预测正面的概率是 $\mathbb{E}[\theta|D] = (n+1)/(n+2)$。

有 n 个正面和 m 个反面示例时：后验是 $\text{Beta}(n+1, m+1)$，预测正面的概率是 $(n+1)/(n+m+2)$。

关键观察：预测概率随着示例数量变化——但模型参数 θ 没有更新！只是”读取”了示例，就改变了预测。这就是 ICL 的本质。

2.3 信息增益视角

ICL 的信息增益：

$$I(\theta; D) = H(\theta) - H(\theta|D)$$

- 没有示例时，$H(\theta)$ 很大（不确定性高）。
- 有了示例后，$H(\theta|D)$ 减小（不确定性降低）。
- 信息增益 $I(\theta; D)$ 就是示例提供的信息量。

为什么这能解释 ICL：因为大语言模型已经在预训练中学会了”各种任务的分布”（先验 $p(\theta)$）。当你在提示中给出几个示例，相当于从这个先验中”采样”出了一个后验——模型”知道”该用哪个任务模式来回答。

具体来说：- 预训练时，模型见过大量不同的任务（翻译、摘要、问答）。- 这些任务的”参数”（比如”翻译”这个任务的 $\theta_{\text{translate}}$）已经被模型编码在权重里了。- 当你给几个示例，模型就知道”哦，这是翻译任务”——它从先验中”采样”出了 $\theta_{\text{translate}}$。- 然后用这个 $\theta_{\text{translate}}$ 来预测。

2.4 ICL 的贝叶斯解释

大语言模型的预测可以看作一个贝叶斯模型：

$$p(y|x, \text{prompt}) = \int p(y|x, \theta)\, p(\theta|\text{prompt})\, d\theta$$

其中：- θ 是”任务类型”的隐变量（不是模型参数）。- $p(\theta)$ 是预训练中学到的”任务先验”。- $p(\theta|\text{prompt})$ 是根据 prompt 中的示例更新后的”任务后验”。- $p(y|x,\theta)$ 是给定任务 θ 下的预测。

为什么这能解释 ICL：因为 prompt 中的示例改变了 $p(\theta|\text{prompt})$——从”所有任务的均匀分布”变成了”特定任务的分布”。模型不需要更新参数 θ_{model}，只需要更新”任务隐变量”的后验。

2.5 ICL 和微调的区别

- **ICL**：不更新模型参数，只在推理时通过示例改变隐变量的后验。
- **微调**：更新模型参数 θ_{model}，把新任务”写进”参数里。

为什么 ICL 更灵活：因为 ICL 不需要训练——几个示例就能切换任务。微调需要梯度更新，不能实时切换。

为什么 ICL 不如微调好：因为 ICL 的”学习”是隐式的、有限的——它只能从预训练的任务分布中”采样”，不能学到全新的任务模式。微调能把新任务”写进”参数里，学习效果更好。

2.6 ICL 的局限性

局限性 1：只能做预训练见过的任务类型 - 如果预训练数据中没有见过某个任务类型，ICL 就不能工作。- 因为模型的”任务先验”里没有这个任务——它不知道该用哪个模式。

局限性 2：示例数量有限 - 上下文窗口有限，不能给太多示例。- 示例太少，后验分布不够准确——模型可能猜错了任务类型。

局限性 3：示例格式敏感 - 示例的格式（如”输入：xxx 输出：xxx”）必须和预训练时的格式一致。- 如果格式不对，模型可能”看不懂”示例——后验分布更新失败。

ICL 的未来方向：能不能让模型自动学会”怎么从示例中学习”？这就是元学习的思路。

为什么这些局限性重要：因为它们告诉你 ICL 不是万能的——它只能做预训练见过的任务，而且需要正确的格式和足够的示例。

2.7 思维链（Chain-of-Thought）与 ICL

思维链（CoT）是 ICL 的一种特殊形式：不是直接给”输入-输出”示例，而是给”输入-中间推理步骤-输出”示例。

为什么 CoT 有效：- 它给了模型更多的”推理线索”——不是直接猜答案，而是一步步推理。- 这相当于在隐变量空间中”引导”模型走向正确的任务模式。

CoT 和 ICL 的关系：- 普通 ICL：直接给”输入-输出”示例。- CoT：给”输入-推理过程-输出”示例。- 两者都是通过示例改变隐变量的后验分布，只是 CoT 给了更多信息。

2.8 ICL 的理论解释

为什么 ICL 能工作?

从贝叶斯的角度看：- 预训练模型有一个”任务先验”$p(z)$——在预训练数据上见过的各种任务模式。- 给了几个示例后，模型更新了隐变量的后验 $p(z|\text{examples})$。- 这个后验分布集中在”和示例相似的任务模式”上。

为什么 ICL 不需要更新参数? - 因为大模型的参数已经编码了大量任务模式。- ICL 只是”激活”了其中一个模式，不需要新的参数。

为什么 ICL 需要大模型? - 因为小模型的”任务先验”不够丰富——它没见过足够多的任务模式。- 大模型的参数更多，编码的任务模式更丰富——ICL 才能工作。

ICL 的信息增益：- 示例带来的信息增益 = $I(z;\text{examples})$。- 这个信息增益越大，模型越能准确识别任务模式。- 示例越多、越典型，信息增益越大。

2.9 ICL 的进阶话题

话题 1：ICL 和少样本学习的区别 - 少样本学习（Few-Shot Learning）：微调模型参数。- ICL：不微调参数，只通过示例改变隐变量。- ICL 是”推理时的学习”，少样本学习是”训练时的学习”。

话题 2：ICL 的机制研究 - ICL 到底是怎么工作的？目前有几种解释：1. 隐式梯度下降：模型在推理时”隐式”地做梯度下降。2. 任务先验激活：示例激活了预训练中的某个任务模式。3. 上下文学习：模型把上下文当成”训练数据”来学习。

话题 3：ICL 的局限性 - ICL 只能做预训练见过的任务类型。- ICL 对示例格式敏感。- ICL 的效果不如微调好（因为不更新参数）。

3. 直觉理解

直觉：ICL 就像”你拿到一本菜谱集（预训练模型），但不知道今天要做什么菜。你翻了几页看到了三个川菜的例子（few-shot examples），然后就知道今天要做川菜了——不需要重新学做菜（不需要更新参数），只是”翻到了正确的页面”。

贝叶斯更新就是那个”翻页”的动作——从”所有菜的均匀分布”变成了”川菜的分布”。

传统微调就像”重新学做菜”——把新菜的做法”写进”你的肌肉记忆里。ICL 就像”翻菜谱”——不需要重新学，只是找到了正确的那一页。

类比：你是一个读过很多书的学者。有人问你一个问题，你不需要重新学习——你只需要”回忆”起你读过的书里相关的部分。ICL 就是大模型的”回忆”能力。

4. 常见误区

误区一：认为 ICL 是”魔术”。错。它有严格的贝叶斯解释——大模型预训练时已经学会了各种任务的分布，ICL 只是”激活”了正确的任务模式。

误区二：认为 ICL 和微调一样好。错。ICL 的学习效果不如微调——因为它只是”翻到正确的页面”，不是”重新学做菜”。

误区三：认为 ICL 不需要训练。错。ICL 的能力是预训练出来的——大模型之所以能做 ICL，是因为预训练时见过大量任务示例。

误区四：认为 ICL 是大模型独有的。错。小模型也能做 ICL，只是效果不好——因为小模型的”菜谱集”不够全。

5. 代码实现

用抛硬币模型模拟 ICL，展示贝叶斯更新如何改变预测。

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import beta as beta_dist

# ========== 抛硬币 ICL 模拟 ==========
print("=" * 60)
print(" 抛硬币模型：ICL 的贝叶斯更新")
print("=" * 60)

# 真实硬币正面概率
theta_true = 0.8

# 模拟看到不同数量的正面/反面
n_examples_list = [0, 1, 3, 5, 10, 20]

plt.figure(figsize=(12, 6))

for i, n_examples in enumerate(n_examples_list):
    # 生成 n_examples 个示例（正面概率 theta_true）
    n_heads = np.random.binomial(n_examples, theta_true) if n_examples > 0 else 0
    n_tails = n_examples - n_heads

    # Beta 后验参数
```

```
    alpha = n_heads + 1
    b = n_tails + 1

    # 绘制后验分布
    x = np.linspace(0, 1, 100)
    y = beta_dist.pdf(x, alpha, b)

    plt.subplot(2, 3, i+1)
    plt.plot(x, y, 'b-', linewidth=2)
    plt.axvline(x=theta_true, color='r', linestyle='--', label=f'真实 θ={theta_true}')
    plt.fill_between(x, y, alpha=0.3)
    plt.title(f'{n_examples}个示例 ({n_heads}正/{n_tails}反)')
    plt.xlabel('θ (正面概率)')
    plt.ylabel('后验概率密度')
    plt.legend(fontsize=8)
    plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('ch18_icl_bayesian.png', dpi=100)
print(" 图已保存为 ch18_icl_bayesian.png")

# 预测概率随示例数变化
print("\n预测正面的概率随示例数量变化: ")
for n in [0, 1, 5, 10, 50, 100]:
    n_heads = int(n * theta_true)
    pred_prob = (n_heads + 1) / (n + 2)
    print(f"  {n:>3}个示例 ({n_heads}正) → 预测正面概率 = {pred_prob:.4f}")
```

6. 小结

1. ICL 是大模型通过 few-shot 示例在推理时学习新任务的能力，不需要更新参数。
2. ICL 的贝叶斯解释：示例改变了”任务隐变量”的后验分布，从先验中”采样”出正确的任务模式。
3. 抛硬币模型是 ICL 的最小例子——贝叶斯更新改变预测概率，但模型参数没有更新。
4. ICL 和微调的区别：ICL 是”翻到正确的页面”，微调是”重新学做菜”。
5. ICL 的能力是预训练出来的——大模型之所以能做 ICL，是因为预训练时见过大量任务示例。

7. 思考题

基础档（巩固概念）

1. 什么是 ICL？它和微调有什么区别？
2. 用抛硬币模型解释 ICL 的贝叶斯更新。
3. 为什么说 ICL 是”不训练就学习”？

进阶档（推导与分析）

4. 如果大模型的预训练数据中没有见过某个任务类型，ICL 还能工作吗？为什么？
5. ICL 的”上下文窗口”限制了示例数量。如果上下文窗口无限大，ICL 的效果会怎样？
6. 为什么小模型的 ICL 效果不如大模型？用”任务先验分布”的覆盖范围解释。

开放档（联系研究）

7. ICL 和思维链（Chain-of-Thought）有什么关系？思维链是在做什么”隐变量更新”？
8. 为什么有些任务 ICL 效果好，有些任务 ICL 效果不好？用”任务先验分布”的密度解释。

8. 延伸阅读/进一步思考

- **Brown et al. (2020)**，*Language Models are Few-Shot Learners*（GPT-3）。ICL 的经典论文。
- **Xie et al. (2021)**，*An Explanation of In-Context Learning as Implicit Bayesian Inference*。ICL 的贝叶斯解释。
- **von Oswald et al. (2022)**，*Transformers Learn In-Context Learning by Implementing Learning Algorithms*。ICL 的机制研究。
- **Olsson et al. (2022)**，*In-Context Learning and Induction Heads*。ICL 的神经机制研究。

第 19 讲 课程学习：从低熵到高熵的信息排布

> **本讲问题**：为什么”从简单到困难”的课程安排能加速训练？信息论如何解释”先学容易样本再学难样本”的优越性？这与数据处理不等式有什么联系？为什么随机打乱数据有时反而不好？

1. 数学工具/预备知识

1.1 熵与样本难度

第 2 讲定义了熵：$H(X) = -\sum p(x) \log p(x)$，度量分布的不确定性。

简单样本（低熵）：模式清晰，$p(y|x)$ 分布尖锐——模型容易预测，损失低，梯度准确。

复杂样本（高熵）：模式模糊，$p(y|x)$ 分布平坦——模型难预测，损失高，梯度噪声大。

为什么简单样本的梯度准确：因为 $p(y|x)$ 分布尖锐，梯度方向明确——模型知道该往哪个方向调整。

为什么复杂样本的梯度噪声大：因为 $p(y|x)$ 分布平坦，梯度方向不明确——模型不知道该往哪个方向调整，容易被噪声带偏。

1.2 数据处理不等式回顾

第 3 讲推导了数据处理不等式：对马尔可夫链 $X \to Y \to Z$，$I(X;Z) \leq I(X;Y)$。

为什么课程学习和 DPI 有关：因为模型的每一层都是在”加工”输入。如果一开始就给高噪声输入，模型的前几层就会把噪声”放大”（因为梯度被噪声主导），后面的层更难纠正。课程学习让前几层先学到”干净”的特征，后面的层再逐步处理噪声。

2. 完整推导

2.1 什么是课程学习?

课程学习（Curriculum Learning）是指：训练时先给模型简单的样本，逐渐过渡到困难的样本。类比人类学习——先学加减法，再学微积分。

为什么这能加速训练：直觉上，简单样本的”梯度信号”更清晰（损失低但梯度方向准确），复杂样本的梯度信号更模糊（损失高但噪声多）。先学简单样本，模型先找到一个好的”起点”，再逐步调整到复杂样本的最优解。

2.2 信息论解释：低熵样本 vs 高熵样本

简单样本（低熵）：模式清晰，$p(y|x)$ 分布尖锐——模型容易预测，损失低，梯度准确。**复杂样本**（高熵）：模式模糊，$p(y|x)$ 分布平坦——模型难预测，损失高，梯度噪声大。

课程学习的好处：1. **先学低熵样本**：梯度信号清晰，模型快速收敛到一个好的区域。2. **再学高熵样本**：在好的区域附近微调，不会被噪声梯度带偏。

为什么随机打乱可能不好：如果一开始就遇到高熵样本，梯度噪声大，模型可能被带偏到损失景观的”坏区域”，之后很难爬出来。

2.3 损失景观的视角

损失景观可以想象成一个山地地形：- 简单样本的损失景观：光滑的碗状，梯度清晰。- 复杂样本的损失景观：崎岖的山地，有很多局部最优和噪声。

课程学习就像”先沿着缓坡走，再爬坡”：- 先学简单样本：沿着缓坡（光滑损失）走到半山腰。- 再学复杂样本：从半山腰开始爬坡（崎岖损失），不容易迷路。

为什么从复杂样本开始不好：一开始就在崎岖山地里，很容易走到一个小山谷（局部最优），出不来。

2.4 课程学习与 DPI 的联系

数据处理不等式说：信息在逐层加工中只会衰减。课程学习可以看作是”在输入空间中做信息排序”——先给”低噪声”的输入，再给”高噪声”的输入。

为什么这和 DPI 有关：因为模型的每一层都是在”加工”输入。如果一开始就给高噪声输入，模型的前几层就会把噪声”放大”（因为梯度被噪声主导），后面的层更难纠正。课程学习让前几层先学到”干净”的特征，后面的层再逐步处理噪声。

2.5 课程学习的具体实现

实现方式 1：按样本大小排序 - 先学小图像，再学大图像。- 先学短句子，再学长句子。

实现方式 2：按标签难度排序 - 先学简单类别（猫、狗），再学复杂类别（波斯猫、金毛犬）。- 先学简单任务（分类），再学复杂任务（生成）。

实现方式 3：按损失大小排序 - 先学模型已经能预测的样本（损失低）。- 再学模型预测不好的样本（损失高）。

为什么按损失排序是最优的：因为损失大小直接反映了样本的难度——损失低的样本简单，损失高的样本复杂。

2.6 课程学习的理论分析

为什么课程学习能加速收敛?

从优化理论的角度看：- 简单样本的损失景观是光滑的碗状——梯度清晰，收敛快。- 复杂样本的损失景观是崎岖的山地——梯度噪声大，收敛慢。

课程学习让模型先在简单样本上找到一个好的起点，再在复杂样本上微调。这比直接在复杂样本上训练收敛更快。

为什么随机打乱可能不好?

如果一开始就遇到复杂样本，梯度噪声大，模型可能被带偏到损失景观的”坏区域”（局部最优）。之后即使加入简单样本，模型也很难爬出来。

为什么课程学习不是所有任务都有效?

因为”难度排序”不一定和真实任务的难度一致。比如：- 图像分类：简单样本可能是”猫、狗”，复杂样本是”波斯猫、金毛犬”。但如果你的任务是”区分波斯猫和普通猫”，那”猫、狗”反而不是简单样本。- 语言模型：简单样本可能是短句子，但如果你的任务是长文本摘要，短句子反而不是简单样本。

2.7 课程学习与迁移学习的关系

课程学习和迁移学习有什么关系?

- **课程学习**：在同一个任务内，从简单样本过渡到复杂样本。
- **迁移学习**：从一个任务迁移到另一个任务。

联系：两者都是”从简单到复杂”的学习策略。课程学习是在样本空间做迁移，迁移学习是在任务空间做迁移。

区别：课程学习不需要额外的数据，只需要重新排序现有数据。迁移学习需要预训练模型和目标任务的数据。

2.8 课程学习的局限性

局限性 1：难度排序不一定准确 - “简单”和”复杂”是主观的——你觉得简单的样本，模型可能觉得复杂。- 比如图像的像素大小：小图像可能”简单”，但小图像的噪声比例更高，可能更难学。

局限性 2：课程学习不是所有任务都有效 - 在某些任务上（如图像分类），课程学习有效。- 在另一些任务上（如生成模型），课程学习可能没用，甚至有害。

局限性 3：课程学习需要额外的计算成本 - 你需要先计算每个样本的难度（如损失大小），然后排序。- 这增加了预处理的计算成本。

怎么判断课程学习是否有效：- 先在你的任务上做一个小实验：用课程学习 vs 随机打乱，看哪个收敛更快、效果更好。- 如果课程学习明显更好，就用；如果差不多，就不用。

2.9 课程学习的进阶话题

话题 1：自步学习（Self-Paced Learning） - 自步学习是课程学习的一种形式——模型自己决定”先学什么”。- 模型先学损失低的样本（简单样本），再学损失高的样本（复杂样本）。- 这比人工排序更灵活——模型自动调整学习顺序。

话题 2：课程学习和迁移学习的结合 - 先用简单任务预训练，再用复杂任务微调。- 这是”任务空间的课程学习”。- 比如：先学图像分类，再学图像生成。

话题 3：课程学习和元学习的结合 - 元学习（Meta-Learning）是”学会学习”。- 课程学习可以作为元学习的训练策略——模型学会”怎么安排学习顺序”。- 这是”学会怎么安排课程”。

2.10 课程学习的实践建议

实践 1：从简单样本开始 - 先用简单样本训练，让模型快速找到一个好的起点。- 然后逐渐加入复杂样本，让模型逐步优化。

实践 2：动态调整学习顺序 - 不要一开始就固定学习顺序——根据模型的表现动态调整。- 比如：如果模型在某个样本上表现好，就减少这个样本的权重；如果表现不好，就增加权重。

实践 3：和正则化结合 - 课程学习和正则化都是”防止过拟合”的方法。- 课程学习是从”数据顺序”的角度防止过拟合，正则化是从”模型复杂度”的角度防止过拟合。- 两者结合效果更好。

3. 直觉理解

直觉：课程学习就像”学车”。你先在空旷的停车场练倒车（简单样本），再去窄路练侧方停车（困难样本）。一上来就上高速公路（纯困难样本），你会慌——因为你连方向盘都没握稳。

简单样本的梯度信号就像”路灯”——照着你往前走。复杂样本的梯度信号就像”大雾”——你不知道往哪走。先有路灯照亮一段路，再进大雾，你就不会迷路。

随机打乱就像”一会儿在停车场一会儿在高速公路”——你刚找到感觉，又被吓到。课程学习是循序渐进，让你有时间适应。

类比：你学数学。先学 1+1=2（简单样本），再学微积分（困难样本）。一上来就学微积分（纯困难样本），你会崩溃——因为你连加减法都不会。

4. 常见误区

误区一：认为课程学习总是更好。错。在某些任务上（如图像分类），随机打乱可能更好——因为课程学习的”难度排序”不一定和真实任务的难度一致。

误区二：认为课程学习就是”先易后难”。错。课程学习有很多种形式——有的按样本大小排序，有的按标签难度排序，有的按损失大小排序。

误区三：认为课程学习对大模型没用。错。大模型的课程学习（如逐步增加上下文长度、逐步增加任务难度）仍然有效。

误区四：认为课程学习是监督学习独有的。错。自监督学习也可以用课程学习——比如先学高频词，再学低频词。

5. 代码实现

模拟课程学习 vs 随机打乱在简单回归任务上的效果。

```python
import numpy as np
import matplotlib.pyplot as plt

# ========== 模拟课程学习 vs 随机打乱 ==========
np.random.seed(42)

# 生成带噪声的数据
n = 200
x = np.linspace(-3, 3, n)
y_true = np.sin(x) + 0.3 * x
noise_level = np.abs(x) * 0.5 + 0.2  # 越靠边缘噪声越大
y = y_true + np.random.randn(n) * noise_level

# 简单线性回归模型（y = w1*x + w2）
def train_model(x_train, y_train, lr=0.01, n_steps=100):
    w1, w2 = 0.0, 0.0
    losses = []
    for _ in range(n_steps):
        pred = w1 * x_train + w2
        grad_w1 = np.mean((pred - y_train) * x_train)
        grad_w2 = np.mean(pred - y_train)
        w1 -= lr * grad_w1
        w2 -= lr * grad_w2
        loss = np.mean((pred - y_train)**2)
        losses.append(loss)
    return w1, w2, losses

# 实验 1：随机打乱
indices_random = np.random.permutation(n)
x_random = x[indices_random]
y_random = y[indices_random]

# 实验 2：课程学习（先简单（中间区域噪声小），后困难（边缘噪声大））
indices_curriculum = np.argsort(noise_level)  # 按噪声从小到大
x_curriculum = x[indices_curriculum]
y_curriculum = y[indices_curriculum]

# 分阶段训练
def train_curriculum(x, y, noise_levels, phases=4, lr=0.01, n_steps_per_phase=50):
    w1, w2 = 0.0, 0.0
    losses = []
    n_per_phase = n // phases
    for phase in range(phases):
        start = phase * n_per_phase
        end = (phase + 1) * n_per_phase if phase < phases - 1 else n
        x_phase = x[start:end]
        y_phase = y[start:end]
        for _ in range(n_steps_per_phase):
            pred = w1 * x_phase + w2
            grad_w1 = np.mean((pred - y_phase) * x_phase)
            grad_w2 = np.mean(pred - y_phase)
            w1 -= lr * grad_w1
            w2 -= lr * grad_w2
            loss = np.mean((w1 * x + w2 - y)**2)
            losses.append(loss)
    return w1, w2, losses

w_rand, b_rand, losses_rand = train_model(x_random, y_random, lr=0.01, n_steps=200)
w_curr, b_curr, losses_curr = train_curriculum(x_curriculum, y_curriculum, noise_level,
 ↪  lr=0.01, n_steps_per_phase=50)
```

```
# ========== 可视化 ==========
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].scatter(x, y, s=10, alpha=0.5, label='带噪声数据')
axes[0].plot(x, y_true, 'k--', label='真实函数')
axes[0].plot(x, w_rand * x + b_rand, 'r-', label='随机打乱')
axes[0].plot(x, w_curr * x + b_curr, 'b-', label='课程学习')
axes[0].set_xlabel('x')
axes[0].set_ylabel('y')
axes[0].set_title('课程学习 vs 随机打乱的拟合效果')
axes[0].legend(fontsize=8)
axes[0].grid(True, alpha=0.3)

axes[1].plot(losses_rand, 'r-', label='随机打乱', alpha=0.7)
axes[1].plot(losses_curr, 'b-', label='课程学习', alpha=0.7)
axes[1].set_xlabel('训练步数')
axes[1].set_ylabel('MSE 损失')
axes[1].set_title('训练损失曲线')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('ch19_curriculum.png', dpi=100)
print(" 图已保存为 ch19_curriculum.png")
```

6. 小结

1. 课程学习：先学简单样本（低熵、低噪声），再学困难样本（高熵、高噪声）。
2. 简单样本的梯度信号清晰，帮助模型快速收敛到好区域；复杂样本的梯度噪声大，容易把模型带偏。
3. 损失景观视角：先沿缓坡（光滑损失）走到半山腰，再爬坡（崎岖损失）。
4. 课程学习不是所有任务都有效——难度排序不一定和真实任务一致。
5. 大模型的课程学习（如逐步增加上下文长度）仍然有效。

7. 思考题

基础档（巩固概念）

1. 什么是课程学习？为什么它能加速训练？
2. 简单样本和复杂样本的熵有什么区别？
3. 为什么随机打乱有时反而不好？

进阶档（推导与分析）

4. 为什么课程学习的”难度排序”不一定和真实任务的难度一致？举一个反例。
5. 课程学习和早停有什么关系？如果课程学习太早停止（还没学复杂样本），会怎样？
6. 自监督学习的课程学习怎么做？比如语言模型先学高频词还是低频词？

开放档（联系研究）

7. 大模型的”Scaling Law”和课程学习有什么关系？大模型训练是怎么做课程学习的？
8. 为什么人类学习都是从简单到困难？这是进化的结果还是教育的发明？

8. 延伸阅读/进一步思考

- **Bengio et al. (2009)**，*Curriculum Learning*。课程学习的原始论文。

- **Weinshall et al. (2018)**，*Curriculum Learning by Instance Selection*。实例选择的课程学习。
- **Soviany et al. (2021)**，*Curriculum Learning: A Survey*。课程学习的综述。
- **Hacohen & Weinshall (2019)**，*On the Power of Curriculum Learning in Deep Learning*。课程学习的理论分析。

第 20 讲 注意力熵诊断

> **本讲问题**：Transformer 的注意力权重分布应该是尖锐的还是均匀的？如何用注意力熵来诊断模型的”注意力健康度”？低熵和高熵分别对应什么问题？注意力层的”信息瓶颈”效应是什么？

1. 数学工具/预备知识

- 注意力权重：$\alpha_{ij} = \text{softmax}(q_i \cdot k_j / \sqrt{d_k})$
- 注意力熵：$H(\alpha_i) = -\sum_j \alpha_{ij} \log \alpha_{ij}$
- 注意力的作用：加权聚合上下文信息。

1.1 注意力权重的两种极端

- **尖锐（低熵）**：注意力集中在少数几个 token 上。模型”聚焦”在最重要的信息上。
- **均匀（高熵）**：注意力分散在所有 token 上。模型”平均”了所有上下文。

理想状态：既不过度聚焦（只看一个 token），也不过度分散（看所有 token）。注意力熵应该在一个合适的范围。

2. 完整推导

2.1 注意力熵的计算

对于第 i 个 query token，它对所有 j 个 key token 的注意力权重为 α_{ij}。注意力熵：

$$H(\alpha_i) = -\sum_j \alpha_{ij} \log \alpha_{ij}$$

单位：如果 key token 数量为 n，最大熵是 $\log n$（均匀分布），最小熵是 0（完全集中）。

归一化熵：为了比较不同序列长度的注意力熵，通常用归一化熵：

$$H_{\text{norm}} = \frac{H(\alpha_i)}{\log n}$$

归一化熵的取值范围是 $[0, 1]$：- 0：完全集中（只关注 1 个 token）。- 1：完全均匀（所有 token 权重一样）。

2.2 低熵注意力（过度聚焦）

症状：注意力权重几乎全部集中在 1-2 个 token 上，其他 token 的注意力接近 0。

问题：1. **信息丢失**：只看少数几个 token，错过了上下文的其他重要信息。2. **过拟合**：模型记住了训练数据中的特定模式，不能泛化。3. **注意力坍塌**：多层之后，所有头都聚焦在同一个 token 上，没有多样性。

原因：- 训练初期，某些 token 的”重要性”信号太强，模型过度依赖它们。- 学习率太大，注意力权重变得过于尖锐。

怎么诊断：1. 画出注意力权重的热力图：如果大部分权重集中在少数几个 token 上，就是低熵。2. 计算归一化注意力熵：如果低于 0.3，就是低熵。3. 检查多头注意力的多样性：如果所有头的注意力模式都差不多，就是注意力坍塌。

2.3 高熵注意力（过度分散）

症状：注意力权重几乎均匀分布在所有 token 上，没有明显的焦点。

问题：1. **信息稀释**：所有 token 的贡献差不多，重要的 token 被淹没了。2. **梯度信号弱**：注意力分散导致梯度信号不明确，模型学得慢。3. **上下文窗口浪费**：长上下文窗口被均匀分配，没有利用”注意力”的优势。

原因：- 模型还没学会”哪些 token 重要”，注意力权重接近均匀。- 训练后期，模型”放弃”了区分重要性，直接平均。

怎么诊断：1. 画出注意力权重的热力图：如果所有 token 的权重差不多，就是高熵。2. 计算归一化注意力熵：如果高于 0.9，就是高熵。3. 检查注意力熵随训练的变化：如果熵一直很高不下降，就是模型没学会区分。

2.4 注意力的信息瓶颈效应

注意力层可以看作一个”信息瓶颈”：- **输入**：上下文的所有 token（n 个）。- **输出**：加权聚合后的一个向量（d_k 维）。

注意力熵越低，信息瓶颈越紧——越只保留少数 token 的信息。**注意力熵越高，信息瓶颈越松**——保留所有 token 的平均信息。

理想的注意力熵：既不能太低（丢信息），也不能太高（稀释信息）。应该根据任务动态调整——有的任务需要聚焦（如机器翻译），有的任务需要分散（如长文档摘要）。

不同层的注意力熵差异：- **低层**：关注语法（如”the”后面是名词），注意力熵适中——聚焦在几个相关 token 上。- **高层**：关注语义（如”这句话在说什么”），注意力熵更低——聚焦在关键 token 上。

为什么低层和高层的熵不同：因为低层处理的是局部模式（语法），只需要看几个 token；高层处理的是全局模式（语义），需要看更多 token 但只关注最重要的几个。

2.5 多头注意力的多样性

多头注意力（Multi-Head Attention）有多个注意力头，每个头关注不同的模式。

多样性怎么度量：- **头间注意力熵的方差**：如果所有头的注意力熵都差不多，多样性低；如果差异大，多样性高。- **头间注意力权重的相似度**：如果所有头的注意力模式都差不多，多样性低；如果差异大，多样性高。

为什么多样性重要：因为不同的头应该关注不同的模式——有的关注语法，有的关注语义，有的关注位置关系。如果所有头都关注同一个模式，多头就没有意义。

2.6 注意力熵的诊断流程

第一步：训练前 - 随机初始化的注意力权重应该接近均匀（高熵）。- 如果初始熵很低，说明初始化有问题。

第二步：训练中 - 监控每层每头的注意力熵。- 如果某层某头的熵突然降到 0，说明注意力坍塌了。- 如果某层的熵一直很高不下降，说明模型没学会区分。

第三步：训练后 - 检查不同层的注意力熵分布。- 理想状态：低层熵适中，高层熵更低（更聚焦）。- 如果所有层的熵都差不多，说明模型没有分层学习。

怎么用诊断结果调整模型：- 注意力坍塌：降低学习率，增加 dropout。- 注意力太分散：增加温度系数（让 softmax 更尖锐）。- 没有分层学习：调整网络结构（如增加层数）。

2.7 注意力熵与模型泛化的关系

注意力熵和泛化的关系：- 训练中注意力熵快速下降：模型快速聚焦，可能过拟合。- 训练中注意力熵缓慢下降：模型逐步学习，泛化更好。

为什么注意力熵下降太快不好：- 因为模型可能过早地”记住”了训练数据中的特定模式。- 它没有给”探索”留出足够的时间——错过了其他可能更好的模式。

怎么判断注意力熵下降是否合理：- 画出注意力熵随训练步数的曲线。- 理想状态：熵在前几轮快速下降（快速学习），然后缓慢下降（逐步优化）。- 如果熵在第一轮就降到 0，说明下降太快。

2.8 注意力熵的进阶话题

话题 1：稀疏注意力的熵诊断 - 稀疏注意力（如 Longformer）只关注局部窗口 + 少数全局 token。- 稀疏注意力的熵应该更低（因为只关注少数 token）。- 如果稀疏注意力的熵和全注意力一样高，说明稀疏注意力没有起作用。

话题 2：多头注意力的模式多样性 - 理想状态：不同的头关注不同的模式（语法、语义、位置关系）。- 如果所有头都关注同一个模式，说明多头没有意义。- 可以用头间注意力权重的相似度来度量多样性。

话题 3：注意力熵和上下文窗口长度的关系 - 上下文窗口越长，最大注意力熵越大（因为有更多 token 可以关注）。- 归一化注意力熵（除以 $\log n$）可以比较不同长度的序列。

2.9 注意力熵的实践建议

实践 1：监控注意力熵 - 在训练过程中，定期记录每层每头的注意力熵。- 如果熵突然降到 0，说明注意力坍塌了。- 如果熵一直很高，说明模型没学会区分。

实践 2：可视化注意力热力图 - 画出注意力权重的热力图，直观看到模型在关注什么。- 低熵的热力图应该是”尖峰”——只关注少数几个 token。- 高熵的热力图应该是”平坦”——所有 token 权重差不多。

实践 3：调整温度系数 - 如果注意力太分散（熵太高），可以降低温度系数，让 softmax 更尖锐。- 如果注意力太集中（熵太低），可以提高温度系数，让 softmax 更平滑。

注意力熵诊断的最终目标：让模型的注意力既不太分散也不太集中——恰到好处。

3. 直觉理解

直觉：注意力就像”你在图书馆里看书”。

低熵注意力 = 你只盯着书架上的一本书看（过度聚焦）。你可能错过了旁边那本书里的重要信息。

高熵注意力 = 你在所有书架上随便翻（过度分散）。你看了很多书，但每本都没看懂。

理想状态 = 你先扫一眼所有书架（高熵），找到最相关的几本（熵降低），然后仔细读（低熵）。

注意力熵诊断就像”检查你是不是在图书馆里瞎逛”——如果熵太高，你在瞎逛；如果熵太低，你在钻牛角尖。

4. 常见误区

误区一：认为注意力熵越低越好。错。太低意味着过度聚焦，丢信息。

误区二：认为注意力熵越高越好。错。太高意味着过度分散，信息稀释。

误区三：认为注意力熵是固定的。错。不同层、不同头的注意力熵差异很大——低层关注语法，高层关注语义。

误区四：认为注意力熵诊断只能用于训练。错。推理时也可以用注意力熵诊断模型的”健康度”。

5. 代码实现

模拟不同注意力熵的效果，可视化低熵、高熵、理想注意力的对比。

```
import numpy as np
import matplotlib.pyplot as plt

# ========== 模拟注意力权重 ==========
np.random.seed(42)

n_tokens = 10
token_labels = [f'token_{i}' for i in range(n_tokens)]

# 三种注意力分布
# 1. 低熵（过度聚焦）
```

```
alpha_low = np.zeros(n_tokens)
alpha_low[3] = 0.9
alpha_low += 0.01  # 加一点小值避免 log(0)
alpha_low /= alpha_low.sum()

# 2. 高熵（过度分散）
alpha_high = np.ones(n_tokens) / n_tokens

# 3. 理想（适中）
alpha_mid = np.exp(-np.arange(n_tokens)**2 / 5)  # 高斯分布
alpha_mid /= alpha_mid.sum()

# ========== 可视化 ==========
fig, axes = plt.subplots(3, 1, figsize=(10, 8))

for ax, alpha, label, color in [
    (axes[0], alpha_low, '低熵注意力', 'red'),
    (axes[1], alpha_mid, '适中注意力', 'green'),
    (axes[2], alpha_high, '高熵注意力', 'blue'),
]:
    H = -np.sum(alpha*np.log(alpha))
    ax.bar(range(n_tokens), alpha, color=color, alpha=0.7)
    ax.set_xticks(range(n_tokens))
    ax.set_xticklabels(token_labels, rotation=45, fontsize=8)
    ax.set_ylabel('注意力权重')
    ax.set_title(f'{label} (H = {H:.3f})')
    ax.grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.savefig('ch20_attention_entropy.png', dpi=100)
print(" 图已保存为 ch20_attention_entropy.png")

# ========== 计算各层注意力熵的变化 ==========
print("\n" + "=" * 60)
print(" 模拟：训练过程中注意力熵的变化")
print("=" * 60)

# 模拟训练过程：从高熵（随机）→ 适中（学习）→ 可能变低（过拟合）
steps = np.arange(100)
entropy_curve = 2.0 - 1.5 * (1 - np.exp(-steps / 20)) + 0.1 * np.sin(steps / 10)

plt.figure(figsize=(10, 4))
plt.plot(steps, entropy_curve, 'b-', linewidth=2)
plt.axhline(y=np.log(n_tokens), color='gray', linestyle='--', label=f'最大熵 = log({n_tokens}) =
 ↪  {np.log(n_tokens):.2f}')
plt.axhline(y=0.5, color='red', linestyle=':', label='低熵警戒线')
plt.xlabel('训练步数')
plt.ylabel('注意力熵')
plt.title('训练过程中注意力熵的变化')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('ch20_attention_entropy_training.png', dpi=100)
print(" 图已保存为 ch20_attention_entropy_training.png")
print("\n典型的注意力熵曲线：")
print("  训练初期：高熵（随机注意力）")
print("  训练中期：熵降低（学会聚焦）")
print("  训练后期：熵可能继续降低（过拟合）")
```

6. 小结

1. 注意力熵度量注意力权重分布的尖锐程度。
2. 低熵（过度聚焦）：丢信息、过拟合、注意力坍塌。
3. 高熵（过度分散）：信息稀释、梯度信号弱、上下文窗口浪费。
4. 注意力层是一个信息瓶颈——熵越低，瓶颈越紧，保留的信息越少。
5. 理想的注意力熵应该适中，根据任务动态调整。

7. 思考题

基础档（巩固概念）

1. 什么是注意力熵？它的取值范围是什么？
2. 低熵注意力和高熵注意力分别有什么问题？
3. 为什么说注意力层是一个信息瓶颈？

进阶档（推导与分析）

4. 为什么不同层的注意力熵不同？低层和高层的注意力熵哪个更低？
5. 多头注意力的”多样性”怎么度量？如果所有头的注意力熵都一样，会怎样？
6. 长上下文窗口的注意力熵应该更高还是更低？为什么？

开放档（联系研究）

7. 大模型的”注意力坍塌”问题（某些头只关注少数 token）怎么用注意力熵诊断？
8. 稀疏注意力（如 Longformer）的注意力熵和全注意力有什么不同？

8. 延伸阅读/进一步思考

- **Vaswani et al. (2017)**，*Attention Is All You Need*。Transformer 的原始论文。
- **Clark et al. (2019)**，*What Does BERT Look At?*。BERT 注意力模式的分析。
- **Michel et al. (2019)**，*Are 16 Heads Really Better than 1?*。注意力头的重要性分析。
- **Kobayashi et al. (2020)**，*Attention is Not All You Need: Pure Attention Loses Rank*。注意力的理论分析。

第 21 讲 读论文的信息论框架

> **本讲问题**：拿到一篇机器学习论文，如何快速识别它的”信息流假设”？为什么说大多数论文本质上都在做信息论意义上的某种权衡？用这个框架读论文能省下多少时间？

1. 数学工具/预备知识

1.1 信息流的三元组

任何机器学习模型都可以抽象为一个信息流三元组：**输入** X → **中间表征** Z → **输出** Y。

- **输入** X：原始数据（图像、文本、语音）。
- **中间表征** Z：模型内部的隐变量（编码器输出、注意力隐状态、瓶颈层）。
- **输出** Y：最终预测（分类标签、重构图像、下一个 token）。

为什么这个抽象有用：因为它把千变万化的模型统一到同一个坐标系下。不管你读的是 CNN、Transformer 还是扩散模型，你都可以问三个问题：1. X 是什么？Y 是什么？2. Z 是什么？它在压缩 X 还是在保留 Y？3. 压缩和保留之间的权衡系数是什么？

1.2 信息瓶颈的回顾

第 13 讲推导了信息瓶颈原理：

$$\min_{p(z|x)} I(X;Z) - \beta\, I(Z;Y)$$

其中：- $I(X;Z)$：表征 Z 中保留了多少关于输入 X 的信息。- $I(Z;Y)$：表征 Z 中保留了多少关于输出 Y 的信息。- β：权衡系数，控制压缩与保留的轻重。

1.3 率失真的回顾

第 7 讲推导了率失真函数：

$$R(D) = \min_{p(x'|x):\mathbb{E}[d(x,x')]\le D} I(X;X')$$

率失真和信息瓶颈的区别：率失真的”失真”是关于 X 本身的（重构），信息瓶颈的”失真”是关于 Y 的（判别）。

2. 完整推导：读论文的三步法

2.1 第一步：识别输入和输出

拿到一篇论文，首先问：- **输入 X 是什么**？图像？文本？语音？表格数据？- **输出 Y 是什么**？分类标签？重构图像？下一个 token？

为什么这很重要：因为输入和输出决定了信息流的起点和终点。如果你连模型在做什么任务都不知道，后面的分析都是空谈。

常见的输入输出对：- 图像分类：X = 图像，Y = 类别标签。- 图像生成：X = 随机噪声，Y = 生成的图像。- 语言模型：X = 前文，Y = 下一个 token。- 机器翻译：X = 源语言句子，Y = 目标语言句子。

2.2 第二步：识别信息流

识别了输入输出之后，问：- **中间表征 Z 是什么**？编码器输出？注意力隐状态？瓶颈层？- **Z 在做什么**？压缩 X？保留 Y？还是两者都做？

典型的信息流模式：

模式 1：重构型（率失真） - $X \to Z \to X'$，目标是重构 X。- 代表：自编码器、VAE。- $I(X;Z)$ 是”率”，重构误差是”失真”。

模式 2：判别型（信息瓶颈） - $X \to Z \to Y$，目标是预测 Y。- 代表：分类网络、对比学习。- $I(X;Z)$ 是”压缩”，$I(Z;Y)$ 是”保留”。

模式 3：生成型 - $Z \to X$，目标是从先验采样生成 X。- 代表：GAN、扩散模型。- Z 是隐变量，X 是生成的数据。

模式 4：序列型 - $X_1, X_2, \dots, X_t \to Z_t \to Y_t$，逐步处理序列。- 代表：RNN、Transformer。- Z_t 是第 t 步的隐状态，Y_t 是第 t 步的输出。

2.3 第三步：识别权衡

识别了信息流之后，问：- **权衡系数 β 是什么**？KL 系数？温度？正则化强度？- **这个权衡的最优值在哪里**？论文是怎么调的？

常见的权衡系数：

模型	权衡系数	作用
VAE	β（KL 系数）	控制压缩程度
对比学习	τ（温度）	控制区分难度
RLHF	β（KL 惩罚）	控制偏离参考模型的程度

模型	权衡系数	作用
正则化	λ（权重衰减）	控制模型复杂度

为什么这很重要：因为权衡系数是模型的”旋钮”——拧大拧小，模型的行为就变了。读懂了权衡系数，就读懂了模型的”性格”。

3. 直觉理解

直觉：读论文就像”看一幅地图”。

信息论框架就是”地图的图例”——输入是起点，输出是终点，中间表征是路径，权衡是路况。

你不需要记住每条路的细节，只需要知道：起点在哪、终点在哪、走哪条路、路况怎么样。

这个框架让你”一眼看懂”论文的核心——它在压缩什么、保留什么、权衡什么。

类比：你拿到一张陌生城市的地图，不需要记住每条街道的名字，只需要知道：我从哪来、要到哪去、走哪条路最快、路况怎么样。信息论框架就是那张”缩略图”。

4. 信息论框架的局限性

4.1 不是所有论文都能用信息流解读

工程类论文：- 比如新的优化器（AdamW、Lion）、新的数据增强方法、新的分布式训练策略。- 这些论文的核心不是信息流，而是工程实现。- 用信息论框架读这类论文，你会觉得”没什么好解读的”——因为它本来就不是信息流问题。

架构类论文：- 比如 CNN、Transformer、Swin Transformer。- 这些论文的核心是新的网络架构，不是信息流的改进。- 虽然你可以用信息流框架来解读，但你会发现”输入输出没变，中间表征没变，只是网络结构变了”。

数据类论文：- 比如新的数据集、新的评估指标。- 这些论文的核心是数据和评估，不是信息流。

4.2 信息论框架是”事后解释”，不是”预测工具”

- 信息论框架能帮你快速理解已经存在的模型，但不能预测什么样的新模型会有效。
- 它是一个”解释工具”，不是”设计工具”。

4.3 信息论框架是”简化”，不是”完整”

- 真实的模型比信息流三元组复杂得多——有注意力机制、残差连接、LayerNorm 等各种细节。
- 信息流框架只抓住了最核心的”压缩-保留-权衡”，忽略了很多细节。
- 所以它适合”快速理解”，不适合”深入复现”。

4.4 怎么用信息论框架和其他方法互补

- **信息论框架**：快速理解模型的核心思想。
- **数学推导**：深入理解模型的技术细节。
- **实验复现**：验证模型的实际效果。

三**者结合**：先用信息论框架快速理解，再用数学推导深入细节，最后用实验验证。这是最有效的读论文方法。

4.5 信息论框架的常见误用

误用 1：把框架当成”唯一”的解读方式 - 有些模型的核心不是信息流——比如新的优化器、新的网络架构。- 强行用信息流框架解读，会显得很牵强。

误用 2：只看信息流，不看技术细节 - 信息流框架告诉你”模型在做什么”，但不告诉你”怎么做的”。- 如果你只看信息流，不看技术细节，你会停留在”表面理解”。

误用 3：用框架来评判论文的好坏 - 信息流框架是一个”理解工具”，不是”评判工具”。- 有些论文虽然信息流简单，但技术贡献很大——比如 Adam 优化器。

4.6 信息论框架的未来发展

未来方向 1：自动化读论文 - 用大模型自动提取论文的信息流（输入、输出、中间表征、权衡）。- 这可以帮助研究者快速筛选大量论文。

未来方向 2：信息流的定量分析 - 不仅定性地描述信息流，还要定量地测量 $I(X;Z)$、$I(Z;Y)$。- 这可以帮助研究者更深入地理解模型。

未来方向 3：信息流指导的模型设计 - 用信息流框架来设计新模型——先定义信息流，再设计具体实现。- 这可以帮助研究者更系统地做创新。

信息论框架的最终价值：让你快速抓住论文的核心——它在压缩什么、保留什么、权衡什么。

> **误区**一：认为信息论框架能解释一切。错。有些论文的核心不是信息流，而是工程技巧（如优化器、数据增强）。
>
> **误区**二：认为所有论文都在做信息瓶颈。错。有些论文的核心是新的网络架构（如 CNN、Transformer），不是信息流。
>
> **误区**三：认为信息论框架是”事后诸葛亮”。错。它能帮你快速判断论文的贡献是什么——是新的压缩方式？新的保留目标？还是新的权衡方法？
>
> **误区四**：认为信息论框架是”唯一的”读论文方法。错。它只是一个工具——有时候用”模型架构”框架读论文更好，有时候用”优化方法”框架读论文更好。

5. 典型论文的信息论解读

5.1 VAE

- **输入** X：图像。
- **输出** Y：重构图像 $\hat{X}$。
- **中间表征** Z：隐变量（高斯分布）。
- **信息流**：$X \to Z \to \hat{X}$（重构型，率失真）。
- **权衡系数**：β（KL 系数），控制压缩程度。
- **核心贡献**：用变分推断把不可计算的后验转化为可优化的 ELBO。

5.2 对比学习（SimCLR）

- **输入** X：图像（两个增强视角）。
- **输出** Y：表征 Z（用于下游分类）。
- **中间表征** Z：投影后的表征。
- **信息流**：最大化 $I(X;Z')$（同视角互信息），最小化 $I(X;Z'')$（不同视角互信息）。
- **权衡系数**：温度 τ，控制区分难度。
- **核心贡献**：把互信息估计转化为对比分类问题（InfoNCE）。

5.3 大语言模型

- **输入** X：前文文本。
- **输出** Y：下一个 token。
- **中间表征** Z：Transformer 隐状态。
- **信息流**：$X_1, \ldots, X_t \to Z_t \to Y_t$（序列型）。
- **权衡系数**：上下文窗口大小、模型深度。
- **核心贡献**：缩放定律——损失随参数量和数据量幂律下降。

5.4 扩散模型

- **输入** X：带噪声的图像。

- **输出** Y：去噪后的图像。
- **中间表征** Z：噪声水平。
- **信息流**：$X \to Z \to \hat{X}$（逐步去噪）。
- **权衡系数**：去噪步数（步数多 = 质量好但慢）。
- **核心贡献**：把生成问题转化为逐步去噪问题，有解析的似然下界。

5.5 RLHF/DPO

- **输入** X：提示词。
- **输出** Y：模型回复。
- **中间表征** Z：微调后的模型参数。
- **信息流**：$X \to Z \to Y$（微调后的模型生成回复）。
- **权衡系数**：β（KL 惩罚强度）。
- **核心贡献**：把强化学习问题转化为监督学习问题（DPO）。

5.6 稀疏注意力（Longformer）

- **输入** X：长序列。
- **输出** Y：序列表示。
- **中间表征** Z：注意力输出。
- **信息流**：$X \to Z \to Y$（只关注局部 + 少数全局 token）。
- **权衡系数**：窗口大小（窗口大 = 关注更多上下文但慢）。
- **核心贡献**：把全注意力的 $O(n^2)$ 复杂度降到 $O(n)$。

6. 怎么用这个框架快速读论文?

6.1 5 分钟快速读

1. **读标题和摘要**（2 分钟）：识别输入输出。
2. **读引言的最后一段**（2 分钟）：识别核心贡献。
3. **读方法的第一个公式**（1 分钟）：识别信息流和权衡。

6.2 30 分钟深入读

1. **读标题和摘要**（5 分钟）：识别输入输出。
2. **读引言**（5 分钟）：识别问题和贡献。
3. **读方法的主公式**（10 分钟）：识别信息流和权衡。
4. **读实验的主表**（5 分钟）：识别关键结果。
5. **读结论**（5 分钟）：识别影响和未来方向。

6.3 为什么这个框架能省时间?

因为你不需要记住每个模型的细节——你只需要抓住它的”信息流假设”。就像你不需要记住每条路的名字，只需要知道起点、终点、路径、路况。

7. 小结

1. 信息论框架读论文：识别输入、输出、中间表征、权衡。
2. 大多数机器学习论文都可以用信息流 $X \to Z \to Y$ 来解读。
3. 典型的信息流模式：重构型（率失真）、判别型（信息瓶颈）、生成型、序列型。
4. 权衡系数是模型的”旋钮”——读懂了权衡系数，就读懂了模型的”性格”。
5. 这个框架帮你快速抓住核心、对比模型、找到创新点。
6. 5 分钟快速读法：读标题摘要 → 读引言最后一段 → 读方法主公式。
7. 30 分钟深入读法：标题摘要 → 引言 → 方法主公式 → 实验主表 → 结论。

8. 思考题

基础档（巩固概念）

1. 用信息论框架解读 VAE：输入、输出、中间表征、权衡分别是什么？
2. 用信息论框架解读对比学习：它在最大化什么、最小化什么？
3. 为什么信息论框架能帮你快速读论文？

进阶档（推导与分析）

4. 用信息论框架解读扩散模型：它的信息流是什么？权衡是什么？
5. 大语言模型的”上下文窗口”在信息论框架下是什么？是压缩还是保留？
6. 为什么有些论文不适合用信息论框架解读？举两个例子。

开放档（联系研究）

7. 你最近读的一篇论文，用信息论框架解读一下：它在压缩什么、保留什么、权衡什么？
8. 如果让你设计一个新模型，你会从信息流的哪个环节创新？压缩方式？保留目标？还是权衡方法？

9. 延伸阅读/进一步思考

- **Tishby, Pereira & Bialek (1999)**，*The Information Bottleneck Method*。信息瓶颈的原始论文。
- **Goldstein et al. (2024)**，*A Roadmap to Interpretable Machine Learning via Information Theory*。信息论在可解释性中的应用。
- **Shwartz-Ziv & Tishby (2017)**，*Opening the Black Box of Deep Neural Networks via Information*。信息平面的经典论文。
- **Bommasani et al. (2021)**，*On the Opportunities and Risks of Foundation Models*。大模型的全景图。

第 22 讲 找创新点方法论

> **本讲问题**：做研究时，如何从信息论的角度找到一个”可做、可证伪、有价值”的创新点？为什么”信息流缺口”是创新点的主要来源？怎么判断一个方向是”真问题”还是”伪需求”？

1. 数学工具/预备知识

1.1 信息流的三元组回顾

第 21 讲建立了信息流三元组：**输入** X → **中间表征** Z → **输出** Y。

- **压缩**：$I(X;Z)$ 减小——表征丢掉了输入的冗余信息。
- **保留**：$I(Z;Y)$ 增大——表征保留了输出的任务信息。
- **权衡**：β 系数控制压缩与保留的轻重。

1.2 创新点的三个标准

一个好的创新点必须满足三个标准：1. **可做**：你有数据、有算力、有方法能做出来。2. **可证伪**：你能设计实验证明你的方法比现有方法好（或不好）。3. **有价值**：你的方法在实际任务中有用，或者在理论上有贡献。

为什么这三个标准缺一不可：- 不可做：想法很好但实现不了——纸上谈兵。- 不可证伪：无法验证好坏——自说自话。- 没价值：做出来也没人用——自娱自乐。

2. 完整推导：信息流缺口的四种类型

2.1 类型 1：压缩方式不对

问题描述：现有模型用某种方式压缩 X，但压缩得不好——丢了有用信息或留了太多噪声。

创新方向：设计更好的压缩方式。

例子 1：VAE 的高斯先验 - 现有 VAE 用标准高斯 $\mathcal{N}(0, I)$ 作为先验。- 问题：高斯分布太简单，不能匹配复杂数据分布。- 创新：用归一化流（第 12 讲）做更复杂的先验分布。

例子 2：对比学习的负样本 - 现有对比学习用随机负样本。- 问题：随机负样本太简单，模型学不到难的区分。- 创新：用 hard negative mining——选最难的负样本。

怎么判断这是真问题还是伪需求：- 真问题：现有压缩方式导致了明显的问题（如 VAE 的 KL 消失、对比学习的模式崩溃）。- 伪需求：现有压缩方式没问题，你只是换了个分布形式。

2.2 类型 2：保留目标不对

问题描述：现有模型保留的是 X 本身（重构），但你想保留的是 Y（任务标签）。

创新方向：从重构目标转向判别目标。

例子 1：自编码器 → 信息瓶颈 - 自编码器：重构 X（$I(X;Z)$ 刚好够重构）。- 问题：重构不代表判别——能重构不代表能分类。- 创新：信息瓶颈（第 13 讲）——保留 $I(Z;Y)$，不保留 $I(X;Z)$。

例子 2：生成模型 → 判别模型 - 生成模型：建模 $p(x)$（数据分布）。- 判别模型：建模 $p(y|x)$（条件分布）。- 创新：从生成转向判别——用更少的参数达到更好的分类效果。

怎么判断这是真问题还是伪需求：- 真问题：现有保留目标导致了明显的问题（如自编码器的表征不适合分类）。- 伪需求：现有保留目标没问题，你只是换了个目标函数。

2.3 类型 3：权衡系数不对

问题描述：现有模型的 β 系数是固定的，但你想让它动态调整。

创新方向：自适应的权衡系数。

例子 1：β-VAE - 现有 VAE 的 $\beta = 1$（固定）。- 问题：β 固定不能适应不同任务的需要。- 创新：β-VAE——让 β 可调整，甚至自适应。

例子 2：课程学习的 β - 训练初期：β 小（先拟合，$I(Z;Y)$ 上升）。- 训练后期：β 大（后压缩，$I(X;Z)$ 下降）。- 创新：动态调整 β，模拟课程学习的效果。

怎么判断这是真问题还是伪需求：- 真问题：现有权衡系数导致了明显的问题（如 VAE 的 KL 消失、对比学习的温度敏感）。- 伪需求：现有权衡系数没问题，你只是换了个调参方式。

2.4 类型 4：信息流方向不对

问题描述：现有模型是 $X \to Z \to Y$（前向），但你想做反向或双向。

创新方向：改变信息流方向。

例子 1：自编码器 → 生成模型 - 自编码器：$X \to Z \to X$（前向重构）。- 生成模型：$Z \to X$（从先验采样生成）。- 创新：从重构转向生成——用隐变量生成新数据。

例子 2：单向模型 → 双向模型 - RNN：$X_1, \dots, X_t \to Z_t$（单向）。- BiRNN：$X_1, \dots, X_t \to Z_t^{\text{forward}}, Z_t^{\text{backward}}$（双向）。- 创新：从单向转向双向——捕捉更多上下文信息。

怎么判断这是真问题还是伪需求：- 真问题：现有信息流方向导致了明显的问题（如 RNN 不能捕捉后文信息）。- 伪需求：现有信息流方向没问题，你只是换了个方向。

3. 实际案例分析

3.1 案例 1：VAE 的 KL 消失问题

问题现象：VAE 训练时，KL 散度趋近于 0，模型退化成自编码器——隐变量没有被利用，生成效果差。

信息流分析：- $I(X;Z)$ 太小（KL 趋近于 0）：表征几乎不保留输入的信息。- $I(Z;Y)$ 也小：表征不保留重构信息。- 这是”压缩方式不对”——KL 项太强，把表征压成了常数。

创新方向：- β-VAE：降低 KL 系数 β，让表征有更多容量。- Free Bits：KL 项设一个下限，低于下限就不优化。- 归一化流先验：用更复杂的先验分布，让表征有更多结构。

为什么这是真问题：- 有实验证据：VAE 的 KL 消失是普遍现象。- 有理论解释：KL 项太强，把表征压成常数。- 有针对性：β 调整直接解决了压缩太强的问题。

3.2 案例 2：对比学习的模式崩溃

问题现象：对比学习训练时，模型只生成少数几种模式，其他模式完全丢失。

信息流分析：- $I(X;Z)$ 太大：表征几乎完整拷贝了输入。- $I(Z;Y)$ 虽然在训练集上大，但在测试集上因为模式崩溃而下降。- 这是”保留目标不对”——模型保留了输入的所有信息，包括噪声。

创新方向：- 增加负样本数量：让模型更难区分，必须学到更多模式。- 温度调整：调整温度系数，让区分难度适中。- 数据增强：增加更多样的增强方式，让模型学到更鲁棒的特征。

3.3 案例 3：大模型的上下文窗口限制

问题现象：大模型的上下文窗口有限（如 4K、8K），长文档处理不好。

信息流分析：- $I(X;Z)$ 太大：上下文窗口内的所有信息都被保留。- $I(Z;Y)$ 随着上下文长度增加而下降：因为模型容量有限，不能同时处理所有信息。- 这是”信息流方向不对”——模型把所有上下文都保留在隐状态里，但隐状态容量有限。

创新方向：- 稀疏注意力：只关注少数关键 token，不保留所有上下文。- 记忆机制：把不常用的信息存到外部记忆里，不占隐状态容量。- 分层处理：先处理局部上下文，再处理全局上下文。

5. 找创新点的常见陷阱

5.1 陷阱 1：为了创新而创新

表现：为了发论文而发明一个新方法，但这个方法并没有解决实际问题。

为什么是陷阱：- 你的方法可能在标准数据集上刷了点，但在实际任务中没用。- 审稿人一眼就能看出来——“这个方法解决了什么问题?”

怎么避免：- 先找到真问题，再设计方法。- 不要先设计方法，再找问题。

5.2 陷阱 2：增量式改进

表现：在现有方法上做很小的改动，提升一点点（如 0.5%）。

为什么是陷阱：- 审稿人会说”这是 incremental work”。- 你没有做出实质性的贡献。

怎么避免：- 找到现有方法的根本问题，而不是表面问题。- 你的改进应该有理论解释，不只是实验提升。

5.3 陷阱 3：过度复杂化

表现：你的方法很复杂，但效果和简单方法差不多。

为什么是陷阱：- 审稿人会说”为什么不用更简单的方法?”- 复杂度没有带来相应的收益。

怎么避免：- 先用简单方法试一下，如果效果差不多，就不要用复杂方法。- 你的复杂度应该有明确的理由（如更好的泛化、更快的训练）。

6. 怎么判断是真问题还是伪需求?

6.1 真问题的三个标准

1. **有实验证据**：这个问题在多个实验中都出现了，不是偶然的。
2. **有理论解释**：这个问题为什么会发生？有清晰的理论解释。
3. **有针对性**：你能设计一个方法，直接解决这个问题。

6.2 伪需求的三个特征

1. **只有一两个实验出现**：不是普遍现象。
2. **没有理论解释**：不知道为什么会发生。
3. **没有针对性**：你设计的方法只是”碰巧”解决了这个问题。

6.3 怎么验证你的问题是真问题?

1. **文献调研**：看有没有其他论文也提到了这个问题。
2. **复现实验**：在你的任务上复现这个问题，看是不是真的存在。
3. **消融实验**：去掉你的改进，看这个问题是不是又出现了。

6.4 创新点的评估标准

好的创新点：1. **有理论深度**：不是随便改改，而是有信息论的理论解释。2. **有实验证据**：在多个任务上都有效。3. **有实用价值**：能解决实际问题，不是为了发论文而发论文。

差的创新点：1. **没有理论深度**：只是”我试了一下，效果好一点”。2. **没有实验证据**：只在一个数据集上有效。3. **没有实用价值**：解决的是不存在的问题。

3.1 真问题的特征

1. **有实验证据**：现有方法在某个任务上确实表现不好（有基准测试的数字）。
2. **有理论解释**：这个不好的原因可以用信息流解释（压缩不对？保留不对？权衡不对?）。
3. **有针对性**：你的方法能针对性地解决这个问题（不是换了个网络架构）。

3.2 伪需求的特征

1. **没有实验证据**：现有方法表现不好，但你不知道为什么。
2. **没有理论解释**：这个不好的原因不是信息流（而是数据不够、算力不够、评估指标不对）。
3. **没有针对性**：你的方法不能针对性地解决问题（只是换了个网络架构，信息流没变）。

3.3 一个快速测试

问自己三个问题：1. **现有方法在哪个任务上表现不好?**（具体的数据集、具体的指标）2. **为什么表现不好?**（用信息流解释——压缩不对？保留不对？权衡不对?）3. **你的方法怎么解决这个问题?**（针对性地改进压缩/保留/权衡）

如果三个问题都能回答清楚，那就是真问题。如果有一个问题答不上来，那就是伪需求。

4. 直觉理解

直觉：找创新点就像”找路”。

信息流就是”地图”——输入是起点，输出是终点，中间表征是路径，权衡是路况。

创新点就是”地图上没画的路”——现有模型没走过的路。

你要找的是：这条路值得走吗（有价值)？你能走过去吗（可做)？你能证明这条路比现有路好吗（可证伪)？

伪需求就是”地图上画了一条路，但其实走不通”——看起来是创新，但实际做不出来或者没用。

类比：你在一个陌生城市找路。现有地图上画了很多路，但有些路是死胡同（伪需求），有些路是新修的但没画（创新点)。你要找到那些”值得走、能走通、比现有路好”的新路。

5. 常见误区

误区一：认为创新点一定要从信息论出发。错。很多创新点是工程技巧（新的优化器、新的数据增强），不是信息流。

误区二：认为”信息流缺口”是唯一的创新来源。错。很多创新点是新的任务、新的数据集、新的评估指标。

误区三：认为创新点一定要”颠覆”现有方法。错。大多数好的论文是”改进”现有方法——在信息流的某个环节做小改进。

误区四：认为”有价值”就是在排行榜上刷点。错。有价值包括理论贡献、工程贡献、应用贡献——不只是刷点。

6. 小结

1. 创新点的三个标准：可做、可证伪、有价值。
2. 信息流缺口的四种类型：压缩方式不对、保留目标不对、权衡系数不对、信息流方向不对。
3. 判断真问题 vs 伪需求：现有方法不好的原因能不能用信息流解释？你的方法能不能针对性解决?
4. 大多数好论文是”改进”现有方法——在信息流的某个环节做小改进。
5. 快速测试：现有方法在哪个任务上表现不好？为什么？你的方法怎么解决?

7. 思考题

基础档（巩固概念）

1. 创新点的三个标准是什么?
2. 信息流缺口的四种类型是什么?
3. 怎么判断一个方向是真问题还是伪需求?

进阶档（推导与分析）

4. 举一个”压缩方式不对”的创新点例子，并解释为什么现有模型的压缩方式不好。
5. 举一个”保留目标不对”的创新点例子，并解释为什么现有模型的保留目标不对。
6. 为什么大多数好论文是”改进”现有方法，而不是”颠覆”现有方法?

开放档（联系研究）

7. 你最近在做的研究，属于信息流缺口的哪种类型?
8. 如果让你在信息流的某个环节做创新，你会选哪个环节？为什么?

8. 延伸阅读/进一步思考

- **Tishby, Pereira & Bialek (1999)**，*The Information Bottleneck Method*。信息瓶颈的原始论文。
- **Kingma & Welling (2013)**，*Auto-Encoding Variational Bayes*。VAE 的原始论文。
- **Chen et al. (2020)**,*A Simple Framework for Contrastive Learning of Visual Representations*（SimCLR）。对比学习的经典论文。
- **Rafailov et al. (2023)**，*Direct Preference Optimization*。DPO 的原始论文。

第 23 讲 实验诊断工具箱：互信息估计与训练动力学观测

本讲问题：训练一个深度网络时，如何用信息论工具诊断”模型到底在学什么”？kNN 互信息估计器、KSG 估计器、有效秩、表征协方差谱——这些工具怎么组合成一套完整的训练体检流程?

1. 数学工具/预备知识

- 互信息估计：从样本数据估计 $I(X;Z)$。
- 有效秩：表征的”有效自由度”。
- 表征协方差谱：表征的方差分布。

1.1 为什么需要诊断工具箱?

训练一个深度网络时，你不知道模型在学什么——它是在学有用的特征，还是在过拟合噪声？诊断工具箱就是”体检”——帮你检查模型的”健康状况”。

2. 完整推导

2.1 kNN 互信息估计器（KSG）

KSG（Kraskov-Stögbauer-Grassberger）估计器是一种从样本数据估计连续变量互信息的方法。

核心思想:用最近邻距离估计概率密度。- 对于每个样本 x_i,找到它在 X 空间的第 k 个最近邻,距离为 ϵ_i。- 同样在 Z 空间找到第 k 个最近邻,距离为 δ_i。- 互信息估计:$I(X;Z) \approx \psi(k)+\psi(N)-\frac{1}{N}\sum_i[\psi(n_x(i))+\psi(n_z(i))]$。

优点：不需要假设分布形式，适用于高维数据。**缺点**：计算复杂度高（$O(N^2)$），对噪声敏感。

2.2 有效秩

有效秩（Effective Rank）度量表征的”有效自由度”:

$$\text{Effective Rank} = \exp\left(-\sum_i p_i \log p_i\right)$$

其中 $p_i = \lambda_i / \sum_j \lambda_j$，$\lambda_i$ 是表征协方差矩阵的特征值。

直觉：有效秩是表征协方差谱的熵——它告诉你表征的方差分布有多”分散”。- 有效秩大：表征的方差分布均匀（很多维度都有用）。- 有效秩小：表征的方差分布集中（只有少数维度有用）。

2.3 表征协方差谱

表征协方差矩阵的特征值分布可以告诉你：- 前几个特征值大：表征的主要方差集中在少数维度（低秩）。- 所有特征值差不多：表征的方差分布均匀（满秩）。

为什么这很重要：如果表征的协方差谱在训练过程中逐渐集中（有效秩下降），说明模型在”压缩”表征——这是信息瓶颈的表现。

2.4 完整的诊断流程

第一步：训练前 - 随机初始化的表征应该是满秩的（有效秩接近维度数）。- 如果有效秩很低，说明初始化有问题。

第二步：训练中 - 监控 $I(X;Z)$ 和 $I(Z;Y)$ 的变化。- 监控有效秩的变化。- 监控表征协方差谱的变化。

第三步：训练后 - 检查不同层的 $I(X;Z)$ 和 $I(Z;Y)$。- 理想状态：浅层 $I(X;Z)$ 大（保留输入信息），深层 $I(Z;Y)$ 大（保留任务信息）。- 如果所有层的 $I(X;Z)$ 都差不多，说明模型没有分层压缩。

怎么用诊断结果调整模型：- $I(X;Z)$ 太大：增加正则化（权重衰减、dropout）。- $I(Z;Y)$ 太小：减少正则化，增加模型容量。- 有效秩太低：减少正则化，增加模型容量。- 有效秩太高：增加正则化，减少模型容量。

2.5 诊断工具箱的局限性

局限性 1：高维数据的 MI 估计不准 - kNN 估计器在高维数据中偏差很大（维度灾难）。- 通常需要先降维（PCA、t-SNE）再估计。

局限性 2：有效秩依赖于表征的定义 - 不同层的表征维度不同，有效秩不能直接比较。- 需要归一化（如用”有效秩/维度数”）。

局限性 3：诊断结果是启发式的 - 这些指标告诉你”模型在学什么”，但不一定告诉你”怎么改进”。- 改进需要结合实验和直觉。

2.6 诊断工具箱的进阶话题

话题 1：跨层的信息流分析 - 不仅要分析单层的 $I(X;Z)$ 和 $I(Z;Y)$，还要分析跨层的信息流。- 比如：第 1 层到第 10 层，$I(X;Z)$ 怎么变化？$I(Z;Y)$ 怎么变化？ - 这能告诉你网络是怎么逐层”压缩输入、保留输出”的。

话题 2：条件互信息的诊断 - 不仅要分析 $I(X;Z)$，还要分析 $I(X;Z|Y)$。- $I(X;Z|Y)$ 告诉你：在给定标签 Y 的条件下，表征 Z 还保留了多少输入 X 的信息。- 如果 $I(X;Z|Y)$ 很大，说明表征保留了很多”和标签无关”的信息——这可能是冗余的。

话题 3：表征的线性可分性 - 表征的线性可分性可以用”线性探针”来度量——用一个线性分类器在表征上训练，看准确率。- 如果线性可分性高，说明表征已经”很好地组织”了类别信息。- 如果线性可分性低，说明表征还需要进一步处理。

2.7 诊断工具箱的实践建议

实践 1：定期监控关键指标 - 在训练过程中，定期记录 $I(X;Z)$、$I(Z;Y)$、有效秩。- 如果某个指标突然变化，说明模型出了问题。

实践 2：对比不同模型的诊断结果 - 把你的模型的诊断结果和 SOTA 模型对比。- 如果你的模型的 $I(X;Z)$ 比 SOTA 大，说明你的模型保留了太多冗余信息。

实践 3：用诊断结果指导改进 - 如果 $I(Z;Y)$ 太小：增加模型容量，减少正则化。- 如果 $I(X;Z)$ 太大：增加正则化，减少模型容量。- 如果有效秩太低：增加正则化强度，让表征更”丰满”。

3. 直觉理解

直觉：诊断工具箱就像”体检”。

互信息估计器就是”量血压”——测量 $I(X;Z)$ 和 $I(Z;Y)$，看看模型有没有学到有用的信息。

有效秩就是”量体温”——看看表征的自由度有没有异常变化。

表征协方差谱就是”抽血化验”——看看表征的方差分布正不正常。

这些工具组合起来，就能给模型做一个”全面体检”——告诉你模型在学什么、学得好不好。

4. 常见误区

误区一：认为互信息估计是精确的。错。KSG 估计器有偏差和方差，尤其是在高维数据中。

误区二：认为有效秩越大越好。错。有效秩大意味着表征太”分散”——很多维度可能是噪声。

误区三：认为诊断工具箱是”事后”的。错。你可以在训练过程中实时监控这些指标，及时调整超参数。

误区四：认为诊断工具箱只适用于图像任务。错。它适用于任何有表征的任务——文本、语音、视频。

5. 代码实现

实现一个简化版的 KSG 互信息估计器，并计算有效秩。

```
import numpy as np
import matplotlib.pyplot as plt

# ========== kNN 互信息估计器（简化版 KSG） ==========
def knn_mutual_information(X, Z, k=5):
    """
    简化版 kNN 互信息估计器。

    X: (N, d_x) 输入数据
    Z: (N, d_z) 表征数据
    k: 最近邻数量
    """
```

```
    N = len(X)

    # 计算 X 空间的最近邻距离
    def avg_neighbor_dist(data, k):
        n = len(data)
        dists = np.zeros(n)
        for i in range(n):
            # 计算到所有其他点的距离
            diff = data - data[i]
            distances = np.sqrt(np.sum(diff**2, axis=1))
            # 排除自己（距离为 0）
            distances[i] = np.inf
            # 找第 k 个最近邻
            nearest_k = np.sort(distances)[k-1]
            dists[i] = nearest_k
        return dists

    dx = avg_neighbor_dist(X, k)
    dz = avg_neighbor_dist(Z, k)

    # 简化的 KSG 估计
    # 完整 KSG 用 digamma 函数，这里用近似
    mi = np.mean(np.log(dx) + np.log(dz)) - np.mean(np.log(dx * dz))
    # 加一个常数项（简化）
    mi += np.log(k) - np.log(N)

    return mi

# ========== 实验：不同相关性的互信息估计 ==========
print("=" * 60)
print(" 实验：kNN 互信息估计")
print("=" * 60)

np.random.seed(42)
N = 500

# 测试不同相关性
correlations = [0.0, 0.3, 0.6, 0.9]

for rho in correlations:
    # 生成二维高斯数据，相关系数 rho
    mean = [0, 0]
    cov = [[1, rho], [rho, 1]]
    data = np.random.multivariate_normal(mean, cov, N)
    X = data[:, 0:1]
    Z = data[:, 1:2]

    # 理论互信息（二元高斯）
    I_true = -0.5 * np.log2(1 - rho**2)

    # 估计互信息
    I_est = knn_mutual_information(X, Z, k=5)

    print(f" 相关系数 ρ = {rho:.1f}: 真实 MI = {I_true:.4f}, 估计 MI = {I_est:.4f}")

# ========== 有效秩计算 ==========
print("\n" + "=" * 60)
print(" 实验：有效秩计算")
print("=" * 60)
```

```
# 模拟不同训练阶段的表征
n_dims = 10
N = 1000

# 阶段 1: 随机表征（满秩）
Z_random = np.random.randn(N, n_dims)

# 阶段 2: 训练后的表征（低秩）
Z_trained = np.random.randn(N, 3) @ np.random.randn(3, n_dims) + 0.1 * np.random.randn(N,
 ↪  n_dims)

def effective_rank(Z):
    """ 计算表征的有效秩。"""
    cov = np.cov(Z.T)
    eigenvalues = np.linalg.eigvalsh(cov)
    eigenvalues = np.maximum(eigenvalues, 0)  # 避免负数
    total = eigenvalues.sum()
    if total == 0:
        return 0
    p = eigenvalues / total
    p = p[p > 1e-10]
    entropy = -np.sum(p * np.log(p))
    return np.exp(entropy)

er_random = effective_rank(Z_random)
er_trained = effective_rank(Z_trained)

print(f" 随机表征的有效秩 = {er_random:.2f}")
print(f" 训练后表征的有效秩 = {er_trained:.2f}")
print(" 结论: 训练后有效秩下降（表征被压缩）")

# ========== 可视化协方差谱 ==========
cov_random = np.cov(Z_random.T)
cov_trained = np.cov(Z_trained.T)

eig_random = np.sort(np.linalg.eigvalsh(cov_random))[::-1]
eig_trained = np.sort(np.linalg.eigvalsh(cov_trained))[::-1]

plt.figure(figsize=(8, 4))
plt.bar(range(n_dims), eig_random / eig_random.sum(), alpha=0.5, label='随机表征')
plt.bar(range(n_dims), eig_trained / eig_trained.sum(), alpha=0.5, label='训练后表征')
plt.xlabel('特征值索引')
plt.ylabel('归一化特征值')
plt.title('表征协方差谱对比')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('ch23_effective_rank.png', dpi=100)
print("\n图已保存为 ch23_effective_rank.png")
```

工程注意事项：上面的 kNN 估计器是简化版。完整的 KSG 估计器用 digamma 函数，而且有两种变体（用 X 空间的邻居数或 Z 空间的邻居数）。在高维数据中，kNN 估计器的偏差会很大——通常需要降维后再估计。

6. 小结

1. kNN 互信息估计器（KSG）用最近邻距离估计连续变量的互信息，不需要假设分布形式。
2. 有效秩度量表征的”有效自由度”——表征协方差谱的熵。
3. 表征协方差谱告诉你表征的方差分布——是满秩还是低秩。
4. 这些工具组合起来，就能给模型做”全面体检”——看看模型在学什么、学得好不好。

5. 诊断工具箱可以在训练过程中实时监控，及时调整超参数。

7. 思考题

基础档（巩固概念）

1. KSG 估计器的核心思想是什么?
2. 有效秩的定义是什么?
3. 表征协方差谱告诉你什么?

进阶档（推导与分析）

4. 为什么 kNN 互信息估计器在高维数据中偏差会很大?（提示：维度灾难）
5. 如果有效秩在训练过程中突然下降，可能是什么原因?
6. 为什么诊断工具箱可以在训练过程中实时监控？这对超参数调整有什么帮助?

开放档（联系研究）

7. 你最近训练的模型，用诊断工具箱检查过吗？它的 $I(X;Z)$、$I(Z;Y)$、有效秩分别是多少?
8. 大语言模型的诊断工具箱和图像模型的有什么不同？语言模型的表征是序列的，怎么估计互信息?

8. 延伸阅读/进一步思考

- **Kraskov et al. (2004)**，*Estimating Mutual Information*。KSG 估计器的原始论文。
- **Koch et al. (2021)**，*Effective Rank of Neural Network Representations*。有效秩的应用。
- **Amico et al. (2021)**，*Fishing for Robustness in Deep Learning*。诊断工具箱的综述。
- **Goldstein et al. (2024)**，*A Roadmap to Interpretable Machine Learning via Information Theory*。信息论诊断的最新进展。

第 24 讲 论文写作的信息论范式

> **本讲问题**：写一篇 ML 论文本质上是在做什么？为什么说”写作 = 编码，阅读 = 解码”？如何用信息论范式重构论文的叙事结构，让审稿人更容易理解你的贡献？为什么”最短描述”原则也适用于论文写作?

1. 数学工具/预备知识

1.1 编码与解码的信息论回顾

第 5 讲推导了信源编码定理：无损压缩的极限是熵 $H(X)$。

- **编码**：把信息压缩成最短的码字——用最少的比特传递最多的内容。
- **解码**：从码字中恢复信息——读者从你的文字中理解你的贡献。
- **交叉熵**：用一种语言描述另一种语言的代价——你的文字和读者的预期之间的差异。

1.2 为什么论文写作也是信息论问题?

论文写作的本质是：把你的研究成果（信息）“编码”成文字，让读者（审稿人）能”解码”并理解你的贡献。

好的论文写作 = 低交叉熵：你的文字和读者的预期之间的差异最小——读者不需要猜你想说什么，直接就能看懂。

差的论文写作 = 高交叉熵：你的文字和读者的预期之间差异很大——读者要猜半天才能理解你的意思。

2. 完整推导：论文的信息论结构

2.1 论文作为一个通信系统

一篇 ML 论文的信息结构可以类比为一个”通信系统”：

- **信源**：你的研究成果（新方法、新实验、新发现）。
- **编码器**：你写的论文（标题、摘要、方法、实验）。
- **信道**：审稿人的阅读过程（可能有噪声——审稿人没时间、不了解背景）。
- **解码器**：审稿人理解你的贡献。
- **噪声**：审稿人的困惑、误解、疲劳。

好的论文写作 = 用最短的文字传递最多的信息——就像好的压缩算法用最少的比特传递最多的内容。

2.2 最短描述原则在论文写作中的应用

MDL 原理（第 8 讲）说：最优的描述是最短的描述。论文写作也是一样：

不要废话：每一句话都应该传递新信息。重复的、显而易见的、无关的内容都是”冗余”。

- 冗余的例子：“深度学习在近年来取得了巨大进展……”（读者都知道）。
- 精简的例子：直接说”我们的方法在 ImageNet 上比 SOTA 提升了 2.3%“（新信息）。

结构清晰：论文的结构应该让读者能快速找到他需要的信息——就像好的代码有清晰的注释。

- 好的结构：摘要（一句话说清楚贡献）→ 引言（详细说清楚贡献）→ 方法（技术细节）→ 实验（验证结果）→ 结论（影响）。
- 差的结构：读者要翻半天才能找到你的核心贡献。

重点突出：你的核心贡献应该在摘要、引言、结论中反复出现——让读者不管从哪里开始读，都能抓住核心。

- 为什么要反复出现？因为审稿人可能只读摘要和引言，不读方法和实验。如果你的核心贡献只在方法部分，审稿人可能看不到。

2.3 论文叙事的信息瓶颈

好的论文叙事应该是一个”信息瓶颈”：

- **压缩**：把复杂的技术细节压缩成清晰的直觉（$I(X;Z)$ 小——丢掉不重要的细节）。
- **保留**：保留核心的贡献和创新点（$I(Z;Y)$ 大——让读者理解你的贡献）。

为什么这很重要：审稿人没有时间读你的所有细节。你要把最重要的信息”压缩”到摘要和引言里，让审稿人在 5 分钟内就能判断你的论文好不好。

压缩的例子：- 复杂的数学推导 → 一句直觉解释。- 大量的实验细节 → 一张关键图表。- 冗长的相关工作 → 一段对比表格。

保留的例子：- 你的核心创新点是什么？（一句话）- 它比现有方法好在哪里？（一张表）- 为什么它有效？（一段直觉解释）

2.4 审稿人视角的”解码”

审稿人读论文时，他的”解码过程”是：

1. **读标题和摘要**（2 分钟）：快速判断论文主题。
2. **读引言**（5 分钟）：快速判断论文贡献。
3. **读方法的主公式**（5 分钟）：快速判断论文的技术合理性。
4. **读实验的主表**（5 分钟）：快速判断论文的实验充分性。
5. **读结论**（2 分钟）：快速判断论文的影响。

好的论文写作 = 让审稿人的每一步”解码”都很顺利——不需要猜、不需要翻、不需要困惑。

为什么审稿人只有 20 分钟？ - 因为审稿人可能要审 10 篇论文，每篇只有 20 分钟。- 如果你的论文在 20 分钟内没让审稿人看懂你的贡献，他可能直接拒稿。

3. 论文各部分的信息论优化

3.1 标题：最短的编码

标题是论文的”第一个码字”——它必须用最少的字传递最多的信息。

好的标题：包含核心方法和核心任务。- 例子：“Direct Preference Optimization: Your Language Model is Secretly a Reward Model”- 为什么好：包含方法（DPO）、核心洞察（LM 是奖励模型）。

差的标题：太泛或太模糊。- 例子：“A New Approach to Deep Learning”- 为什么差：读者不知道你在做什么。

标题写作的三个原则：1. **具体**：不要用”A New Method for…“，要说具体方法名。2. **有洞察**：标题里要有一个让人眼前一亮的洞察。3. **简洁**：不超过 15 个词。

3.2 摘要：最高密度的编码

摘要是论文的”压缩包”——它必须用 150-200 字传递论文的全部核心信息。

好的摘要结构：1. 问题（1 句）：现有方法有什么问题？2. 方法（2 句）：你提出了什么方法？3. 结果（2 句）：你的方法在什么任务上提升了多少？4. 影响（1 句）：你的方法有什么意义？

为什么这很重要：因为大多数读者只读摘要——如果摘要没说清楚，他们不会读全文。

摘要写作的常见错误：- 太长：超过 200 字，读者不耐烦。- 太泛：“我们提出了一种新方法…”——什么方法？- 没有数字：“我们的方法效果更好”——好多少？

3.3 引言：最关键的解码

引言是论文的”说明书”——它必须让审稿人在 5 分钟内理解你的贡献。

好的引言结构：1. **第一段**：问题背景（1 段）。2. **第二段**：现有方法的不足（1 段）。3. **第三段**：你的方法和核心贡献（1 段）。4. **第四段**：实验结果总结（1 段）。5. **最后一段**：论文结构（1 段）。

为什么最后一段要写论文结构？ - 因为审稿人可能跳着读——告诉他”第二部分是方法，第三部分是实验”，他就能快速找到他想看的部分。

引言写作的常见错误：- 背景太长：花了 3 段讲背景，没有讲你的方法。- 贡献不明确：没有用 bullet points 列出核心贡献。- 没有实验结果：引言里没有说你的方法提升了多少。

3.4 方法：技术细节的编码

方法部分是论文的”代码”——它必须让懂技术的读者能复现你的结果。

好的方法结构：1. **问题定义**（1 节）：你要解决什么问题？用什么数学形式？2. **方法推导**（1-2 节）：你的方法是怎么推导出来的？3. **实现细节**（1 节）：你的方法是怎么实现的？

为什么要先问题定义？ - 因为读者可能不熟悉你的任务——你要先把问题定义清楚，再讲你的方法。

方法写作的常见错误：- 跳过推导：直接给公式，不解释怎么来的。- 没有直觉：只有公式，没有直觉解释。- 实现细节太少：别人没法复现。

3.5 实验：结果的编码

实验部分是论文的”证据”——它必须让审稿人相信你的方法确实有效。

好的实验结构：1. **主表**（1 张）：你的方法 vs 所有 SOTA 方法。2. **消融实验**（1-2 张）：每个组件的贡献。3. **可视化**（1-2 张）：定性结果。

为什么主表最重要？ - 因为审稿人可能只看主表——如果主表显示你的方法比 SOTA 好，他就相信你的方法有效。

实验写作的常见错误：- 只有主表，没有消融实验：读者不知道哪个组件起作用。- 没有可视化：读者看不到定性结果。- 没有统计显著性：不知道提升是不是偶然。

3.6 结论：最短的总结

结论是论文的”结尾”——它必须让审稿人在 2 分钟内记住你的贡献。

好的结论结构：1. **一句话总结**：你的方法是什么？2. **关键结果**：你的方法提升了多少？3. **未来方向**：你的方法还有什么可以改进的？

为什么结论重要：因为审稿人可能只看摘要和结论——如果结论没说清楚，他可能忘了你的贡献。

结论写作的常见错误：- 重复摘要：结论应该是”升华”，不是重复。- 没有未来方向：审稿人想知道你的工作可以怎么扩展。- 太长：结论应该短——半页就够了。

4. 论文写作的信息论原则

4.1 最短描述长度原则

论文写作的目标是什么？是”用最少的字数传递最多的信息”。

这就是最短描述长度（MDL）原则——在所有可能的表述方式中，选最短、最清晰的。

为什么这很重要：因为审稿人的时间有限——你用越短的字数把问题说清楚，他越容易理解你的贡献。

4.2 冗余信息的代价

论文中的冗余信息（如重复的背景介绍、不必要的铺垫）有什么代价？

- **增加审稿人的认知负担**：他要读更多的字，才能找到核心信息。
- **稀释你的贡献**：审稿人可能淹没在冗余信息中，忘了你的核心贡献。

怎么减少冗余：- 每一段只讲一个核心观点。- 删掉所有”虽然…但是…“的废话。- 用 bullet points，不用长段落。

4.3 信息密度

论文的信息密度 = 核心信息量 / 总字数。

高信息密度的论文：- 每一段都有新信息。- 每一句话都有意义。- 审稿人读完后，记住了你的核心贡献。

低信息密度的论文：- 很多段落是重复的背景介绍。- 很多句子是”众所周知…“的废话。- 审稿人读完后，忘了你的核心贡献。

怎么提高信息密度：- 先写核心贡献（3-5 个 bullet points），再围绕它们展开。- 删掉所有不直接支持核心贡献的段落。- 用数字和公式，不用模糊的形容词。

论文写作的最终目标：让审稿人用最少的时间，理解你的核心贡献。

4.4 不同期刊的信息密度要求

- **顶会论文**：信息密度最高——审稿人时间有限，必须快速抓住核心。
- **期刊论文**：信息密度可以低一些——可以更详细地展开推导。
- **技术报告**：信息密度最低——可以把所有细节都写进去。

为什么顶会要求高信息密度：因为顶会的审稿人要审很多论文，没有时间读长文。你必须在最短的篇幅内把核心贡献说清楚。

4. 直觉理解

直觉：论文写作就像”寄快递”。

你的研究成果是”包裹”（信息）。你写的论文是”包装”（编码）。审稿人是”收件人”（解码）。

好的包装 = 用最少的材料（最短的文字）保护最重要的东西（核心贡献），让收件人一眼就能打开。

差的包装 = 用一堆废话把包裹包得严严实实，收件人半天打不开，打开了也不知道里面是什么。

最短描述原则就是”最好的包装”——不多不少，刚好让收件人理解你的贡献。

类比：你寄一个生日礼物给朋友。好的包装 = 漂亮的盒子 + 清晰的标签 + 打开方式说明。差的包装 = 一堆报纸 + 没有标签 + 不知道从哪里打开。

5. 常见误区

误区一：认为论文越长越好。错。好的论文是”最短的能说清楚的论文”——不是越长越显得工作量大。

误区二：认为论文的核心贡献在方法部分。错。核心贡献应该在摘要和引言里——审稿人可能只看这两部分。

误区三：认为论文写作是”最后一步”。错。论文写作应该在研究开始时就规划——你的叙事结构决定了你的研究方向。

误区四：认为审稿人会仔细读你的所有细节。错。审稿人可能只有 10-30 分钟读你的论文。你要把最重要的信息”压缩”到前几页。

6. 小结

1. 论文写作的本质是信息论问题——把研究成果编码成文字，让审稿人解码。
2. 好的论文写作 = 低交叉熵——你的文字和审稿人的预期之间的差异最小。
3. 最短描述原则：每一句话都应该传递新信息，不要废话。
4. 论文叙事的信息瓶颈：压缩技术细节，保留核心贡献。
5. 审稿人视角：让审稿人的每一步解码都很顺利。
6. 各部分的优化：标题（最短编码）、摘要（最高密度）、引言（最关键解码）、方法（技术细节）、实验（结果证据）。

7. 思考题

基础档（巩固概念）

1. 为什么论文写作也是信息论问题?
2. 什么是”最短描述原则”在论文写作中的应用?
3. 为什么说论文叙事应该是一个”信息瓶颈”?

进阶档（推导与分析）

4. 你的论文摘要能不能用一句话说清楚核心贡献？如果不能，问题出在哪里?
5. 为什么审稿人可能只看摘要和引言？这对论文写作有什么启示?
6. 为什么论文写作应该在研究开始时就规划，而不是最后才写?

开放档（联系研究）

7. 你最近读的一篇论文，用信息论范式分析一下：它的编码好不好？解码顺不顺利?
8. 如果让你用”最短描述原则”重写你的论文摘要，你会怎么写?

8. 延伸阅读/进一步思考

- **Rissanen (1978)**，*Modeling by Shortest Data Description*。MDL 原理的原始论文。
- **Knuth (1984)**，*Mathematical Writing*。数学写作的经典指南。
- **Strunk & White (1999)**，*The Elements of Style*。写作的经典指南。
- **Hutter (2006)**，*The Human Knowledge Compression Prize*。压缩即智能的思想。
